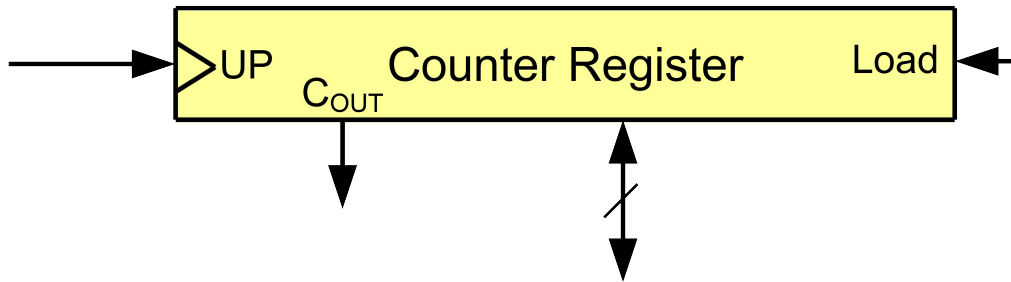


Timer/counter

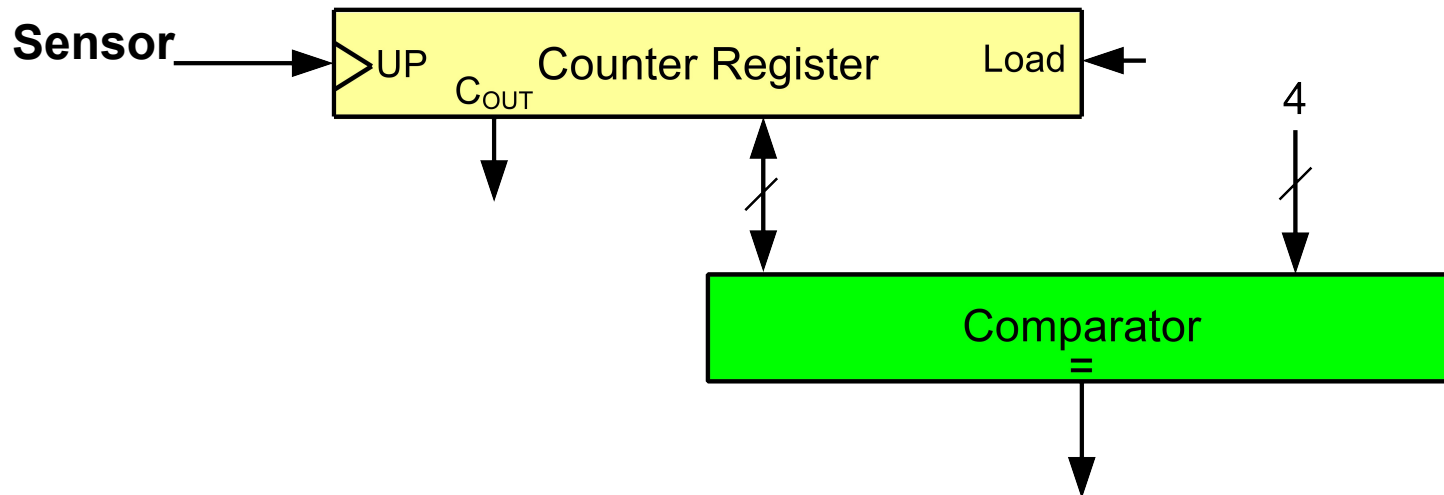
Chapter 9

A counter register



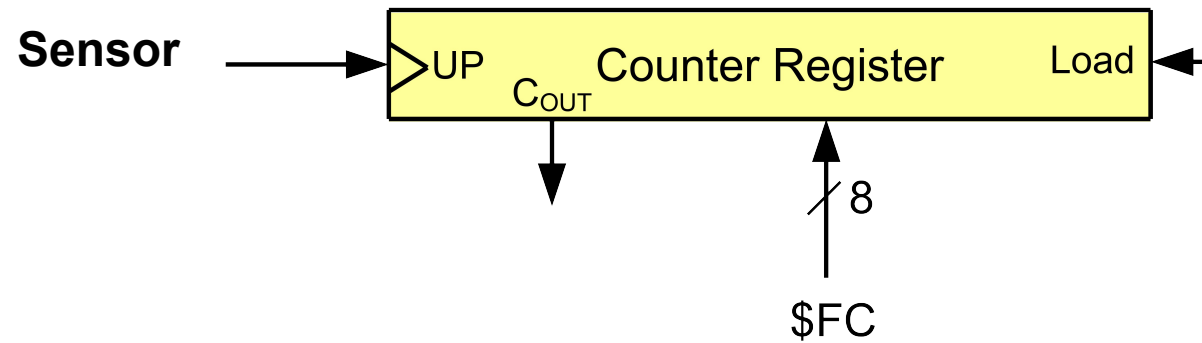
A simple design (counting people)

First design

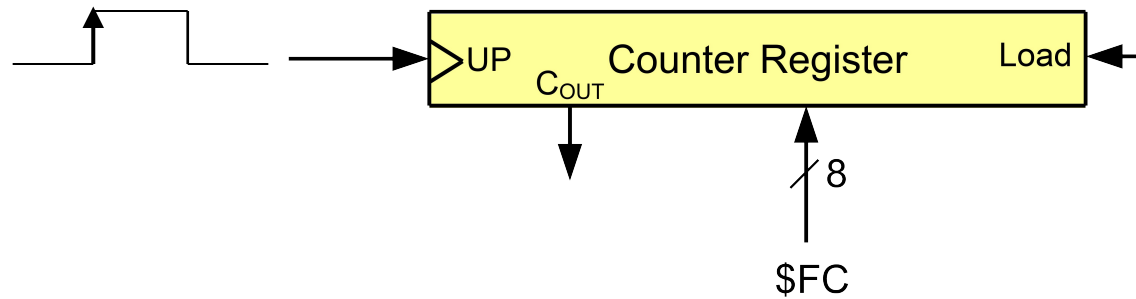


A simple design (counting people)

Second design

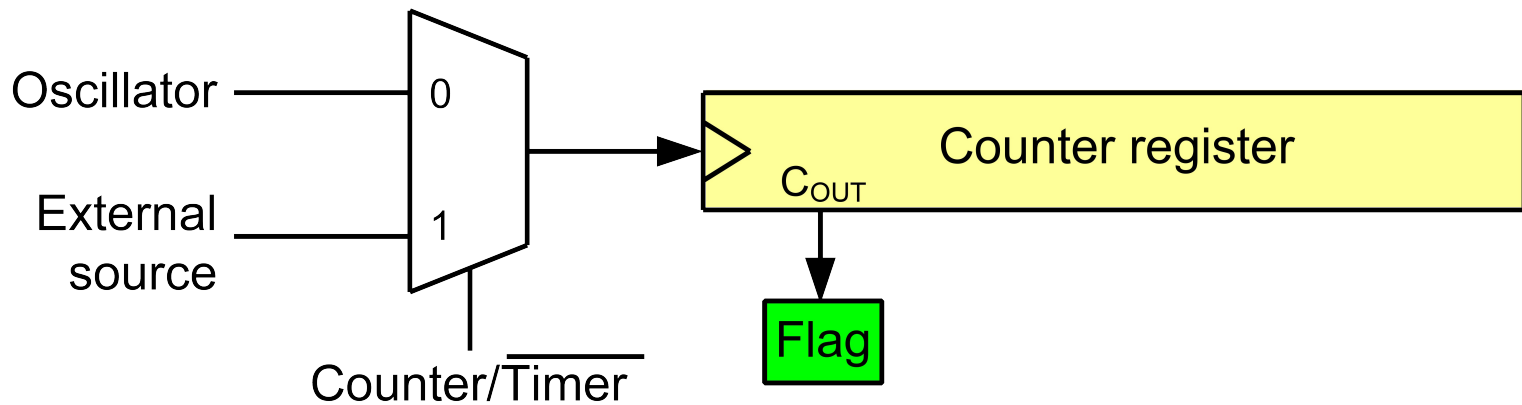


A simple design (making delay)



A generic timer/counter

- Delay generating
- Counting
- Wave-form generating
- Capturing

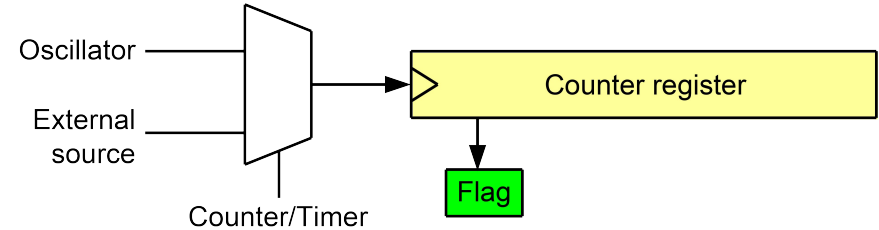


Timers in AVR

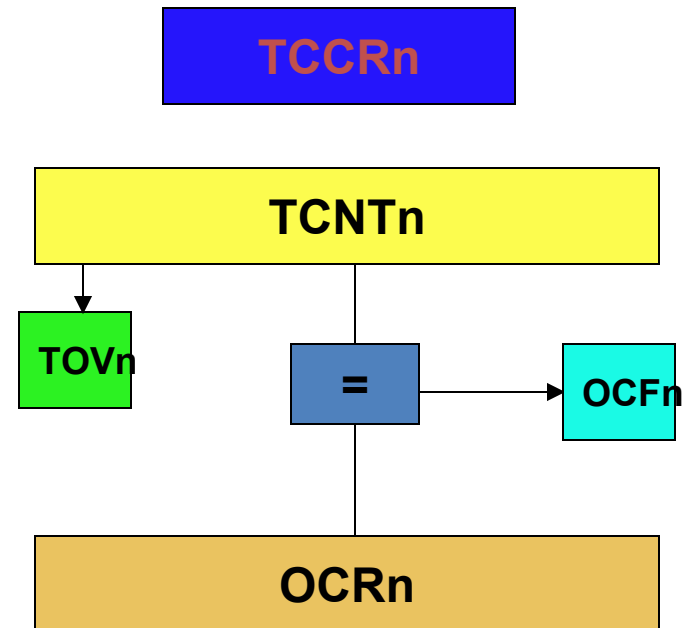
- 1 to 6 timers
 - 3 timers in ATmega32
- 8-bit and 16-bit timers
 - two 8-bit timers and one 16-bit timer in ATmega32

Timer in AVR

- TCNTn (Timer/Counter register)
- TOVn (Timer Overflow flag)
- TCCRn (Timer Counter control register)
- OCRn (output compare register)
- OCFn (output compare match flag)

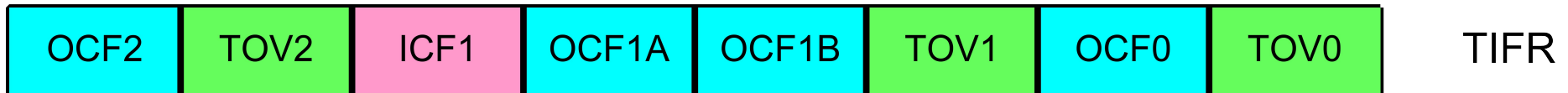
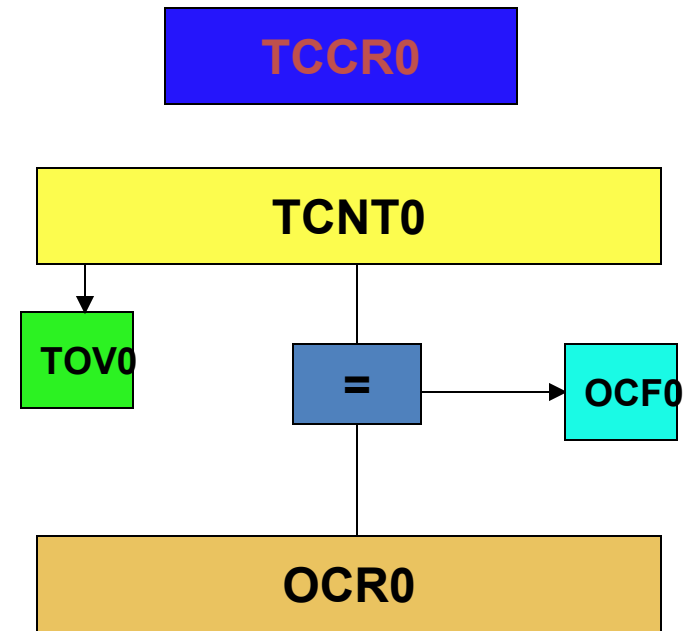


Comment:
All of the timer registers are
byte-addressable I/O registers



Timer 0 (an 8-bit timer)

Timer 0



Timer/Counter Interrupt Mask Register – TIMSK

Bit	7	6	5	4	3	2	1	0	
	OCIE2	TOIE2	TICIE1	OCIE1A	OCIE1B	TOIE1	OCIE0	TOIE0	TIMSK
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Timer/Counter Interrupt Flag Register – TIFR

Bit	7	6	5	4	3	2	1	0	
	OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0	TIFR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Three main registers in diagram, plus two more, to interact with this particular timer!
 - TCCR controls the timer functioning, waveform generation mode (PWM, etc), and clock prescaling! TCNT contains the current clock value, which can be reset and changed at will. Remember that this is incremented by the pre-scaled clock as configured by the TCCR register!
 - OCR contains the target clock value at which time an interrupt should be generated or a pin state should be changed!
 - A fourth register, ASSR, which is not shown on the functional diagram is used on this particular timer to select between the internal and external clocks, as well as indicate when the timer is busy!
 - TIMSK (Timer Interrupt MaSK), also not shown, is used to enable interrupts on specific timer events!

Timer/Counter Control Register – TCCR0

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Timer/Counter Register – TCNT0

Bit	7	6	5	4	3	2	1	0	
	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Output Compare Register – OCR0

Bit	7	6	5	4	3	2	1	0	
	OCR0[7:0]								OCR0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Timer/Counter Registers

■ Timer/Counter Control Register TCCR0

FOC: Force output compare

- When writing a logical one to a FOC bit, an immediate compare match is forced on the waveform generation unit. The OC output is changed according to the corresponding COM bits setting.
- In other words, writing 1 to a FOC bit causes an artificial match event on that channel.



FOC: Force output compare – force a known state on the OC pin – usually to initialize.

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 7 – FOC0: Force Output Compare

The FOC0 bit is only active when the WGM00 bit specifies a non-PWM mode. However, for ensuring compatibility with future devices, this bit must be set to zero when TCCR0 is written when operating in PWM mode. When writing a logical one to the FOC0 bit, an immediate compare match is forced on the Waveform Generation unit. The OC0 output is changed according to its COM01:0 bits setting. Note that the FOC0 bit is implemented as a strobe. Therefore it is the value present in the COM01:0 bits that determines the effect of the forced compare.

A FOC0 strobe will not generate any interrupt, nor will it clear the timer in CTC mode using OCR0 as TOP.

The FOC0 bit is always read as zero.

• Bit 6, 3 – WGM01:0: Waveform Generation Mode

These bits control the counting sequence of the counter, the source for the maximum (TOP) counter value, and what type of Waveform Generation to be used. Modes of operation supported by the Timer/Counter unit are: Normal mode, Clear Timer on Compare Match (CTC) mode, and two types of Pulse Width Modulation (PWM) modes. See Table 38 and "Modes of Operation" on page 74.

Table 38. Waveform Generation Mode Bit Description⁽¹⁾

Mode	WGM01 (CTC0)	WGM00 (PWM0)	Timer/Counter Mode of Operation	TOP	Update of OCR0	TOV0 Flag Set-on
0	0	0	Normal	0xFF	Immediate	MAX
1	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	1	0	CTC	OCR0	Immediate	MAX
3	1	1	Fast PWM	0xFF	TOP	MAX

Note: 1. The CTC0 and PWM0 bit definition names are now obsolete. Use the WGM01:0 definitions. However, the functionality and location of these bits are compatible with previous versions of the timer.

Timer/Counter Registers (cont'd)

■ Timer/Counter Control Register TCCR0

- **Bit 5:4 – COM01:0: Compare Match Output Mode**

These bits control the Output Compare pin (OC0) behavior. If one or both of the COM01:0 bits are set, the OC0 output overrides the normal port functionality of the I/O pin it is connected to. However, note that the Data Direction Register (DDR) bit corresponding to the OC0 pin must be set in order to enable the output driver.

When OC0 is connected to the pin, the function of the COM01:0 bits depends on the WGM01:0 bit setting. Table 39 shows the COM01:0 bit functionality when the WGM01:0 bits are set to a normal or CTC mode (non-PWM).

Table 39. Compare Output Mode, non-PWM Mode

COM01	COM00	Description
0	0	Normal port operation, OC0 disconnected.
0	1	Toggle OC0 on compare match
1	0	Clear OC0 on compare match
1	1	Set OC0 on compare match

Timer/Counter Registers (cont'd)

■ Timer/Counter Control Register TCCR0

- Bit 2:0 – CS02:0: Clock Select

The three Clock Select bits select the clock source to be used by the Timer/Counter.

Table 42. Clock Select Bit Description

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$\text{clk}_{\text{I/O}}$ /(No prescaling)
0	1	0	$\text{clk}_{\text{I/O}}/8$ (From prescaler)
0	1	1	$\text{clk}_{\text{I/O}}/64$ (From prescaler)
1	0	0	$\text{clk}_{\text{I/O}}/256$ (From prescaler)
1	0	1	$\text{clk}_{\text{I/O}}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

If external pin modes are used for the Timer/Counter0, transitions on the T0 pin will clock the counter even if the pin is configured as an output. This feature allows software control of the counting.

Setting Register Values

- Defined names for each register and bit
 - Set timer to clear on match
 - Set prescaler to 1024

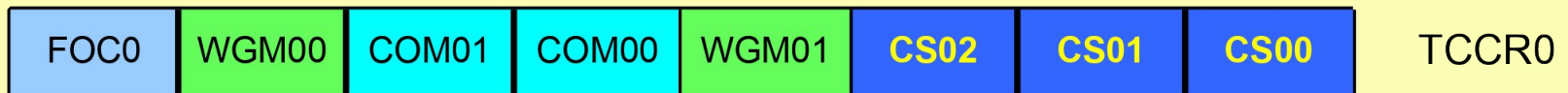
```
TCCR0 = (1<<WGM01) | (1<<CS02) | (1<<CS00);
```

- Set count value to compare against

```
OCR0 = 150;
```

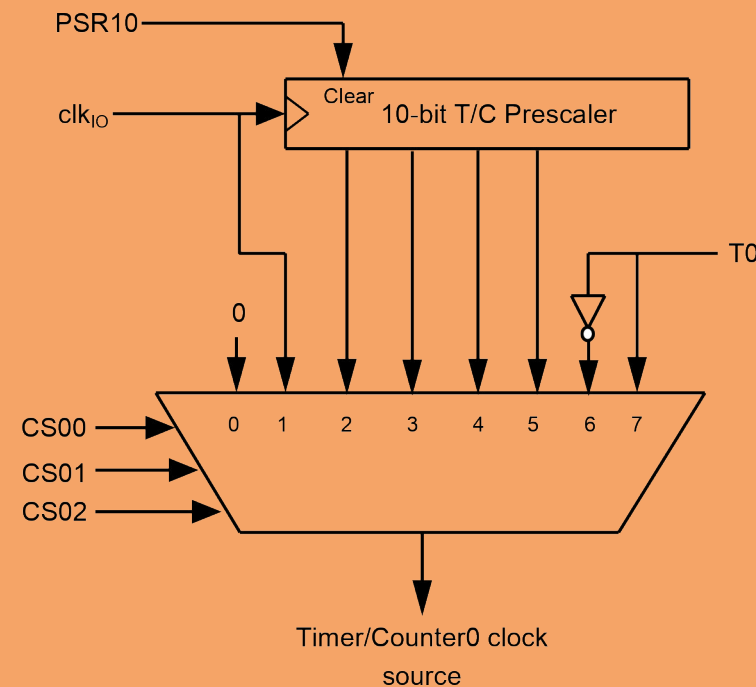
- Set timer to interrupt when it reaches count

```
TIMSK = (1<<OCIE0);
```

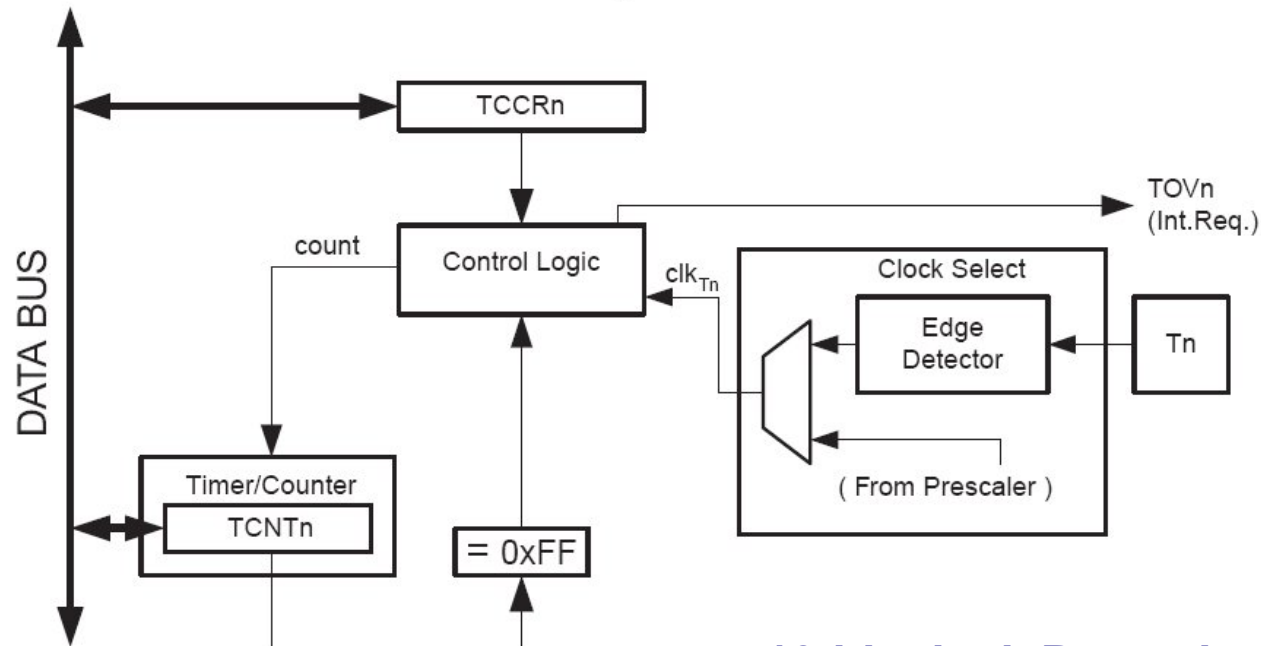



Timer Mode (WGM)

WGM00	WGM01	Comment
0	0	Normal
0	1	CTC (Clear Timer on Compare Match)
1	0	PWM, phase correct
1	1	Fast PWM



8-bit Timer / Counter0



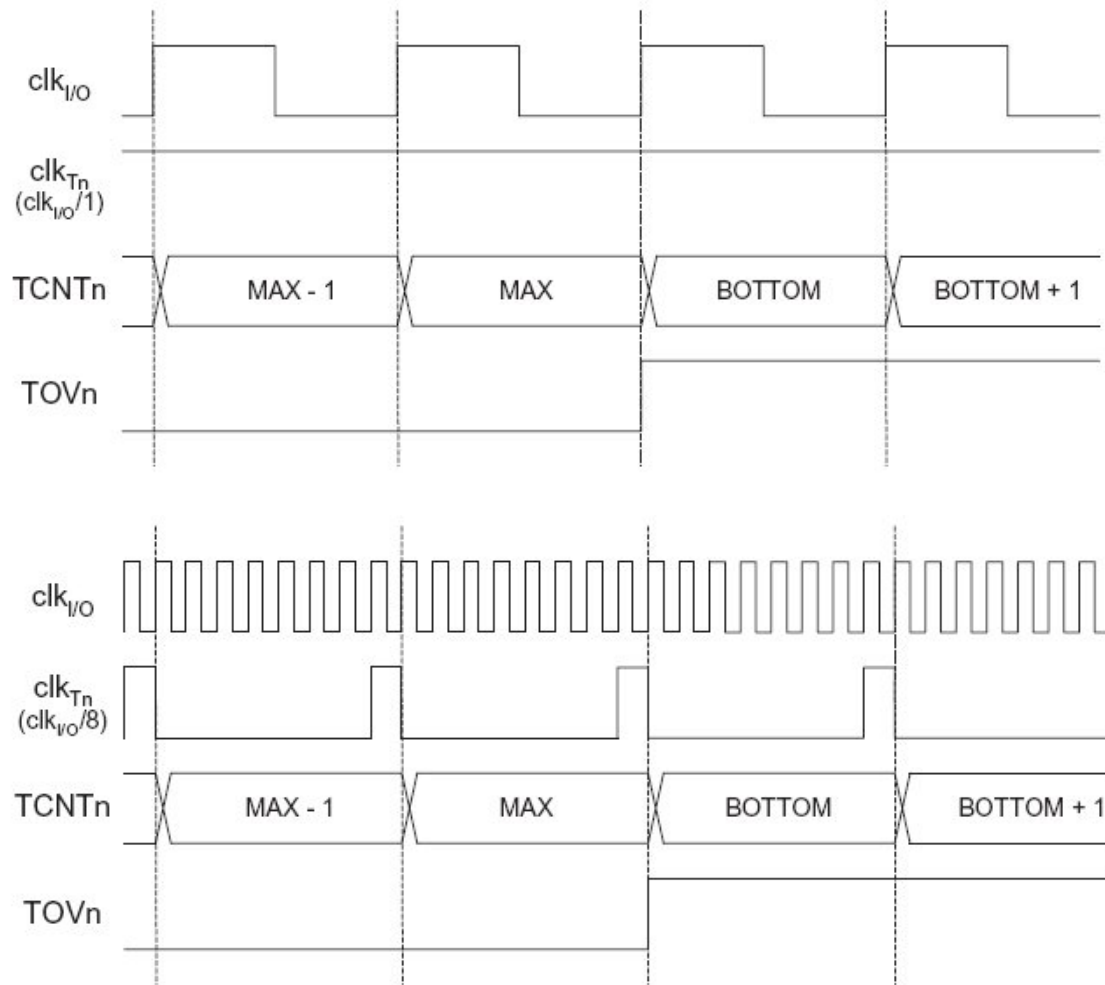
- 10-bit clock Prescaler
timer-clk (t0) = $\text{clk (IO)} / \text{prescaler}$
- External clock source T0 connected to PD4
cannot be prescaled, $\text{clk(ext)} \leq \text{clk (IO)} / 2.5$

8-bit Timer / Counter0 - prescaler operation

No prescaler

MAX=0xff
BOTTOM=0

Prescaler = 8



8-bit Timer / Counter0 usage

Timer/Counter0 Control Register (TCCR0), Bits CS02-CS00 select Clock Source and Prescaler Value :

Bit	7	6	5	4	3	2	1	0	
	–	–	–	–	–	CS02	CS01	CS00	TCCR0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$\text{clk}_{I/O}/(\text{No prescaling})$
0	1	0	$\text{clk}_{I/O}/8$ (From prescaler)
0	1	1	$\text{clk}_{I/O}/64$ (From prescaler)
1	0	0	$\text{clk}_{I/O}/256$ (From prescaler)
1	0	1	$\text{clk}_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

8-bit Timer / Counter0 usage

Timer/Counter0 Register (TCNT0) :
read/write, incremented per CLK cycle, overflow: 0xff

Bit	7	6	5	4	3	2	1	0	
	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- A Reload-Value is used to fine-tune the interrupt interval
- write Reload-Value to TCNT0 in the ISR

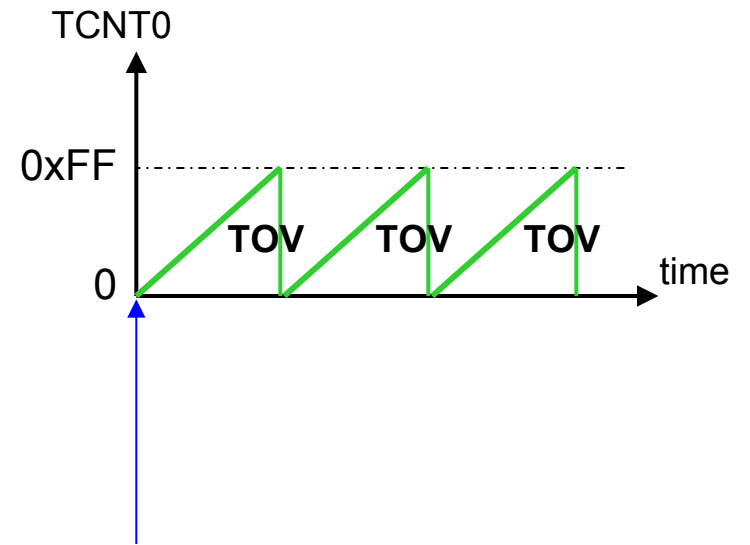
Special Function I/O Register – SFIOR

Register – SFIOR

7

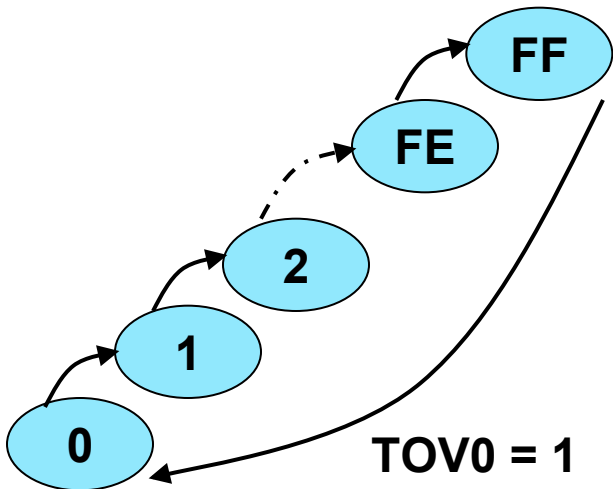
[illegible]

Normal mode



TOV0:

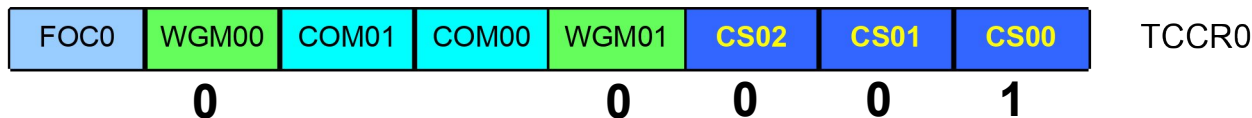
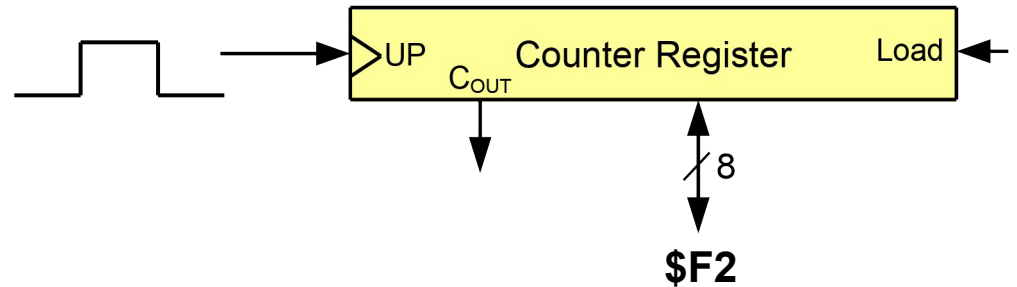
1



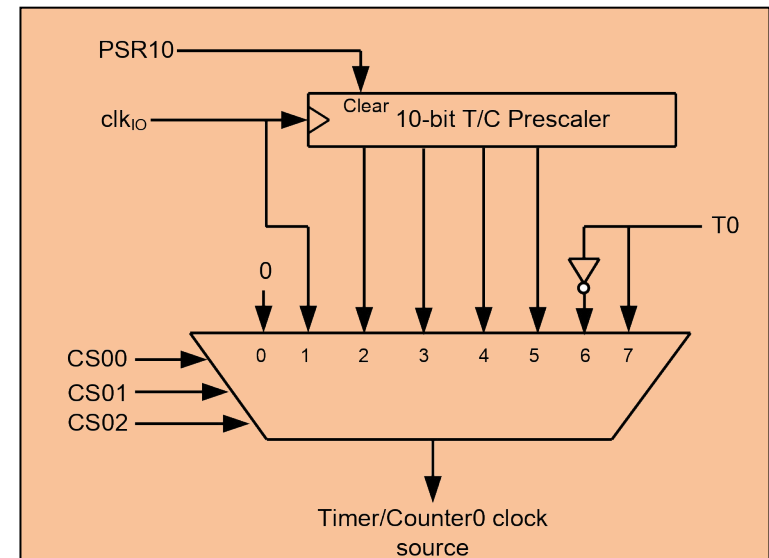
Example 1: Write a program that waits 14 machine cycles in Normal mode.

14 = \$0E

\$100
-\$0E
\$F2



WGM00	WGM01	Comment
0	0	Normal
0	1	CTC
1	0	PWM, phase correct
1	1	Fast PWM

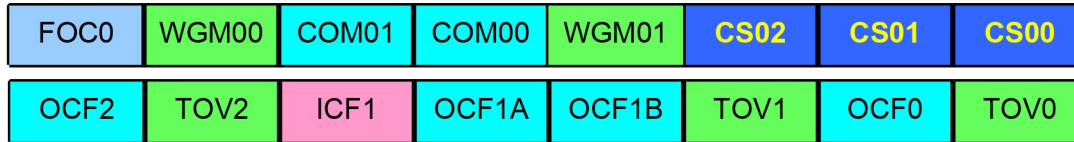


Example 1: write a program that waits 14 machine cycles in Normal mode.

\$100

-\$0E

\$F2



TCCR0

TIFR

```
.INCLUDE "M32DEF.INC"
```

```
LDI R16,0x20
SBI DDRB,5 ;PB5 as an output
LDI R17,0
OUT PORTB,R17
```

```
BEGIN: LDI R20,0xF2
OUT TCNT0,R20
LDI R20,0x01
OUT TCCR0,R20 ;Ti
AGAIN: IN R20,TIFR
SBRS R20,TOV0 ;if
```

```
inst. RJMP AGAIN
LDI R20,0x0
OUT TCCR0,R20
LDI R20,(1<<TOV0)
OUT TIFR,R20
```

```
EOR R17,R16 ;toggle D5 of R17
```

```
OUT PORTB,R17 ;toggle PB5
```

```
DDRB = 1<<5;
PORTB &= ~(1<<5); //PB5=0
while (1)
{
    TCNT0 = 0xF2;
    TCCR0 = 0x01;
    while (TIFR & (1<<TOV0)) {}
}
```

Question: How to calculate the delay generated by the timer?

Answer:

- 1) Calculate how much a machine clock lasts.
 $T = 1/f$
- 2) Calculate how many machine clocks it waits.
- 3) Delay = $T * \text{number of machine cycles}$

In example 1 calculate the delay. Imagine XTAL = 10 MHz.

Solution 1 (inaccurate):

1) Calculating T:

$$T = 1/f = 1/10M = 0.1\mu s$$

2) Calculating num of machine cycles:

\$100

-\$F2

\$0E = 14

3) Calculating delay

$$14 * 0.1\mu s = 1.4 \mu s$$

```
.INCLUDE "M32DEF.INC"
```

```
LDI    R16,0x20
SBI    DDRB,5    ;PB5 as an output
LDI    R17,0
OUT    PORTB,R17
BEGIN: LDI    R20,0xF2
OUT    TCNT0,R20    ;load timer0
LDI    R20,0x01
OUT    TCCR0,R20 ;Timer0,Normal mode,int clk
AGAIN: IN     R20,TIFR    ;read TIFR
SBRS   R20,0 ;if TOV0 is set skip next inst.
RJMP   AGAIN
LDI    R20,0x0
OUT    TCCR0,R20    ;stop Timer0
LDI    R20,0x01
OUT    TIFR,R20    ;clear TOV0 flag

EOR    R17,R16    ;toggle D5 of R17
OUT    PORTB,R17    ;toggle PB5
RJMP   BEGIN
```

Accurate calculating

Other than timer, executing the instructions consumes time; so if we want to calculate the accurate delay a program causes we should add the delay caused by instructions to the delay caused by the timer

	LDI	R16,0x20	
	SBI	DDRB,5	
	LDI	R17,0	
	OUT	PORTB,R17	
BEGIN:	LDI	R20,0xF2	1
	OUT	TCNT0,R20	1
	LDI	R20,0x01	1
	OUT	TCCR0,R20	1
AGAIN:	IN	R20,TIFR	1
	SBRs	R20,0	1 / 2
	RJMP	AGAIN	2
	LDI	R20,0x0	1
	OUT	TCCR0,R20	1
	LDI	R20,0x01	1
	OUT	TIFR,R20	1
	EOR	R17,R16	1
	OUT	PORTB,R17	1
	RJMP	BEGIN	2
			18

Delay caused by timer = $14 * 0.1\mu\text{s} = 1.4\mu\text{s}$

Delay caused by instructions = $18 * 0.1\mu\text{s} = 1.8\mu\text{s}$

Total delay = $3.2\mu\text{s}$ @ wave period = $2 * 3.2\mu\text{s} = 6.4\mu\text{s}$ @ wave frequency = 156.25 KHz

Finding values to be loaded into the timer

1. Calculate the period of clock source.
 - Period = $1 / \text{Frequency}$
 - E.g. For XTAL = 8 MHz $\Rightarrow T = 1/8\text{MHz}$
2. Divide the desired time delay by period of clock.
3. Perform $256 - n$, where n is the decimal value we got in Step 2.
4. Set TCNT0 = $256 - n$

Example 2: Assuming that XTAL = 10 MHz, write a program to generate a square wave with a period of 10 μ s on pin PORTB.3.

- For a square wave with $T = 10 \mu$ s we must have a time delay of 5 μ s. Because XTAL = 10 MHz, the counter counts up every 0.1 μ s. This means that we need 5μ s / 0.1 μ s = 50 clocks. $256 - 50 = 206$.

```
.INCLUDE "M32DEF.INC"
```

```
        LDI        R16,0x08
        SBI        DDRB,3    ;PB3 as an output
        LDI        R17,0
        OUT        PORTB,R17
BEGIN:   LDI        R20,206
        OUT        TCNT0,R20    ;load timer0
        LDI        R20,0x01
        OUT        TCCR0,R20 ;Timer0,Normal mode,int clk
AGAIN:   IN        R20,TIFR      ;read TIFR
        SBRS       R20,TOV0 ;if TOV0 is set skip next
        RJMP       AGAIN
        LDI        R20,0x0
        OUT        TCCR0,R20    ;stop Timer0
        LDI        R20,0x01
        OUT        TIFR,R20     ;clear TOV0 flag
        EOR        R17,R16      ;toggle D3 of R17
        OUT        PORTB,R17    ;toggle PB3
        RJMP       BEGIN
```

```
DDRB = 1<<3;
PORTB &= ~ (1<<3);
while (1)
{
    TCNT0 = 206;
    TCCR0 = 0x01;
    while((TIFR&0x01) == 0);
    TCCR0 = 0;
    TIFR = 1<<TOV0;
    PORTB = PORTB ^ (1<<3);
}
```

XOR

Example 3: Modify TCNT0 in Example 2 to get the largest time delay possible with no prescaler.

Find the delay in μs . In your calculation, do not include the overhead due to instructions.

- To get the largest delay we make TCNT0 zero. This will count up from 00 to 0xFF and then roll over to zero.

```
.INCLUDE "M32DEF.INC"
```

```
LDI    R16,1<<3
SBI    DDRB,3      ;PB3 as an output
LDI    R17,0
OUT    PORTB,R17
BEGIN: LDI    R20,0x0
OUT    TCNT0,R20    ;load Timer0
LDI    R20,0x01
OUT    TCCR0,
AGAIN: IN     R20,TI
SBRS   R20,TC
RJMP   AGAIN
LDI    R20,0x
OUT    TCCR0,
LDI    R20,0x
OUT    TIFR,R
EOR    R17,R1
OUT    PORTB,
RJMP   BEGIN
```

```
DDRB = 1 << 3;
PORTB &= ~(1<<3);
while (1)
{
    TCNT0 = 0x0;
    TCCR0 = 0x01;

while((TIFR&(1<<TOV0))==0);
    TCCR0 = 0;
    TIFR = 0x01;
    PORTB = PORTB^(1<<3);
}
```

Solution

1) Calculating T:

$$T = 1/f = 1/10\text{MHz} = 0.1\mu\text{s}$$

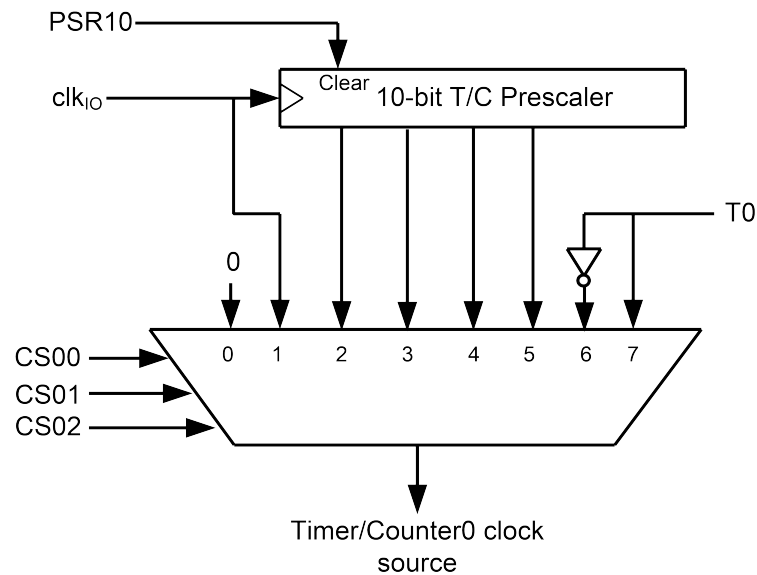
2) Calculating delay

$$256 * 0.1\mu\text{s} = 25.6\mu\text{s}$$

Generating Large Delays

- Using loop
- Prescaler
- Bigger counters

Prescaler and generating a large time delay



CTC (Clear Timer on Compare match) mode

Timer/Counter

Interrupt Flag Register

– TIFR

Bit	7	6	5	4	3	2	1	0	
	OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0	TIFR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

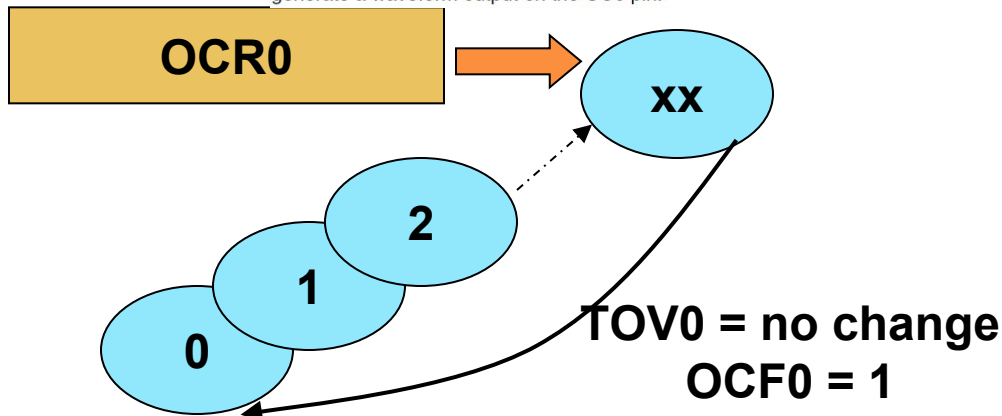
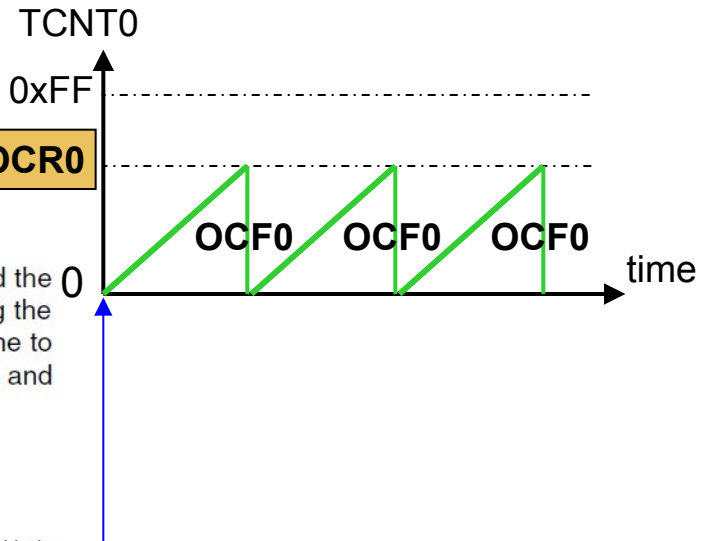
• Bit 1 – OCF0: Output Compare Flag 0

The OCF0 bit is set (one) when a compare match occurs between the Timer/Counter0 and the data in OCR0 – Output Compare Register0. OCF0 is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF0 is cleared by writing a logic one to the flag. When the I-bit in SREG, OCIE0 (Timer/Counter0 Compare Match Interrupt Enable), and OCF0 are set (one), the Timer/Counter0 Compare Match Interrupt is executed.

Output Compare Register – OCR0

Bit	7	6	5	4	3	2	1	0	
	OCR0[7:0]								OCR0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Output Compare Register contains an 8-bit value that is continuously compared with the counter value (TCNT0). A match can be used to generate an output compare interrupt, or to generate a waveform output on the OC0 pin.



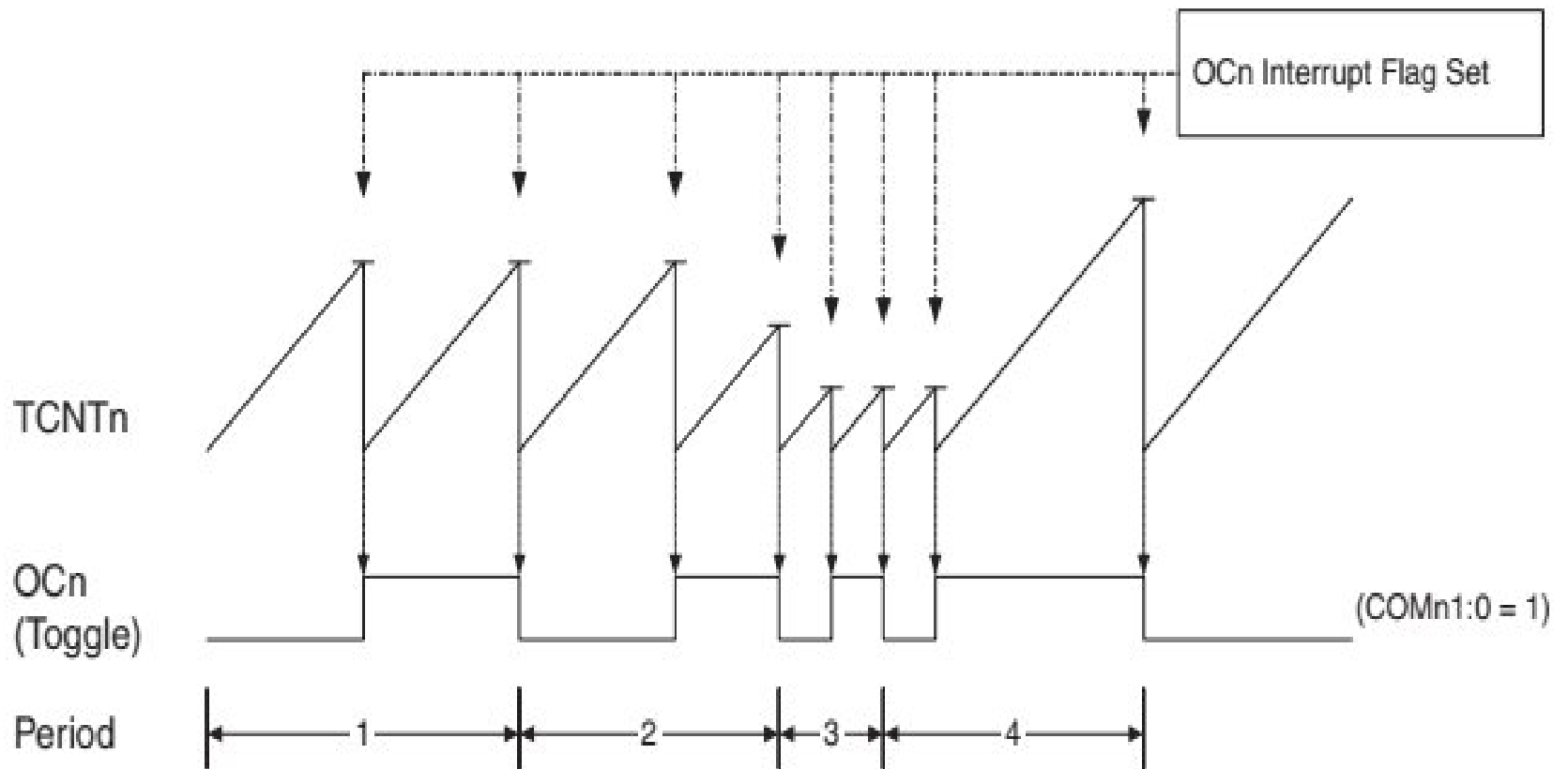
TOV0:

0

OCF0:

1

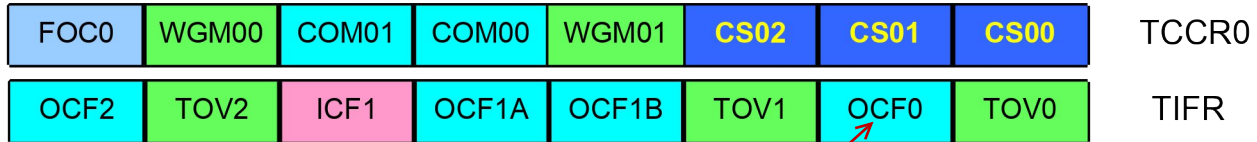
Figure 31. CTC Mode, Timing Diagram



Change OCR0 will change frequency of output

Example 4: Rewrite example 2 using CTC

Rewrite example 2 using CTC



- For a square wave with $T = 10 \mu s$ we must have a time delay of $5 \mu s$. Because $XTAL = 10 \text{ MHz}$, the counter counts up every $0.1 \mu s$. This means that we need $5 \mu s / 0.1 \mu s = 50$ clocks. Therefore, we have $OCR0 = 49$.

```
.INCLUDE "M32DEF.INC"

LDI    R16,0x08
SBI    DDRB,3    ;PB3 as an output
LDI    R17,0
OUT    PORTB,R17
LDI    R20,49
OUT    OCR0,R20 ;load timer0

BEGIN: LDI    R20,0x09
OUT    TCCR0,R20 ;Timer0,CTC mode,int clk

AGAIN: IN     R20,TIFR    ;read TIFR
SBRS   R20,OCF0 ;if OCF0 is set skip next
RJMP   AGAIN

LDI    R20,0x0
OUT    TCCR0,R20    ;stop Timer0

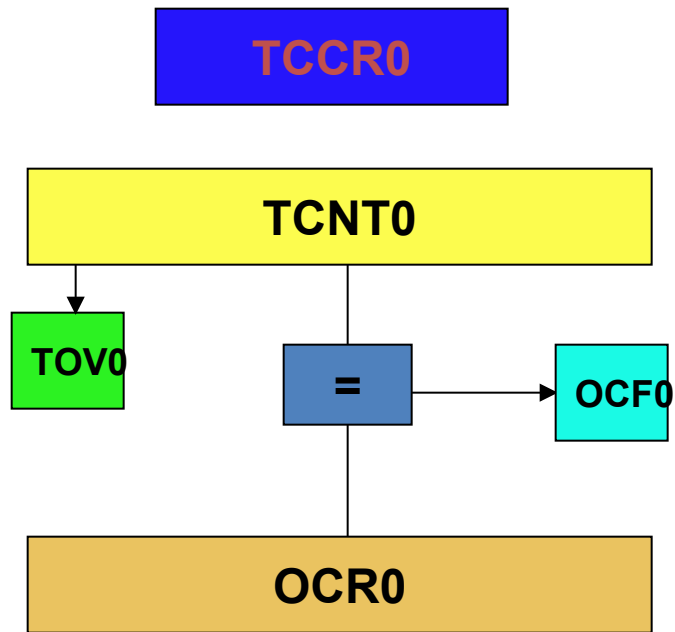
LDI    R20,0x02
OUT    TIFR,R20     ;clear OCF0 flag
EOR    R17,R16      ;toggle D3 of R17
OUT    PORTB,R17    ;toggle PB3
RJMP   BEGIN
```

```
DDRB |= 1<<3;
PORTB &= ~(1<<3);
while (1)
{
    OCR0 = 49;
    TCCR0 = 0x09;

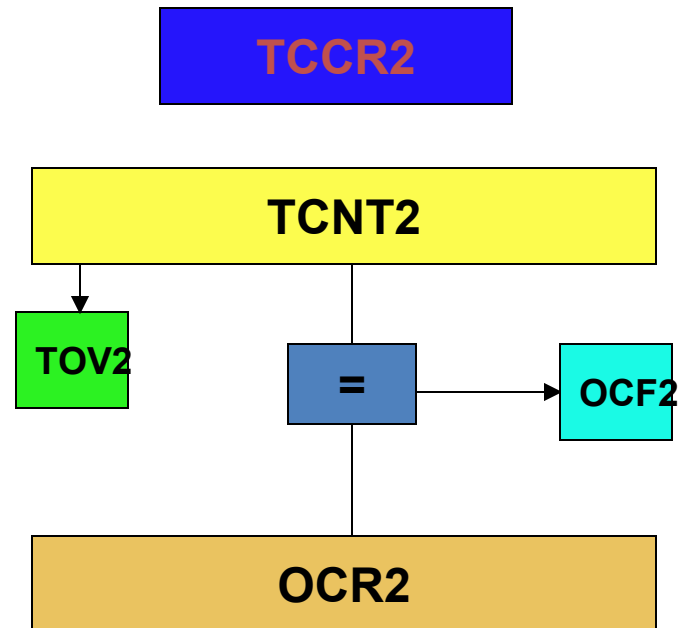
    while((TIFR&(1<<OCF0))==0);
    TCCR0 = 0; //stop timer0
    TIFR = 0x02;
    PORTB.3 = ~PORTB.3;
}
```


Timer2

- Timer0



- Timer2



The difference between Timer0 and

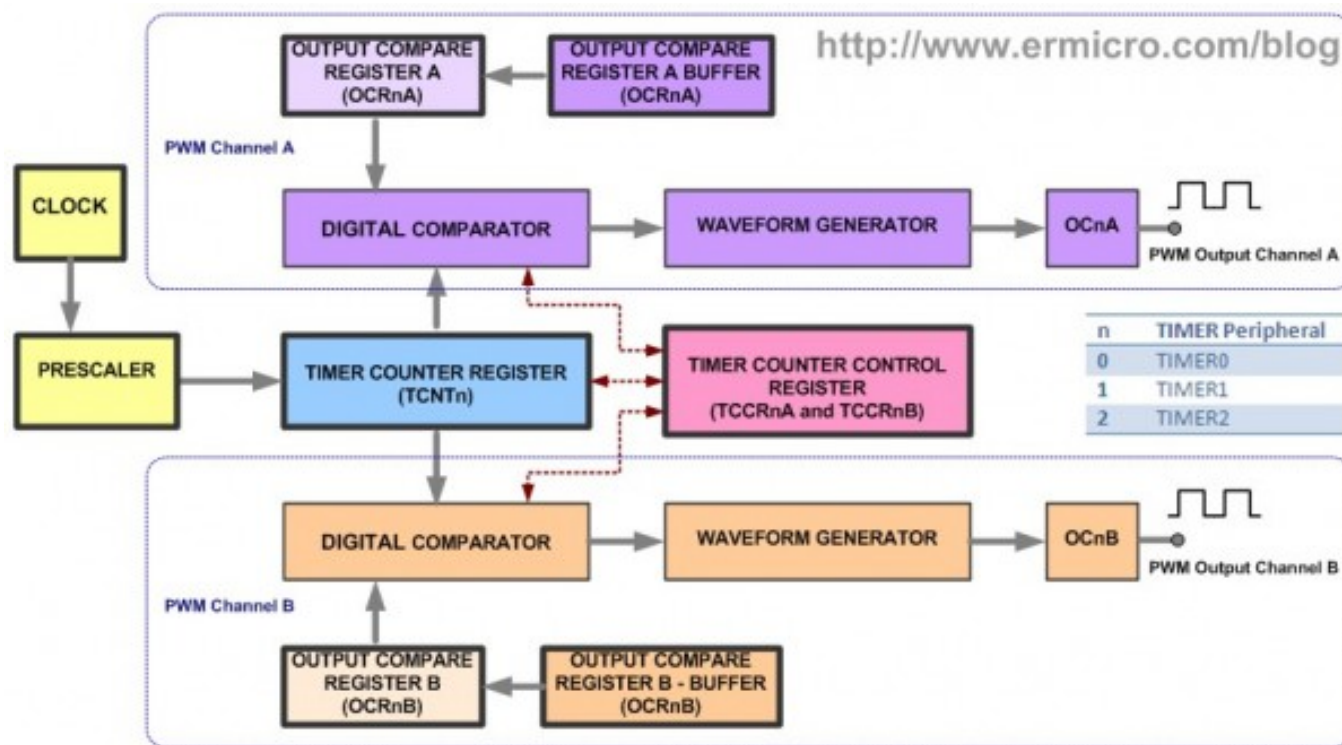
Timer2

- Timer0

CS02	CS01	CS00	Comment
0	0	0	Timer/Counter stopped
0	0	1	clk (No Prescaling)
0	1	0	clk / 8
0	1	1	clk / 64
1	0	0	clk / 256
1	0	1	clk / 1024
1	1	0	External clock (falling edge)
1	1	1	External clock (rising edge)

CS22	CS21	CS20	Comment
0	0	0	Timer/Counter stopped
0	0	1	clk (No Prescaling)
0	1	0	clk / 8
0	1	1	clk / 32
1	0	0	clk / 64
1	0	1	clk / 128
1	1	0	clk / 256
1	1	1	clk / 1024

Timer 0/2: PWM



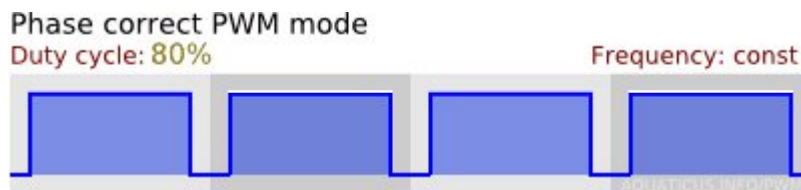
Simplified Atmel AVR ATmega48/88/168/328 Microcontroller PWM Peripheral

SOURCE	SIZE	PWM Out	I/O Pins
TIMER0	8-bit	OC0A	PD6
		OC0B	PD5
TIMER1	16-bit	OC1A	PB1
		OC1B	PB2
TIMER2	8-bit	OC2A	PB3
		OC2B	PD3

(PCINT14/RESET) PC6	1	28	PC5 (ADC5/SCL/PCINT13)
(PCINT16/RXD) PD0	2	27	PC4 (ADC4/SDA/PCINT12)
(PCINT17/TXD) PD1	3	26	PC3 (ADC3/PCINT11)
(PCINT18/INT0) PD2	4	25	PC2 (ADC2/PCINT10)
(PCINT19/OC2B/INT1) PD3	5	24	PC1 (ADC1/PCINT9)
(PCINT20/XCK/T0) PD4	6	23	PC0 (ADC0/PCINT8)
VCC	7	22	GND
GND	8	21	AREF
(PCINT8/XTAL1/TOSC1) PB6	9	20	AVCC
(PCINT7/XTAL2/TOSC2) PB7	10	19	PB5 (SCK/PCINT5)
(PCINT21/OC0B/T1) PD5	11	18	PB4 (MISO/PCINT4)
(PCINT22/OC0A/AIN0) PD6	12	17	PB3 (MOSI/OC2A/PCINT3)
(PCINT23/AIN1) PD7	13	16	PB2 (SS/OC1B/PCINT2)
(PCINT0/CLKO/ICP1) PB0	14	15	PB1 (OC1A/PCINT1)

Atmel AVR ATmega48/88/168/328 Pulse Width Modulation (PWM) Peripheral Output Pins

- The difference is that with Fast PWM when the duty cycle changes, the center position of the pulse changes position (therefore the phase changes). With phase correct PWM when the duty cycle changes, the center position of the pulse remains constant (and therefore the phase remains constant).



STANDARD PWM

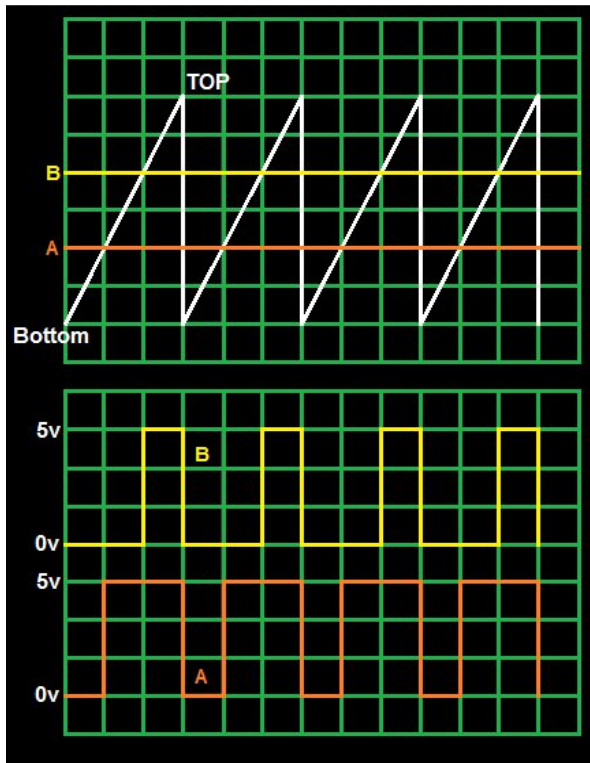


Phase Correct PWM

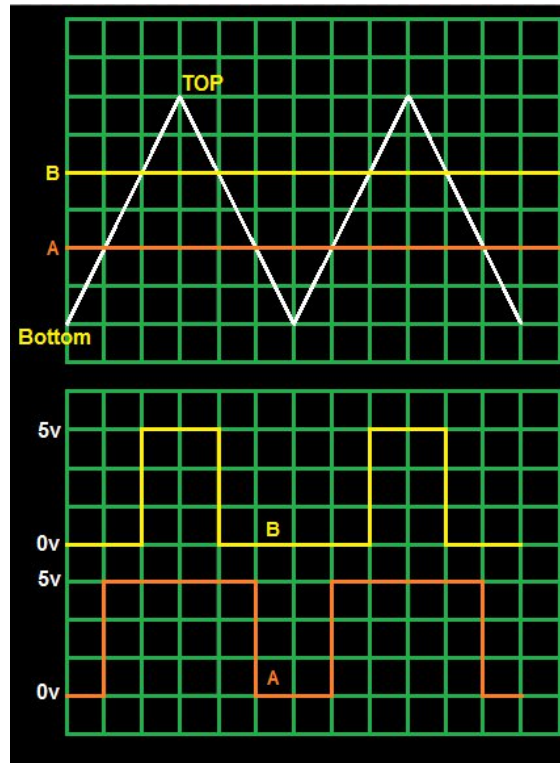


03 PWM modes

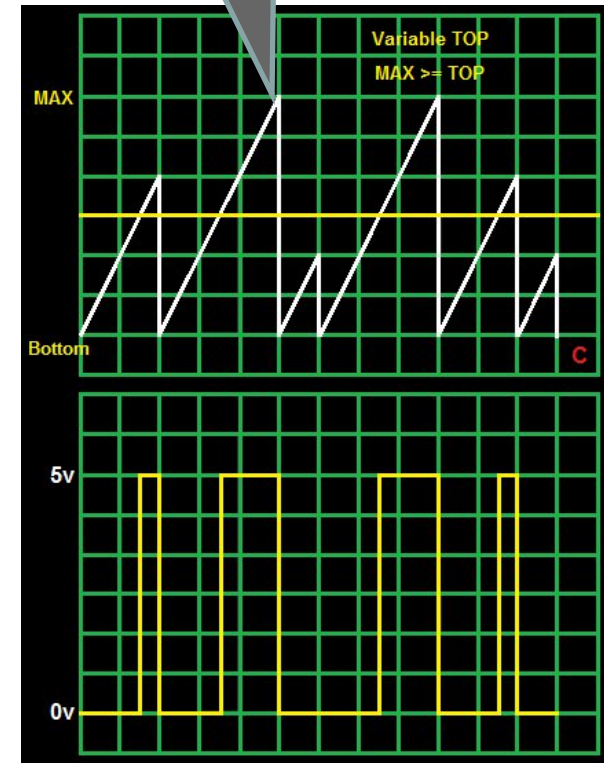
Change TOP value to vary frequency



Fast PWM



Phase Correct PWM



Frequency and Phase Correct PWM

Making Choices: PWM

- **Choosing Timer**

- TIMER0 and TIMER2 provide 8bit accuracy, TIMER1 provides 16bit accuracy. In 8bit accuracy, you have 256 individual steps, whereas in 16bit accuracy => 65536 steps.
 - The speed of a DC motor: use an 8bit timer for this. Even 8bit is too much
 - But if servo motors, you have to use 16bit timer.

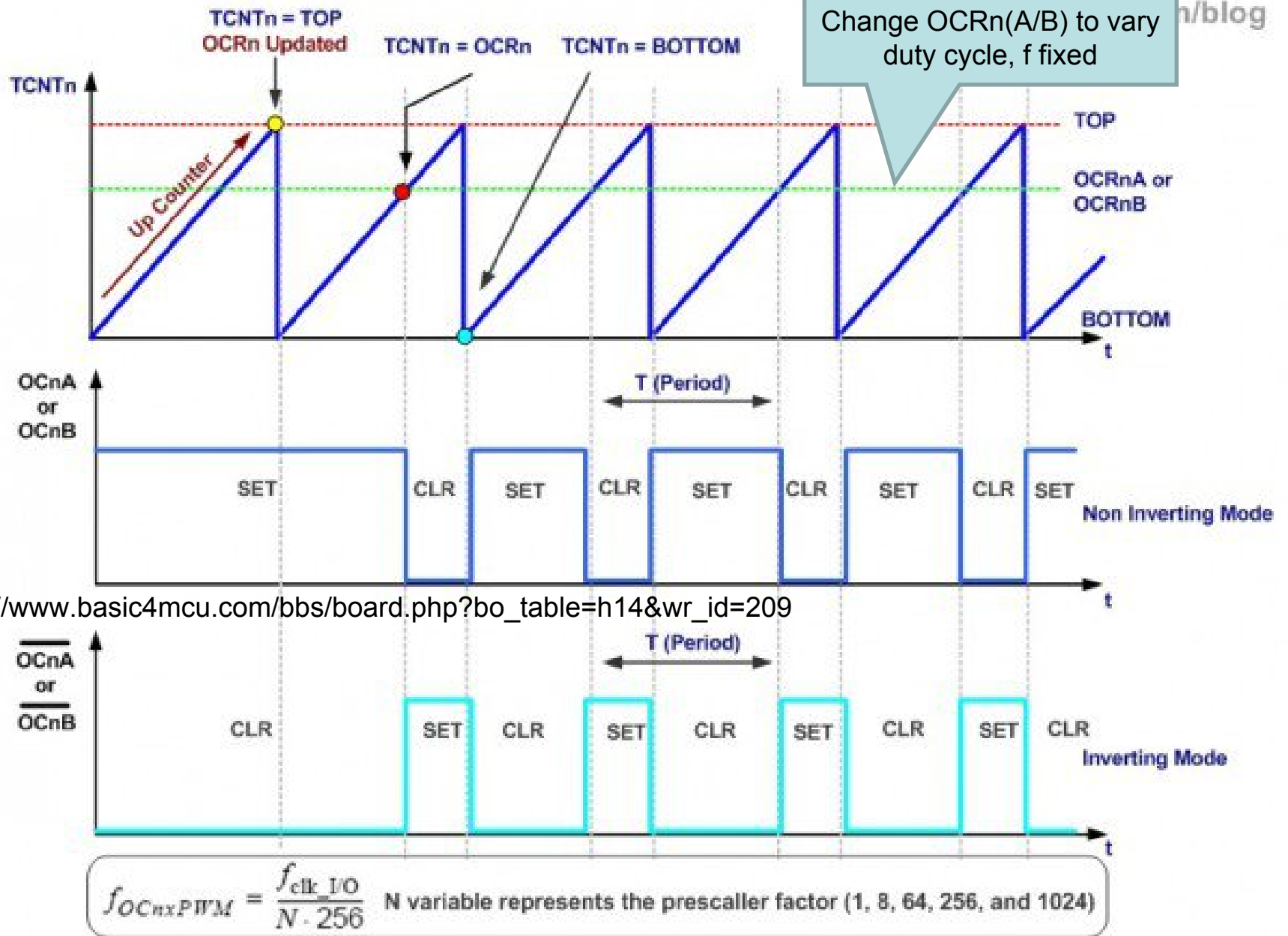
- **Choosing Mode of Operation (4 modes)**

(normal non-PWM, fast PWM, phase correct, phase & freq PWM)

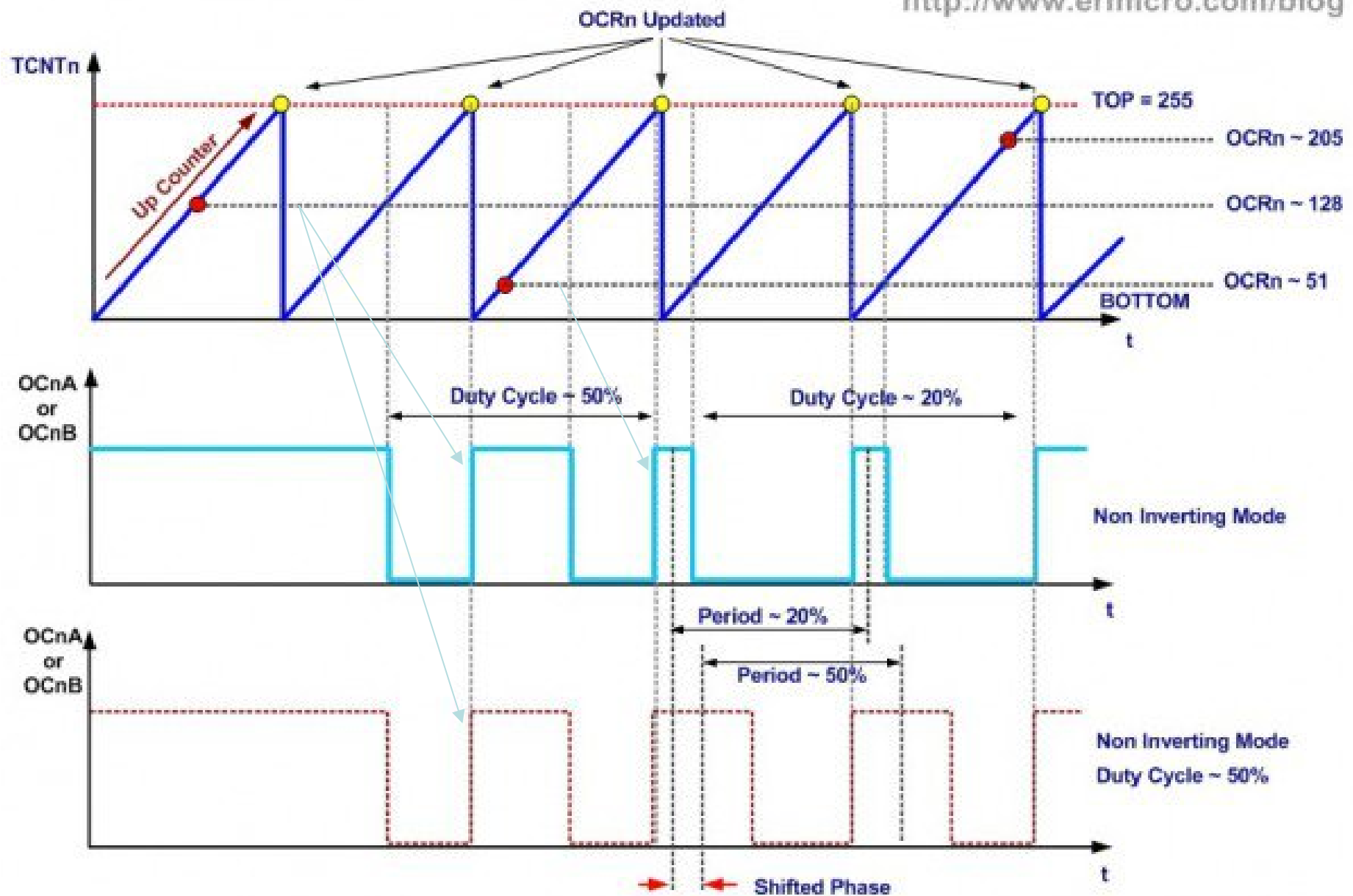
- Control speed of DC motors or brightness of LEDs, go for any one of them.
- Fast PWM would be better for:
 - This high frequency makes the fast PWM mode well suited for power regulation, rectification, and DAC applications. High frequency allows physically small sized external components (coils, capacitors), and therefore reduces total system cost.
 - For telecommunication, or signal sampling,
- For general applications, phase correct PWM would do.
- Phase & frequency correct: we seldom change PWM frequency on fly => common application is to generate sound, tone

- **Choosing Compare Output Modes**

- Out of the three modes, inverted, non-inverted and toggle mode, non-inverted mode is the most reasonable. This is because upon increasing the compare voltage, the duty cycle increases.



Atmel AVR ATmega48/88/168/328 Fast PWM Mode

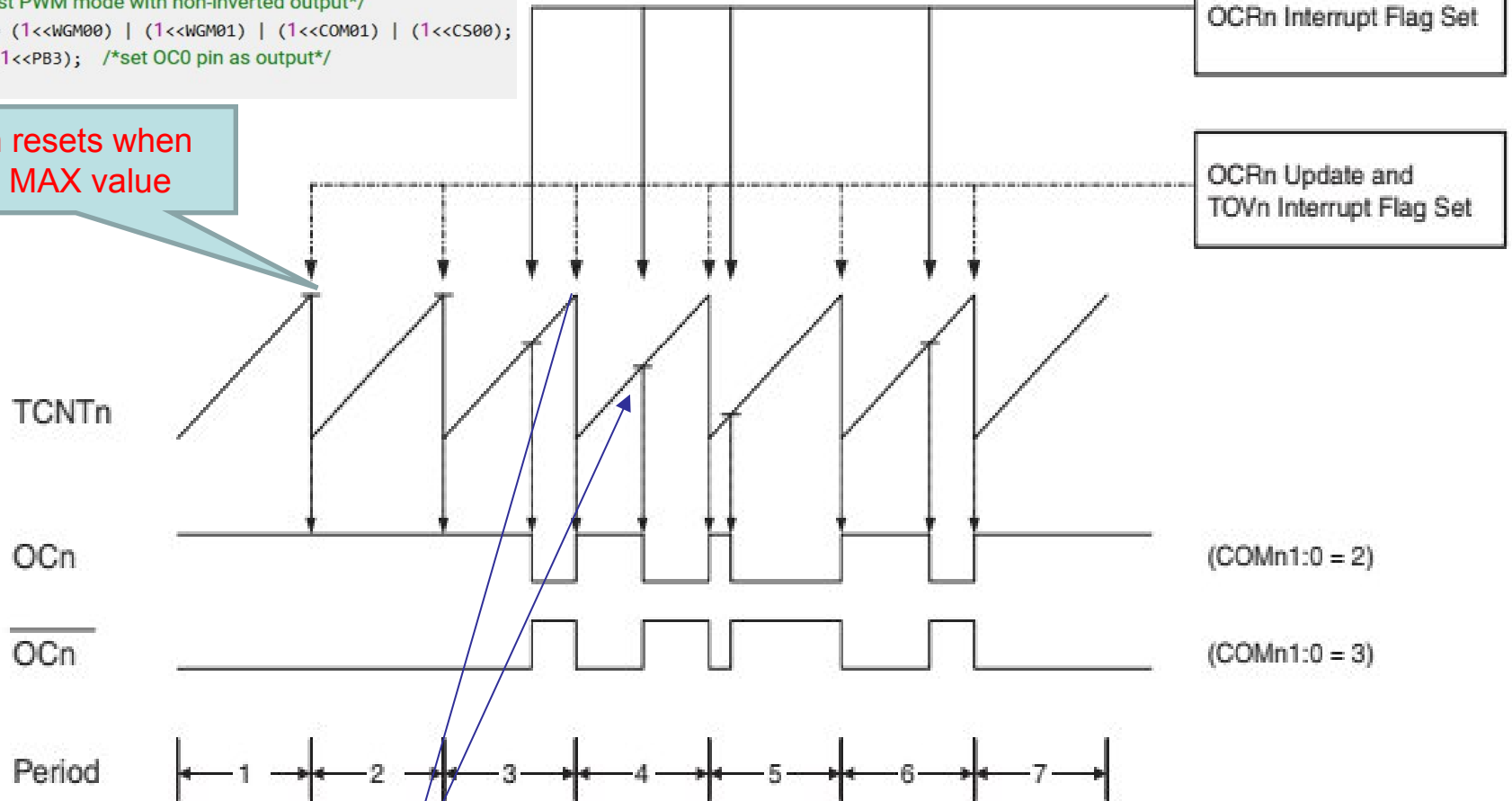


Atmel AVR ATmega48/88/168/328 Shifted Phase Effect in Fast PWM Mode

Figure 32. Fast PWM Mode, Timing Diagram

```
void PWM_init()
{
    /*set fast PWM mode with non-inverted output*/
    TCCR0 = (1<<WGM00) | (1<<WGM01) | (1<<COM01) | (1<<CS00);
    DDRB|=(1<<PB3); /*set OC0 pin as output*/
}
```

TCNTn resets when reach MAX value



- Change or update OCR0 at this point if needed to set up new matching moment => change duty cycle (but f= fixed)
- Clear OC0A/OC0B at match and Set OC0A/OC0B at BOTTOM

The PWM frequency for the output can be calculated by the following equation:

$$f_{OCnPWM} = \frac{f_{clk_I/O}}{N \cdot 256}$$

The N variable represents the prescale factor (1, 8, 64, 256, or 1024).

Setting Duty cycle: load value in OCR0 register to set duty cycle. 255 for 100% duty cycle and 0 for 0% duty cycle => load value 127 in OCR0, Duty cycle will be 50%.

TCCR0A – Timer/Counter Control Register A (TIMER0)

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Compare Output Mode, Fast PWM (TIMER0 Channel A)

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected. WGM02 = 1: Toggle OC0A on Compare Match.
1	0	Clear OC0A on Compare Match, set OC0A at BOTTOM, (non-inverting mode).
1	1	Set OC0A on Compare Match, clear OC0A at BOTTOM, (inverting mode).

Compare Output Mode, Fast PWM (TIMER0 Channel B)

COM0B1	COM0B0	Description
0	0	Normal port operation, OC0B disconnected.
0	1	Reserved
1	0	Clear OC0B on Compare Match, set OC0B at BOTTOM, (non-inverting mode).
1	1	Set OC0B on Compare Match, clear OC0B at BOTTOM, (inverting mode).

Wave Generation Mode Bit Description

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

BOTTOM	The counter reaches the BOTTOM when it becomes 0x00.
MAX	The counter reaches its MAXimum when it becomes 0xFF (decimal 255).
TOP	The counter reaches the TOP when it becomes equal to the highest value in the count sequence. The TOP value can be assigned to be the fixed value 0xFF (MAX) or the value stored in the OCR0 Register. The assignment is dependent on the mode of operation.

Atmel AVR ATmega48/88/168/328 TIMER0 Fast PWM Mode

TCCR0B – Timer/Counter Control Register B (TIMER0)

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The FOC0 bit is only active when the WGM00 bit specifies a non-PWM mode

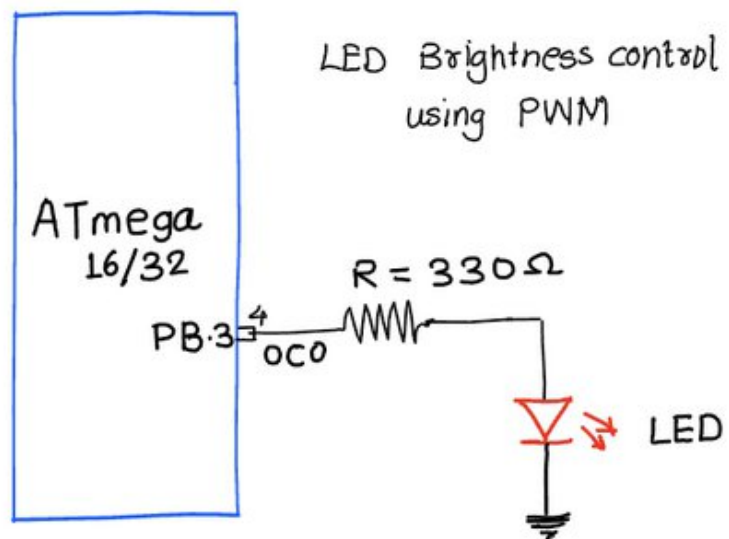
TIMER0 Clock Set Bit Description (CS02, CS01, and CS00)

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$\text{clk}_{\text{IO}}/(\text{No prescaling})$
0	1	0	$\text{clk}_{\text{IO}}/8$ (From prescaler)
0	1	1	$\text{clk}_{\text{IO}}/64$ (From prescaler)
1	0	0	$\text{clk}_{\text{IO}}/256$ (From prescaler)
1	0	1	$\text{clk}_{\text{IO}}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

Atmel AVR ATmega48/88/168/328 TIMER0 Fast PWM Mode Prescaler

Example

Control LED brightness using Fast PWM.



```
/*  
AVR ATmega16 PWM to control LED brightness  
http://www.electronicwings.com  
*/  
  
#define F_CPU 8000000UL  
#include "avr/io.h"  
#include <util/delay.h>  
  
void PWM_init()  
{  
    /*set fast PWM mode with non-inverted output*/  
    TCCR0 = (1<<WGM00) | (1<<WGM01) | (1<<COM01) | (1<<CS00);  
    DDRB |= (1<<PB3); /*set OC0 pin as output*/  
}  
  
int main ()  
{  
    unsigned char duty;  
  
    PWM_init();  
    while (1)  
    {  
        for(duty=0; duty<255; duty++)  
        {  
            OCR0=duty; /*increase the LED light intensity*/  
            _delay_ms(8);  
        }  
        for(duty=255; duty>1; duty--)  
        {  
            OCR0=duty; /*decrease the LED light intensity*/  
            _delay_ms(8);  
        }  
    }  
}
```

```
#include <avr/io.h>
#include <util/delay.h>
```

```
int main(void)
```

```
{
    unsigned char duty_cyc_a,duty_cyc_b;
```

```
    // Initial PORT Used
```

```
    DDRD = 0b11111111;    // Set PORTD: Output
```

```
    PORTD = 0x00;
```

```
    // Initial TIMER0 Fast PWM
```

```
    // Fast PWM Frequency = fclk / (N * 256), Where N is the Prescaler
```

```
    // f_PWM = 11059200 / (64 * 256) = 675 Hz
```

```
    TCCR0A = 0b10100011; // Fast PWM 8 Bit, Clear OCA0/OCB0 on Compare Match, Set on TOP
```

```
    TCCR0B = 0b00000011; // Used 64 Prescaler FOC0A & FOC0B = 0
```

```
    TCNT0 = 0;    // Reset TCNT0
```

```
    OCR0A = 0;    // Initial the Output Compare register A & B
```

```
    OCR0B = 0;
```

```
    duty_cyc_a=0; // Initial Duty Cycle for Channel A
```

```
    duty_cyc_b=255;    // Initial Duty Cycle for Channel B
```

```
    for(;;) {    // Loop Forever
```

```
        while(duty_cyc_a < 255) {
```

```
            OCR0A=duty_cyc_a++;
```

```
            OCR0B=duty_cyc_b--;
```

```
            _delay_ms(10);
```

```
        }
```

```
        while(duty_cyc_b < 255) {
```

```
            OCR0A=duty_cyc_a--;
```

```
            OCR0B=duty_cyc_b++;
```

```
            _delay_ms(10);
```

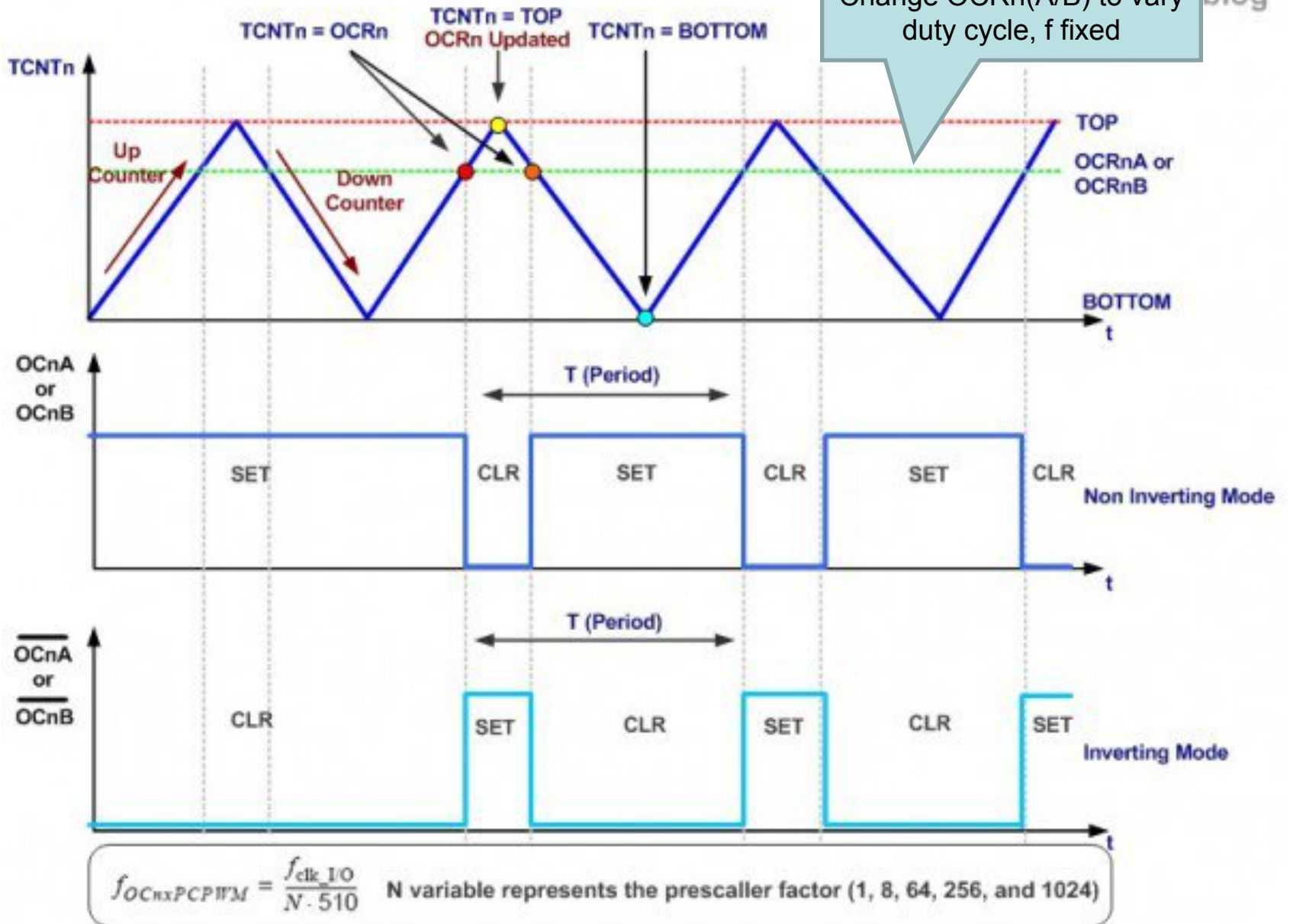
```
        }
```

```
    }
```

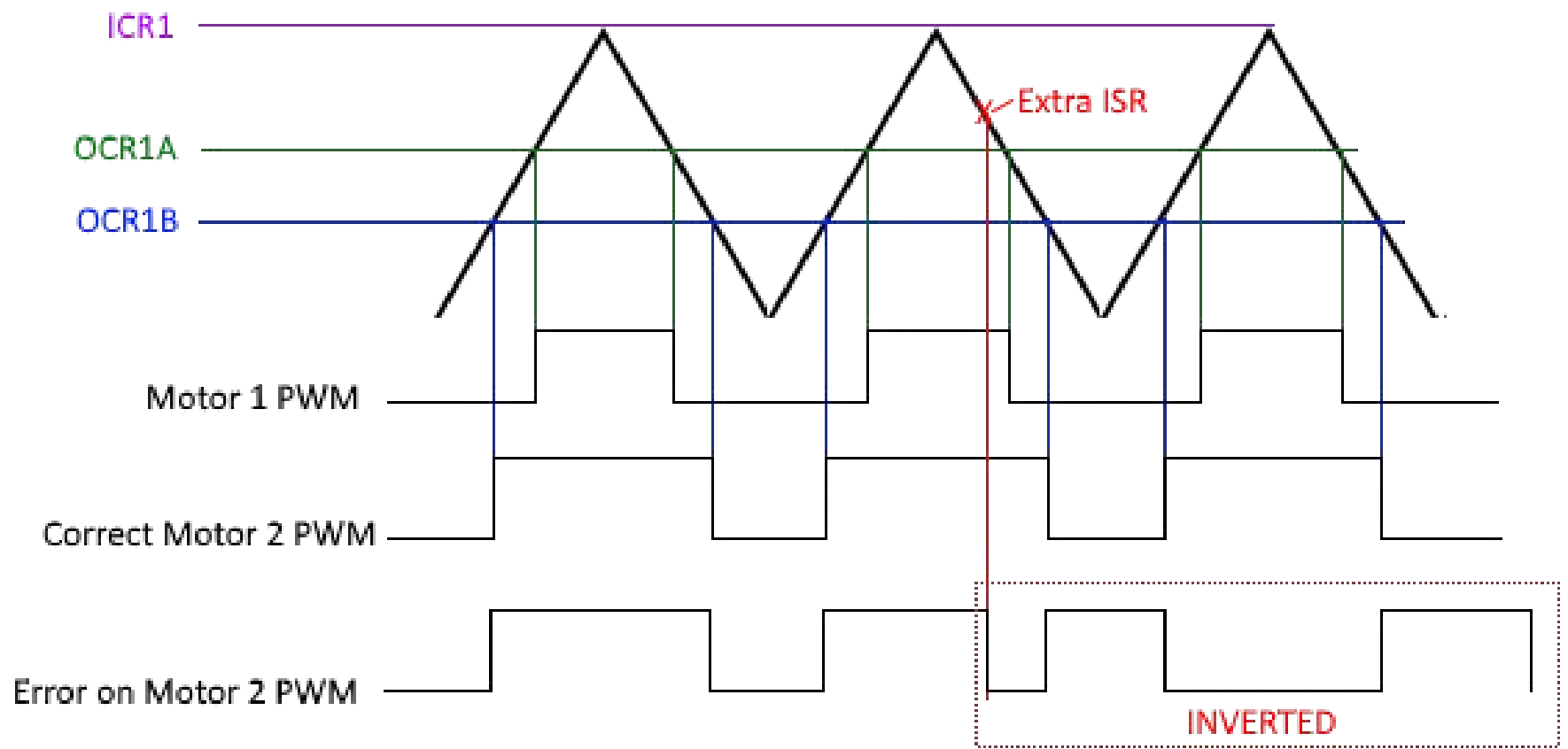
```
    return 0;    // Standard Return Code
```

```
}
```


Change OCRn(A/B) to vary duty cycle, f fixed



Atmel AVR ATmega48/88/168/328 Phase Correct PWM Mode



TCCR0A – Timer/Counter Control Register A (TIMER0)

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Compare Output Mode, Phase Correct PWM (TIMER0 Channel A)

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected. WGM02 = 1: Toggle OC0A on Compare Match.
1	0	Clear OC0 on compare match when up-counting. Set OC0 on compare match when downcounting.
1	1	Set OC0 on compare match when up-counting. Clear OC0 on compare match when downcounting.

Compare Output Mode, Phase Correct PWM (TIMER0 Channel B)

COM0B1	COM0B0	Description
0	0	Normal port operation, OC0B disconnected.
0	1	Reserved
1	0	Clear OC0 on compare match when up-counting. Set OC0 on compare match when downcounting.
1	1	Set OC0 on compare match when up-counting. Clear OC0 on compare match when downcounting.

Wave Generation Mode Bit Description

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Atmel AVR ATmega48/88/168/328 TIMER0 Phase Correct PWM Mode

TCCR0B – Timer/Counter Control Register B (TIMER0)

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TIMER0 Clock Set Bit Description (CS02, CS01, and CS00)

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$\text{clk}_{\text{IO}}/(\text{No prescaling})$
0	1	0	$\text{clk}_{\text{IO}}/8$ (From prescaler)
0	1	1	$\text{clk}_{\text{IO}}/64$ (From prescaler)
1	0	0	$\text{clk}_{\text{IO}}/256$ (From prescaler)
1	0	1	$\text{clk}_{\text{IO}}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

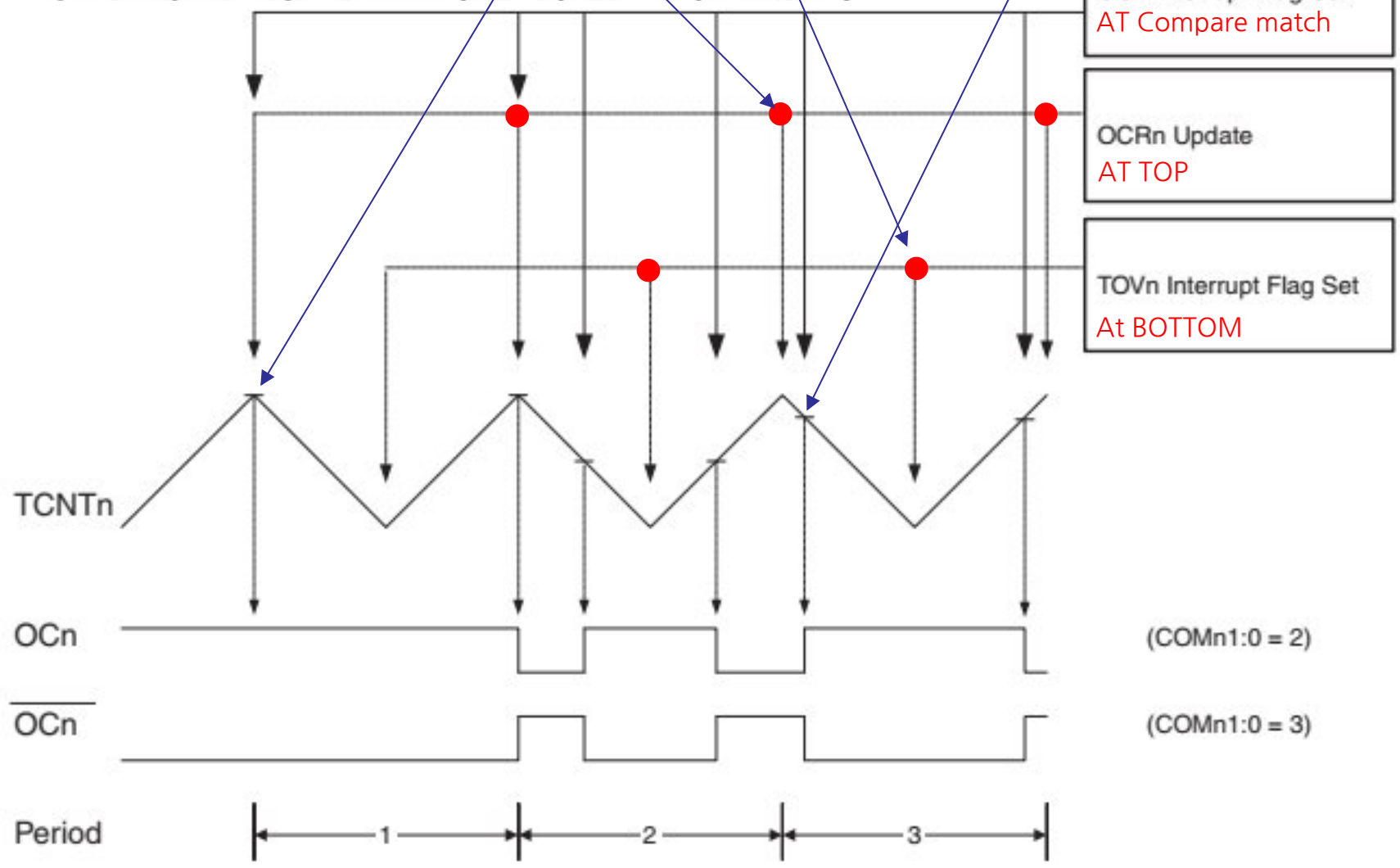
Atmel AVR ATmega48/88/168/328 TIMER0 Phase Correct PWM Mode Prescaler

Compare Output Mode, Phase Correct PWM (TIMER0 Channel A)

1	0
---	---

Clear OC0 on compare match when up-counting. Set OC0 on compare match when downcounting.

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM



On this C code, use TIMER0 Phase Correct PWM mode on both channel A & B.

```
int main(void)
{
    unsigned char duty_cyc_a,duty_cyc_b; // Initial PORT Used
    DDRD = 0b11111111; // Set PORTD: Output
    PORTD = 0x00;
    // Initial TIMER0 Phase Correct PWM // Fast PWM Frequency = fclk / (N * 510), Where N is the Prescaler
    // f_PWM = 11059200 / (64 * 510) = 338.82 Hz
    TCCR0A=0b10100001; //Phase Correct PWM 8bit, Clear OCA0/OCB0 on UP Compare Match, on DOWN Compare Match
    TCCR0B = 0b00000011; // Used 64 Prescaler
    TCNT0 = 0; // Reset TCNT0
    duty_cyc_a=0, duty_cyc_b=255; // Initial Duty Cycle for Channel A & B
    OCR0A=0, OCR0B=255; // Initial the Output Compare register A & B
    for(;;)
    { // Loop Forever
        while(duty_cyc_a < 255)
        {
            OCR0A=duty_cyc_a++;
            OCR0B=duty_cyc_b--; _delay_ms(10);
        }
        while(duty_cyc_b < 255)
        {
            OCR0A=duty_cyc_a--; OCR0B=duty_cyc_b++; _delay_ms(10); }
    }
    return 0; // Standard Return Code}
```

Wave Generation Mode bits **WGM02=0, WGM01=0, and WGM00 = 1**
=> Timer0 in phase correct PWM
//Clear OC0A, OC0B on Match, set OC0A, OC0B at Bottom
COM0A1 = 1; COM0A0=0; COM0B1 = 1; COM0B0=0

f_PWM = 11059200 / (64 * 510) = 338.82 Hz

```

#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    unsigned char duty_cyc_a,duty_cyc_b;

    // Initial PORT Used
    DDRD = 0b11111111;    // Set PORTD: Output
    PORTD = 0x00;

    // Initial TIMERO Phase Correct PWM
    // Fast PWM Frequency = fclk / (N * 510), Where N is the Prescaler
    // f_PWM = 11059200 / (64 * 510) = 338.82 Hz
    TCCR0A = 0b10100001; // Phase Correct PWM 8 Bit, Clear OCA0/OCB0 on Compare Match, Set on TOP
    TCCR0B = 0b00000011; // Used 64 Prescaler
    TCNT0 = 0;           // Reset TCNT0
    OCR0A = 0;           // Initial the Output Compare register A & B
    OCR0B = 0;

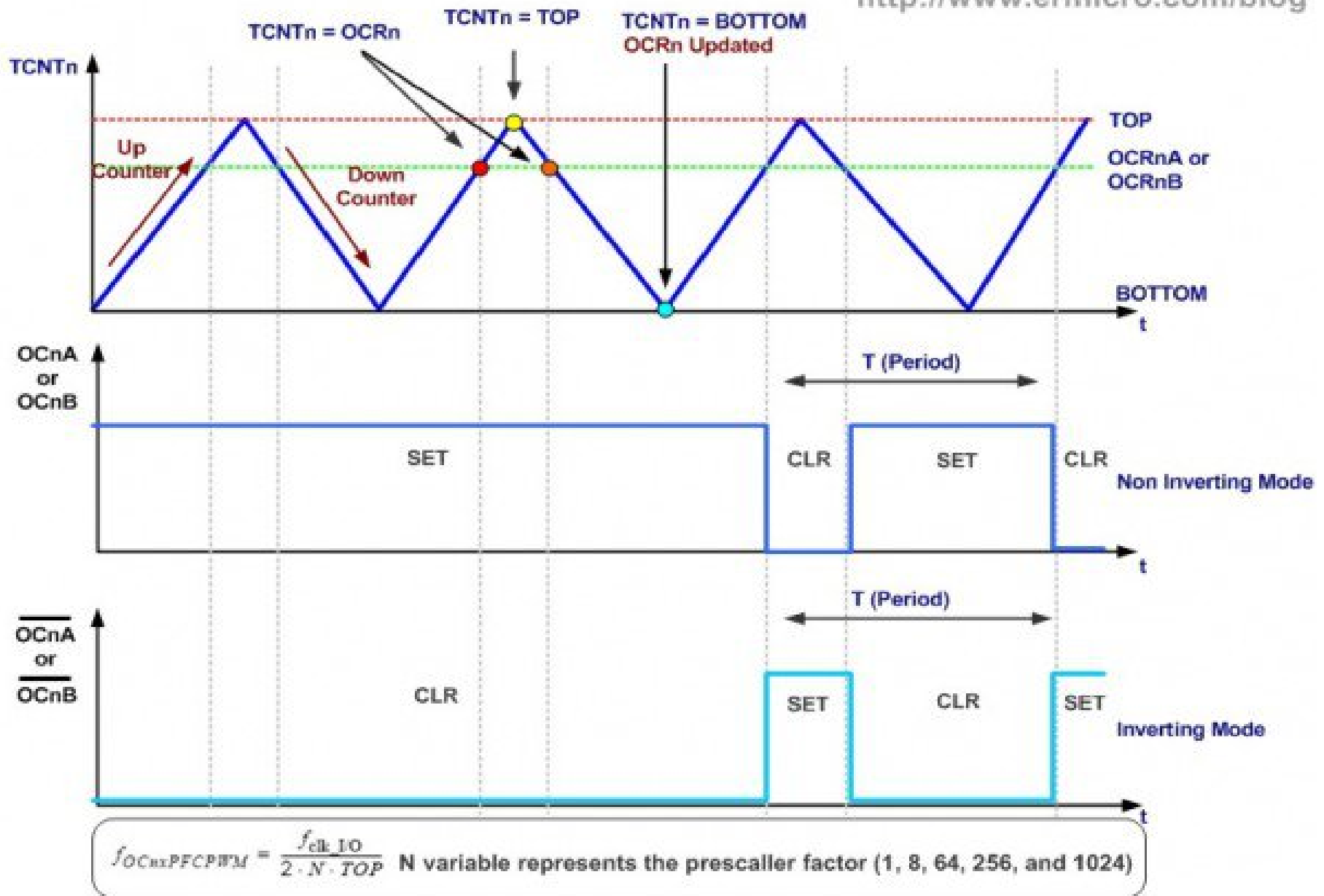
    duty_cyc_a=0;        // Initial Duty Cycle for Channel A
    duty_cyc_b=255;      // Initial Duty Cycle for Channel B

    for(;;) {            // Loop Forever
        while(duty_cyc_a < 255) {
            OCR0A=duty_cyc_a++;
            OCR0B=duty_cyc_b--;
            _delay_ms(10);
        }

        while(duty_cyc_b < 255) {
            OCR0A=duty_cyc_a--;
            OCR0B=duty_cyc_b++;
            _delay_ms(10);
        }
    }

    return 0;            // Standard Return Code
}

```



Atmel AVR ATmega48/88/168/328 TIMER1 Phase and Frequency Correct PWM Mode

AVAILABLE IN TIMER 1 ONLY (see details later)

Bit	7	6	5	4	3	2	1	0	
(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Compare Output Mode, Phase and Frequency Correct PWM (TIMER1 Channel A and B)

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 9 or 11: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on Compare Match when up-counting. Set OC1A/OC1B on Compare Match when downcounting.
1	1	Set OC1A/OC1B on Compare Match when up-counting. Clear OC1A/OC1B on Compare Match when downcounting.

Wave Generation Mode Bit Description

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x at	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

Atmel AVR ATmega48/88/168/328 TIMER1 Phase and Frequency Correct PWM Mode

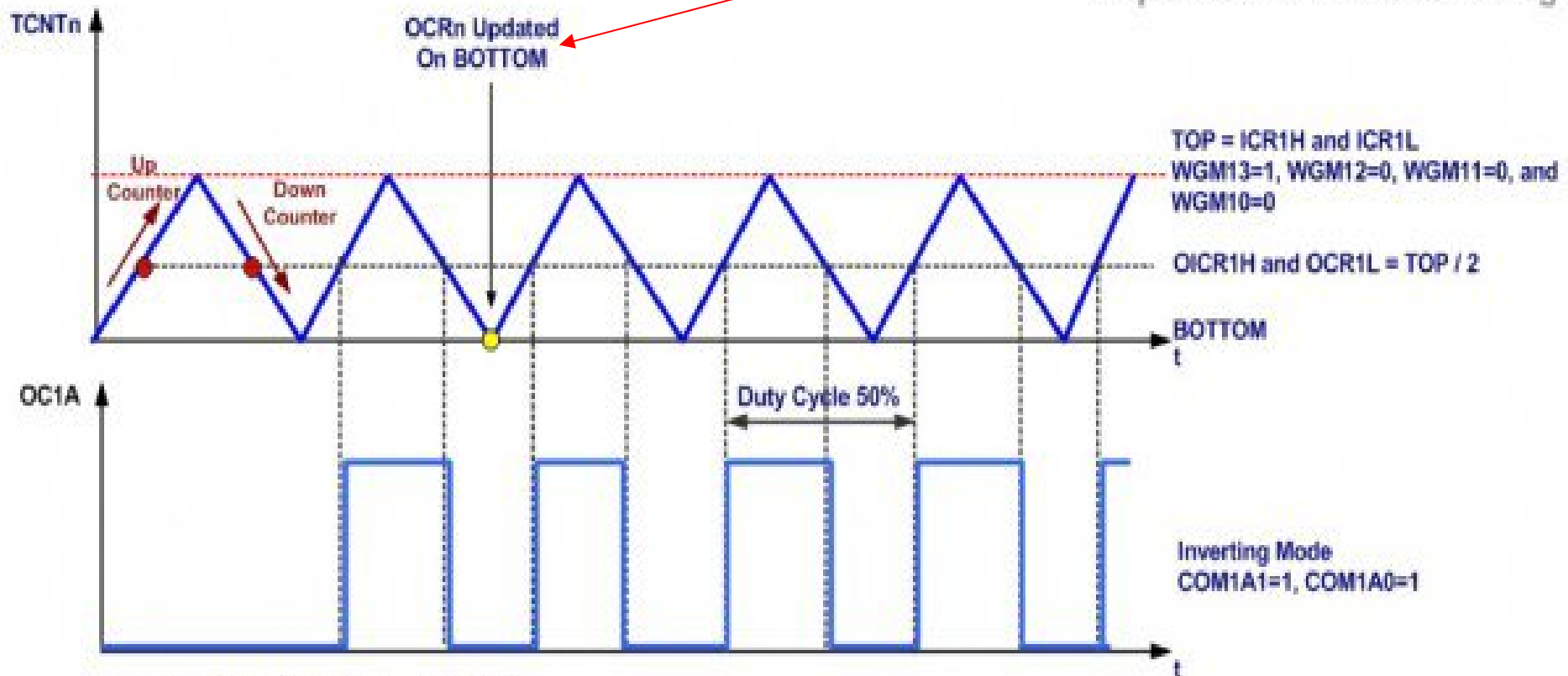
Compare Output Mode, Phase and Frequency Correct PWM (TIMER1 Channel A)

COM1A1/COM1B1	COM1A0/COM1B0	Description
1	1	Set OC1A/OC1B on Compare Match when up-counting. Clear OC1A/OC1B on Compare Match when downcounting.

Wave Generation Mode Bit Description

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x at	TOV1 Flag Set on
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM

<http://www.ermicro.com/blog>



Prescaler: CS12=0, CS11=1, and CS10=0

Atmel AVR ATmega48/88/168/328 TIMER1 Phase and Frequency Correct PWM Mode with TOP on ICR1

```
Init_Phase_Freq_Correct_PWM(void)
{
// Initial PORT Used
DDRD = 0b11111111;
// Set PORTD as Output
PORTD = 0b00000000;
DDRB = 0b11111110; // Set PB0 as Input and other as Output
PORTB = 0b00000000;

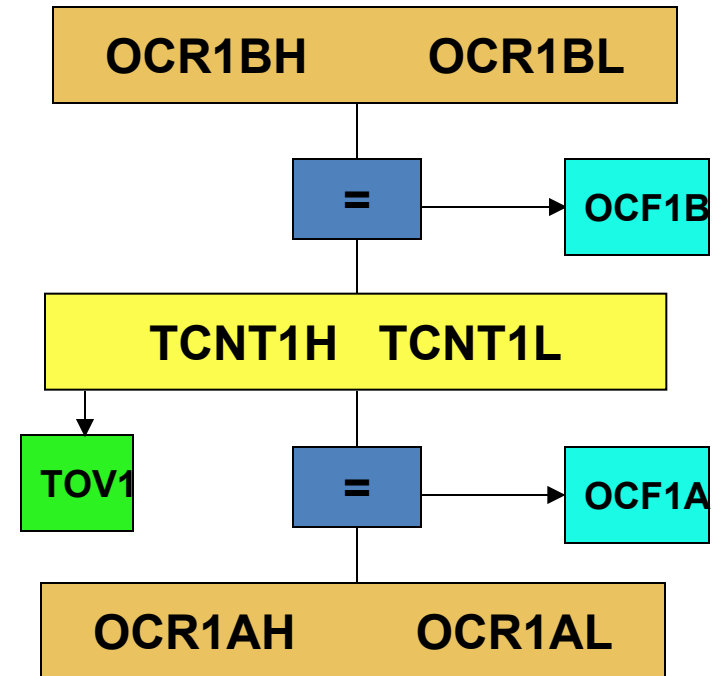
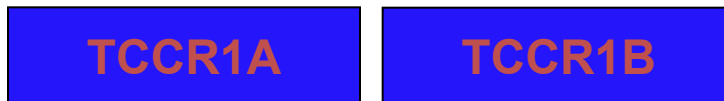
// Initial TIMER1 Phase and Frequency Correct PWM
// Set the Timer/Counter Control Register
TCCR1A = 0b11000000; // Set OC1A when up counting, Clear when down counting
TCCR1B = 0b00010000; // Phase/Freq-correct PWM, top value = ICR1, Prescaler: Off
}
```

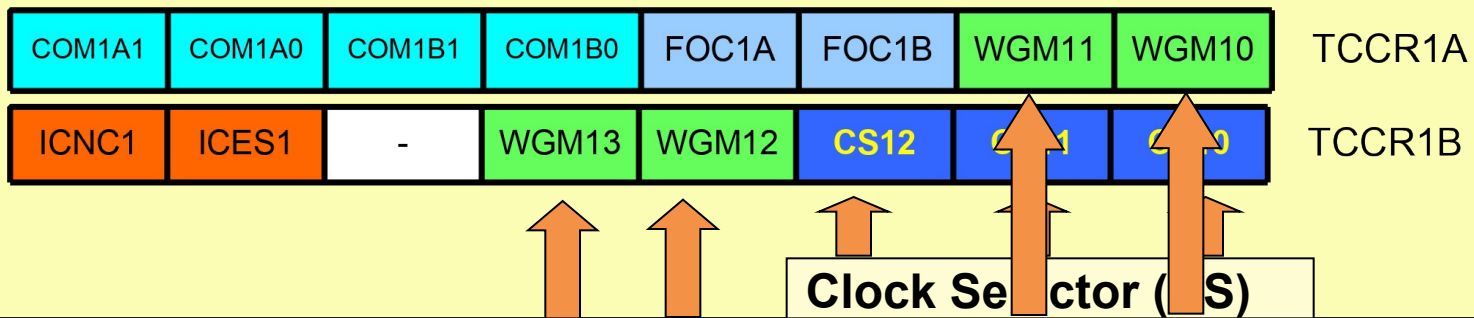
*The Phase and Frequency Correct PWM Mode feature is only available on TIMER1 (16-bit). Basically the Phase and Frequency Correct PWM mode use the same dual slope technique used in Phase Correct PWM mode to generate the PWM signal. **These two modes actually are identical if we never change the PWM signal frequency**, but if we need to change the PWM signal frequency on fly, then we need to use the AVR ATmega168 microcontroller Phase and Frequency Correct mode to generate the PWM signal.*

Timer 1: PWM

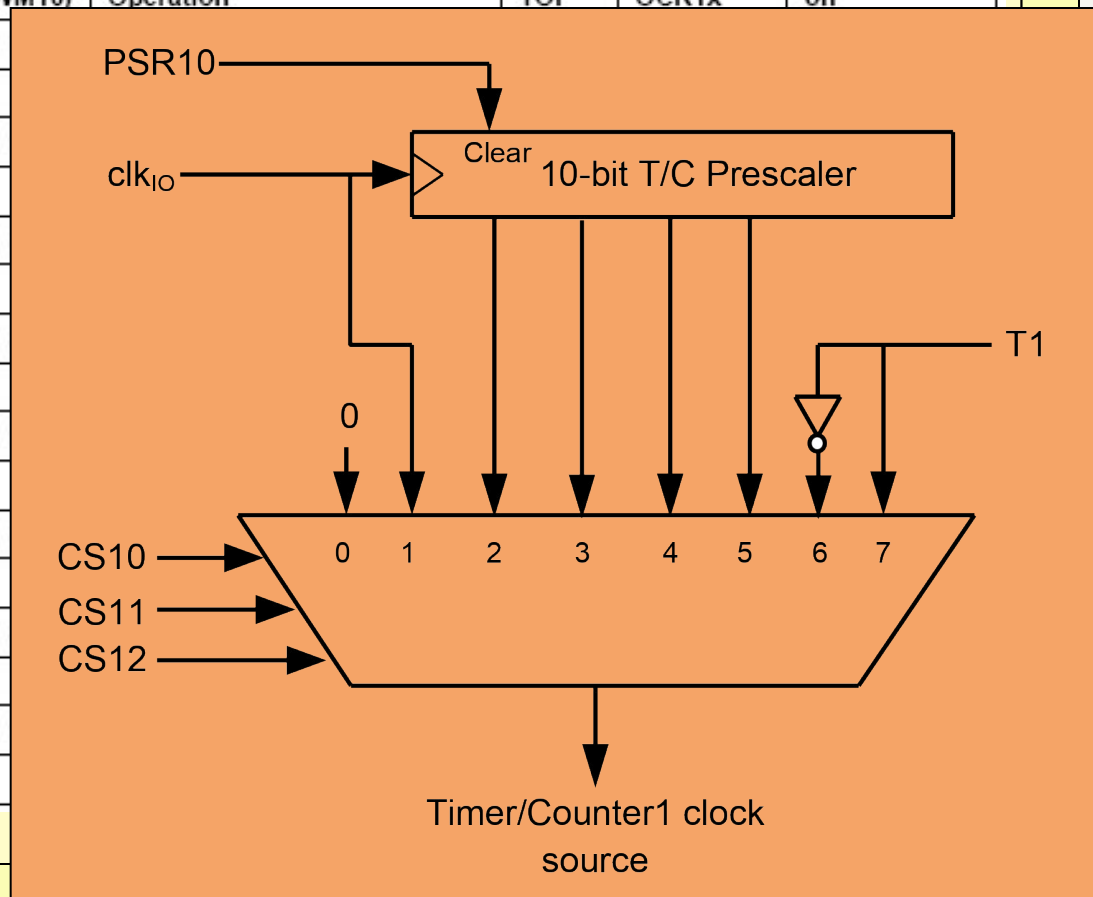
<http://www.ermicro.com/blog/?p=1971>

Timer 1





Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x	TOV1 Flag Set on
0	0	0	0	0				
1	0	0	0	0				
2	0	0	1	0				
3	0	0	1	0				
4	0	1	0	0				
5	0	1	0	0				
6	0	1	1	0				
7	0	1	1	0				
8	1	0	0	0				
9	1	0	0	0				
10	1	0	1	0				
11	1	0	1	0				
12	1	1	0	0				
13	1	1	0	0				
14	1	1	1	0				
15	1	1	1	0				



Atmega32(L)

Table 47. Waveform Generation Mode Bit Description⁽¹⁾

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

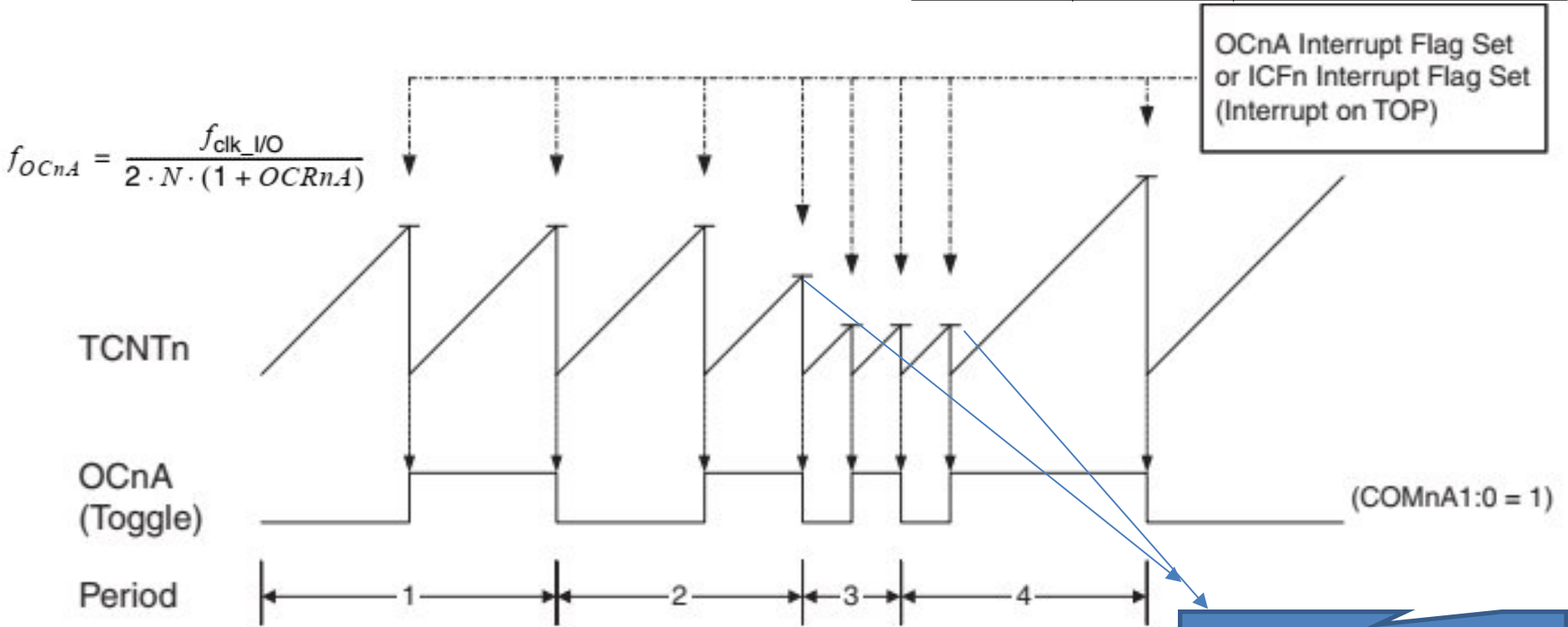
Note: 1. The CTC1 and PWM11:0 bit definition names are obsolete. Use the WGM12:0 definitions. However, the functionality and location of these bits are compatible with previous versions of the timer.

Timer 1: CTC

Figure 45. CTC Mode, Timing Diagram

Table 44. Compare Output Mode, non-PWM

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	Toggle OC1A/OC1B on compare match
1	0	Clear OC1A/OC1B on compare match (Set output to low level)
1	1	Set OC1A/OC1B on compare match (Set output to high level)



Change/Update OCR1A/B will change freq, D fixed

An interrupt can be generated at each time the counter value reaches TOP value by either using OCF1A or ICF1 Flag according to the register used to define the TOP value. **If interrupt is enabled, interrupt ISR can be used for updating the TOP value.**

BOTTOM	The counter reaches the <i>BOTTOM</i> when it becomes 0x0000.
MAX	The counter reaches its <i>MAX</i> imum when it becomes 0xFFFF (decimal 65535).
TOP	The counter reaches the <i>TOP</i> when it becomes equal to the highest value in the count sequence. The TOP value can be assigned to be one of the fixed values: 0x00FF, 0x01FF, or 0x03FF, or to the value stored in the OCR1A or ICR1 Register. The assignment is dependent of the mode of operation.

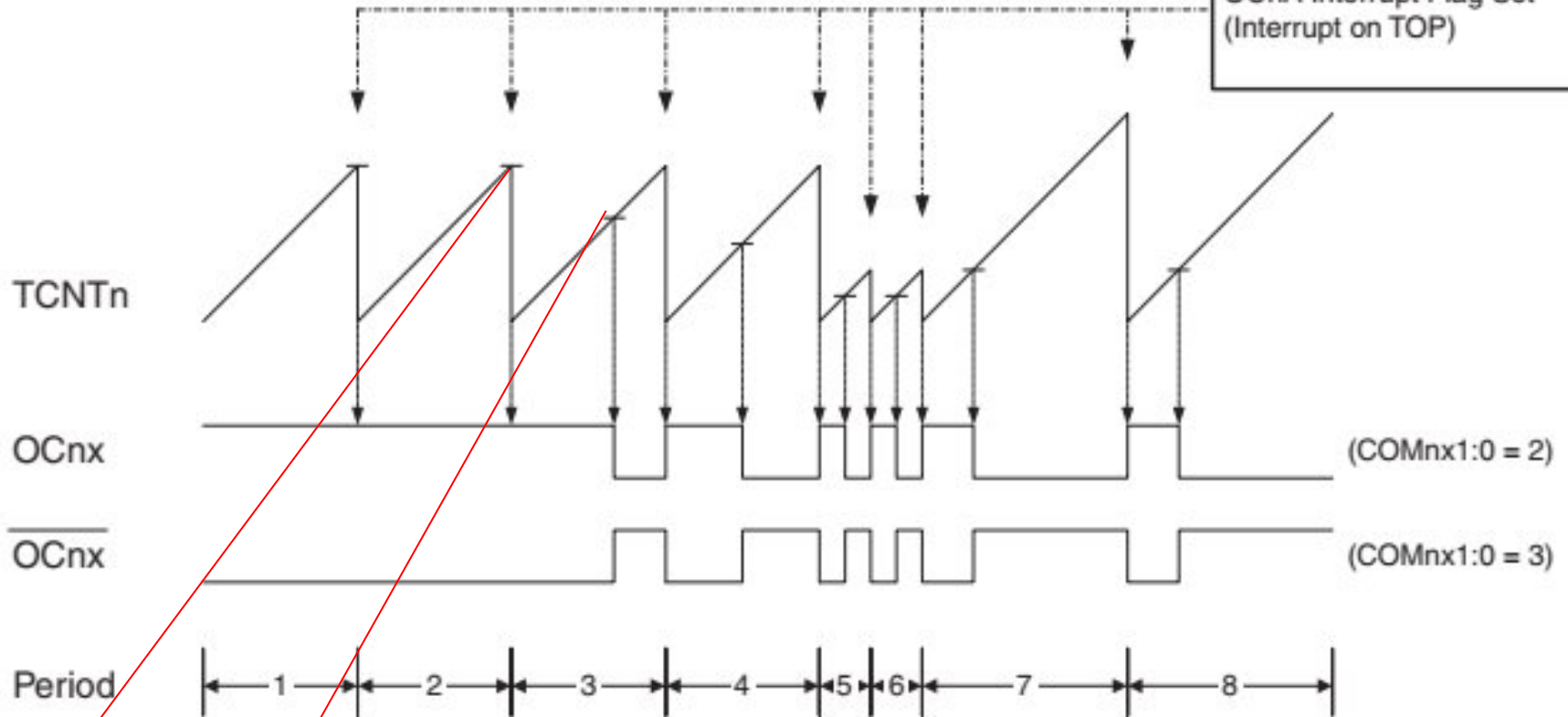
Figure 46. Fast PWM Mode, Timing Diagram

The PWM frequency for the output can be calculated by the following equation:

$$f_{OCnXPWM} = \frac{f_{clk_I/O}}{N \cdot (1 + TOP)}$$

The N variable represents the prescaler divider (1, 8, 64, 256, or 1024).

OCRn_x / TOP Update and
TOVn Interrupt Flag Set and
OCnA Interrupt Flag Set
OCnA Interrupt Flag Set
(Interrupt on TOP)



TCNT1 counts from BOTTOM to TOP then restarts from BOTTOM

Both TOP and Compare values can be changed/updated => change both duty cycle and frequency

Figure 46. Fast PWM Mode, Timing Diagram

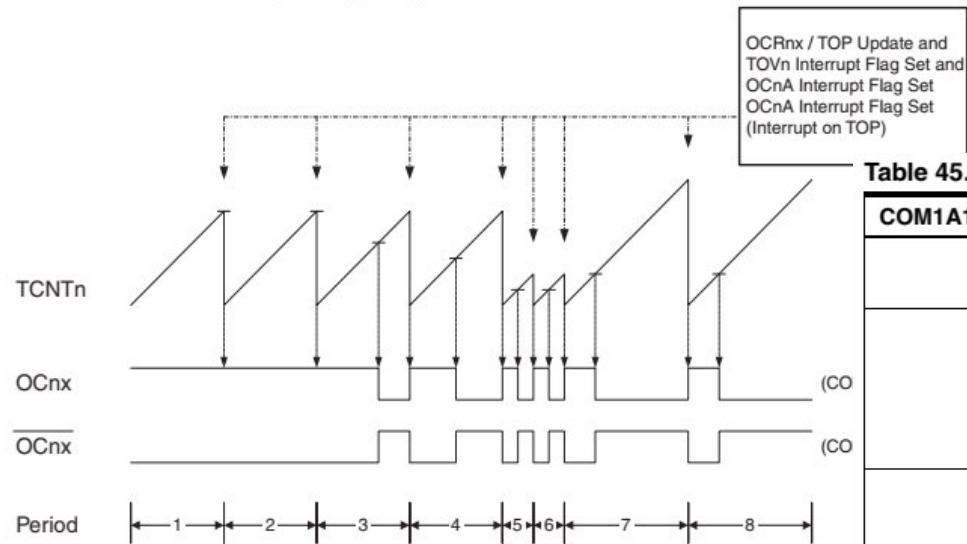


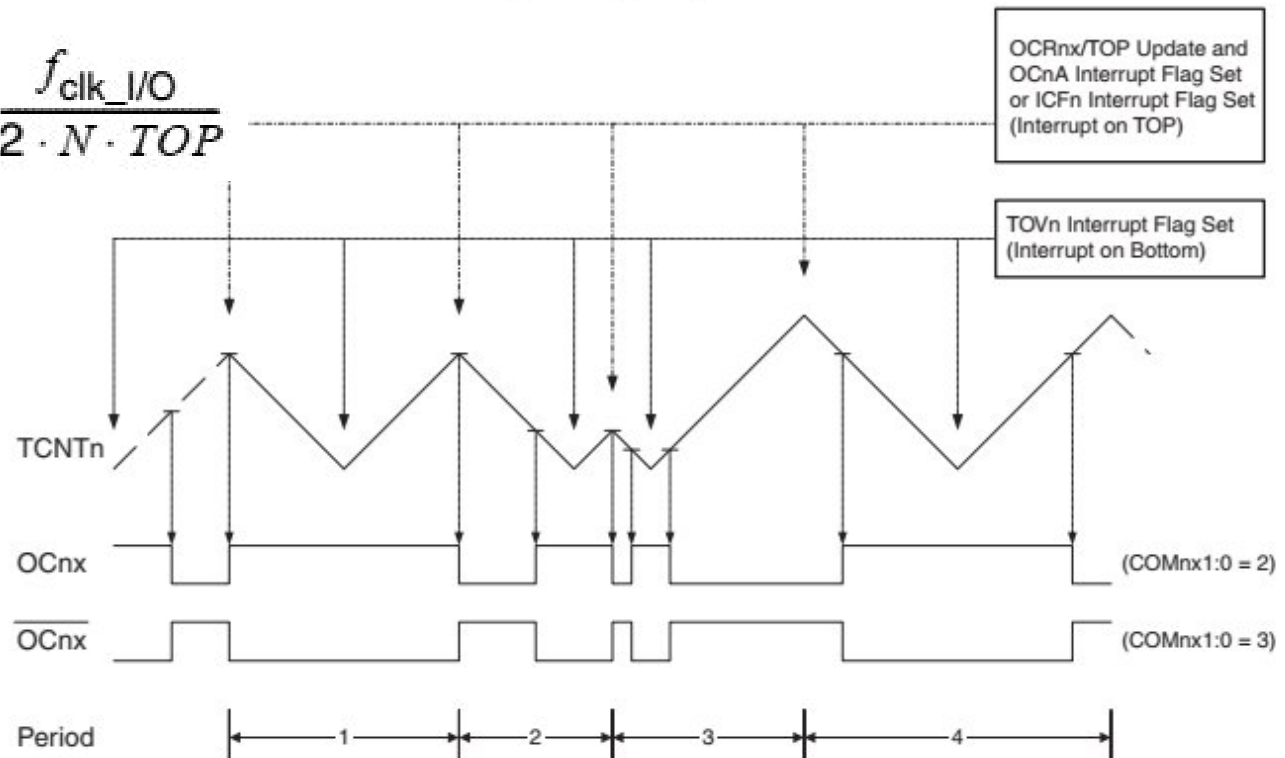
Table 45. Compare Output Mode, Fast PWM⁽¹⁾

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 15: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM13:0 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match, set OC1A/OC1B at BOTTOM, (non-inverting mode)
1	1	Set OC1A/OC1B on compare match, clear OC1A/OC1B at BOTTOM, (inverting mode)

- The Timer/Counter Overflow Flag (TOV1) is set each time the counter reaches TOP. In addition the OC1A or ICF1 Flag is set at the same timer clock cycle as TOV1 is set when either OCR1A or ICR1 is used for defining the TOP value. If one of the interrupts are enabled, the interrupt handler routine can be used for updating the TOP and compare values. When changing the TOP value the program must ensure that the new TOP value is higher or equal to the value of all of the Compare Registers. If the TOP value is lower than any of the Compare Registers, a compare match will never occur between the TCNT1 and the OCR1x.
- Note that when using fixed TOP values the unused bits are masked to zero when any of the OCR1x Registers are written.
- **Using the ICR1 Register for defining TOP works well when using fixed TOP values.** By using ICR1, the OCR1A Register is free to be used for generating a PWM output on OC1A. However, **if the base PWM frequency is actively changed (by changing the TOP value), using the OCR1A as TOP is clearly a better choice due to its double buffer feature**
- **The OCR1A Register is double buffered.** This feature allows the OCR1A I/O location to be written anytime. When **new** OCR1A I/O location is written **the new value written will be put into the OCR1A Buffer Register. The OCR1A Compare Register will then be updated with the value in the Buffer Register at the next timer clock cycle the TCNT1 matches TOP.** The update is done at the same timer clock cycle as the TCNT1 is cleared and the TOV1 Flag is set

Figure 47. Phase Correct PWM Mode, Timing Diagram

$$f_{OCnxPCPWM} = \frac{f_{clk_I/O}}{2 \cdot N \cdot TOP}$$



As Figure 47 shows the output generated is not symmetrical in all periods.

As third period shown in Figure 47 illustrates, changing the TOP actively while the Timer/Counter is running in the phase correct mode can result in an unsymmetrical output. The reason for this can be found in the time of update of the OCR1x Register. Since the OCR1x update occurs at TOP, the PWM period starts and ends at TOP. This implies that the length of the falling slope is determined by the previous TOP value, while the length of the rising slope is determined by the new TOP value. When these two values differ the two slopes of the period will differ in length. The difference in length gives the unsymmetrical result on the output.

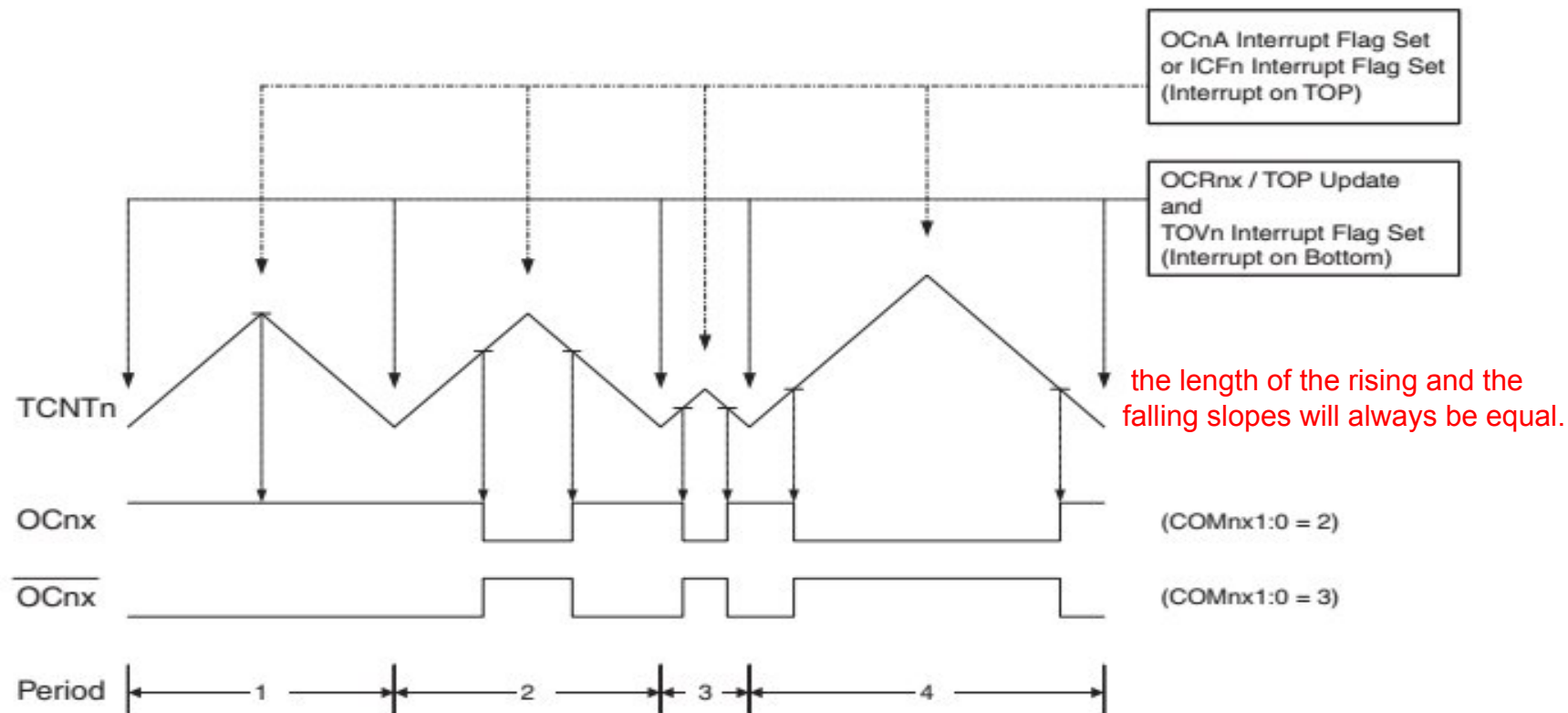
It is recommended to use the phase and frequency correct mode instead of the phase correct mode when changing the TOP value while Timer/Counter is running. When using a static TOP value there are practically no differences between 02 modes of operation.

Table 46. Compare Output Mode, Phase Correct and Phase and Frequency Correct PWM ⁽¹⁾

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 9 or 14: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM13:0 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match when up-counting. Set OC1A/OC1B on compare match when downcounting.
1	1	Set OC1A/OC1B on compare match when up-counting. Clear OC1A/OC1B on compare match when downcounting.

Note: 1. A special case occurs when OCR1A/OCR1B equals TOP and COM1A1/COM1B1 is set. [See “Phase Correct PWM Mode” on page 101.](#) for more details.

Figure 48. Phase and Frequency Correct PWM Mode, Timing Diagram



The main difference between the phase correct, and the phase and frequency correct PWM mode is the time the **OCR1x Register is updated** by the OCR1x Buffer Register

- Phase correct: at TOP => delay???
- Phase and frequency correct: updated at BOTTOM

As Figure 48 shows the output generated is, **in contrast to the phase correct mode**, symmetrical in all periods. Since the OCR1x Registers are updated at BOTTOM, the length of the rising and the falling slopes will always be equal. This gives symmetrical output pulses and is therefore frequency correct.

PWM: Phase Correct >< Phase & Freq Correct

- Differ from the Phase Correct PWM Mode, in Phase and Frequency Correct PWM Mode the Output Compare Register (**OCRnA** and **OCRnB**) is updated from the buffer when the Timer Counter Register (**TCNTn**) reaches BOTTOM instead of TOP in Phase Correct PWM Mode. The frequency could be change by changing the TOP value, here you could understand why we need to use the Phase and Frequency Correct PWM mode, because as we change the frequency and at the same time the PWM peripheral update the Output Compare register (**OCRnA** and **OCRnB**) then there will be a glitch in the PWM frequency signal.
- In Phase and Frequency Correct PWM mode because the Output Compare Register is updated at the BOTTOM therefore the rising and falling length of the PWM signal is always equal this result in frequency being corrected when we change the frequency on fly.

```

Init_Phase_Freq_Correct_PWM(void)
{
// Initial PORT Used
DDRD = 0b11111111;
// Set PORTD as Output
PORTD = 0b00000000;
DDRB = 0b11111110; // Set PB0 as Input and other as Output
PORTB = 0b00000000;

// Initial TIMER1 Phase and Frequency Correct PWM
// Set the Timer/Counter Control Register
TCCR1A = 0b11000000; // Set OC1A when up counting, Clear when down counting
TCCR1B = 0b00010000; // Phase/Freq-correct PWM, top value = ICR1, Prescaler: Off
}

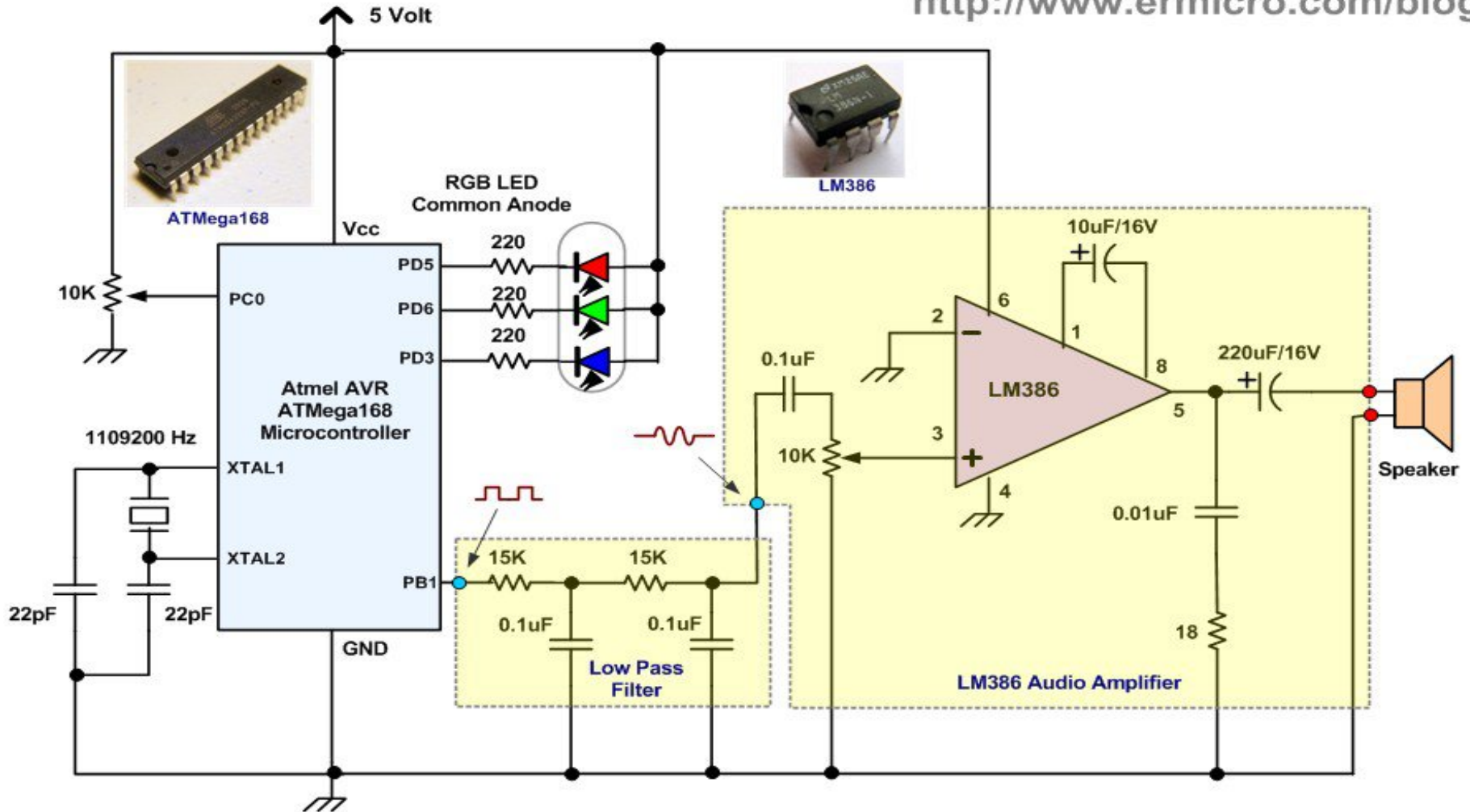
```

*The Phase and Frequency Correct PWM Mode feature is only available on TIMER1 (16-bit). Basically the Phase and Frequency Correct PWM mode use the same dual slope technique used in Phase Correct PWM mode to generate the PWM signal. **These two modes actually are identical if we never change the PWM signal frequency**, but if we need to change the PWM signal frequency on fly, then we need to use the AVR ATmega168 microcontroller Phase and Frequency Correct mode to generate the PWM signal.*

The RGB LED Light and Sound Show

Put all together the AVR ATmega168 basic PWM modes, making some interesting RGB LED and Sound show as shown on this following schematic

<http://www.ermicro.com/blog>



AVRJazz Mega168 Board RGB LED and Sound Show Project Simplified Schematic


```

volatile unsigned char duty_cyc_a,duty_cyc_b, duty_cyc_c,led_a,led_b,led_c; //Global variables
volatile unsigned int tempo;
Init_PWM(void)
{
    // Initial PORT Used
    DDRD = 0b11111111; // Set PORTD as Output
    PORTD = 0b00000000;
    DDRB = 0b11111110; // Set PB0 as Input and other as Output
    PORTB = 0b00000000;
    .....
    // Initial TIMER0 Fast PWM
    // Fast PWM Frequency = fclk / (N * 256), Where N is the prescaler
    // f_PWM = 11059200 / (64 * 256) = 675 Hz
    TCCR0A = 0b10100011; // Fast PWM 8 Bit, Clear OCA0/OCB0 on Compare Match, Set on TOP
    TCCR0B = 0b00000011; // Used 64 Prescaler
    TCNT0 = 0;          // Reset TCNT0
    OCR0A = 0;          // Initial the Output Compare register A & B
    OCR0B = 0;

    // Initial TIMER1 Phase and Frequency Correct PWM
    // Set the Timer/Counter Control Register
    TCCR1A = 0b11000000; // Set OC1A when up counting, Clear when down counting
    TCCR1B = 0b00010000; // Phase/Freq-correct PWM, top value = ICR1, Prescaler: Off

    TIMSK1 = (1<<TOIE1); // Enable Overflow Interrupt

    // Initial TIMER2 Phase Correct PWM Mode
    // Phase Correct PWM Frequency = fclk / (N * 512), Where N is the prescaler
    // f_PWM = 11059200 / (64 * 512) = 337.5 Hz
    TCCR2A = 0b00100001; // Fast PWM 8 Bit, Clear OC2B on Compare Match, Set on TOP
    TCCR2B = 0b00000011; // Used 64 Prescaler
    TCNT2 = 0;          // Reset TCNT2
    OCR2B = 0;          // Initial the Output Compare register A & B

    duty_cyc_a=(unsigned char) rand() % 255; led_a=1;
    duty_cyc_b=(unsigned char) rand() % 255; led_b=1;
    duty_cyc_c=(unsigned char) rand() % 255; led_c=1;

    sei();          // Enable Interrupt}
    .....
}

```

Summary of OC Normal Mode

- Good for generating **waveforms of any shapes**
- Programming: Use ISR to pre-set the timing of the next event
- Cons:
 - Interrupt overhead can be high
 - Cannot generate high-frequency waveforms
 - CPU cannot sleep into deep power-saving modes

Summary of CTC Mode

- Good for generating square waveform
- Programming
 - WGM=0100: Put TOP into OCRnA
 - WGM=1100: Put TOP into ICRn
- Cycle time is $2 \times (TOP + 1)$, one event for every TOP+1 cycles
- Caveats:
 - Borrow either OCRnA or ICRn to set TOP
 - TCNT reset to zero after reaching TOP

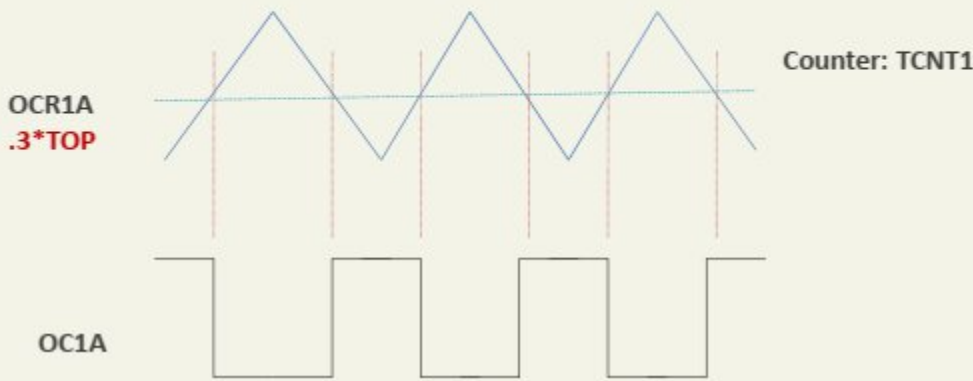
Summary of Fast PWM

- Good for generating Pulse Width Modulation Waveforms
- Two parameters: Pulse Width and Period Length (they decide the timing of two events)
- Programming with WGM = 1111
 - Top is OCRnA, stores Period_Length-1
 - OCRnB/OCRnC stores Pulse_Width-1
 - Caveats: OCRnA is borrowed to set TOP; TCNT reset to zero after reaching TOP

Correct Phase PWM

- Pulse Width Modulation Mode Counter is programmed in PWM mode through WGM bits and compare (COM1A) bits.
- A fixed TOP value is set (255, 511, 1023).
- Counter counts from BOTTOM (0) to TOP and then down repeatedly

- **OCR1x** register value is used to control the duty cycle.
- **OCR1x** better be less than **TOP**.



- **WGM11:WGM10:**
- **00: PWM disabled**
- **01: 8-bit: TOP-255**
- **10:9-bit: TOP-511**
- **11:10-bit: TOP-1023**

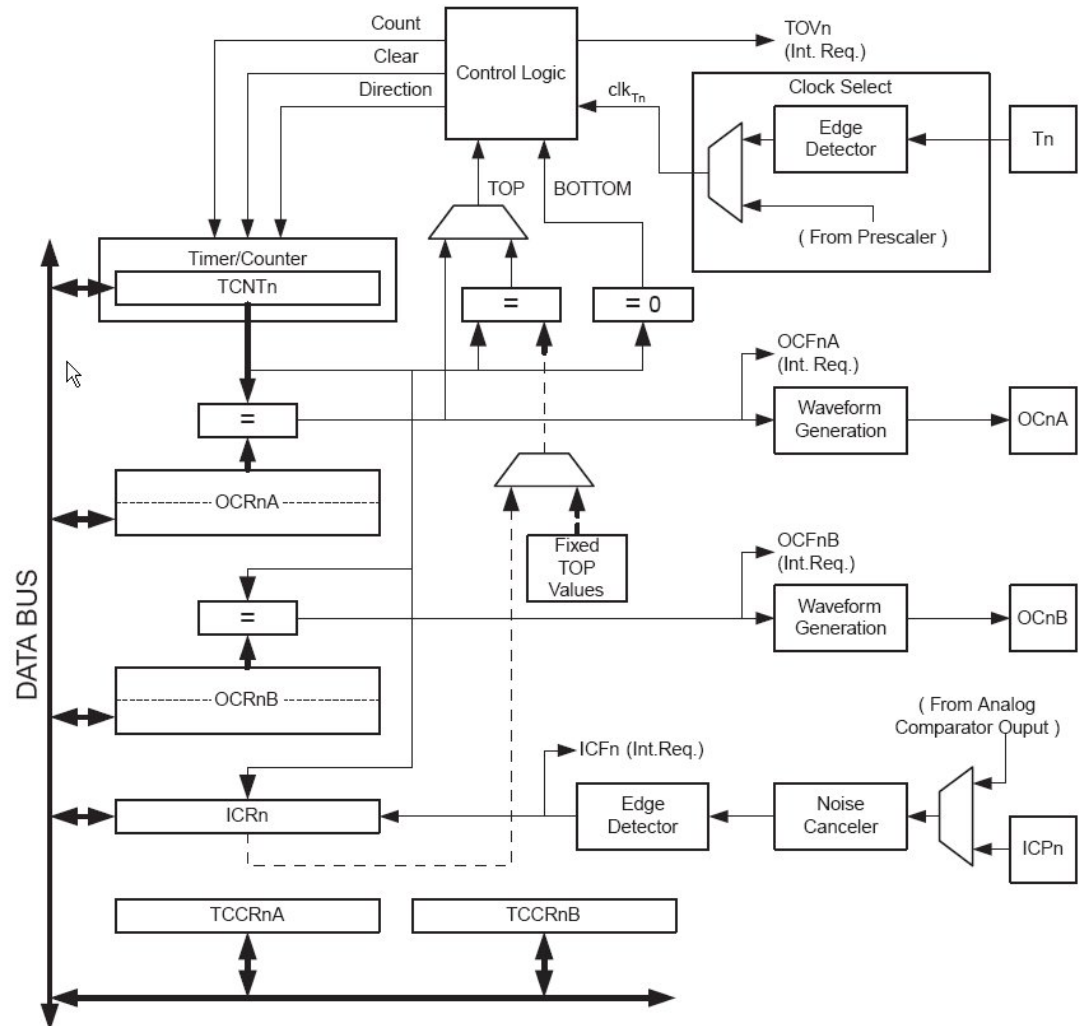
$$f_{PWM} = f_{system} / (2 * prescalar * TOP)$$

16 – bit Timer / Counter 1

- Two Independent Output Compare Units
 - can be used as Pulse Width Modulator (PWM)
 - Clear Timer on Compare Match (Auto Reload)
 - One Input Capture Unit
 - can be used as External Event Counter
 - Four Independent Interrupt Sources
TOV1, OCF1A, OCF1B, and ICF1
-

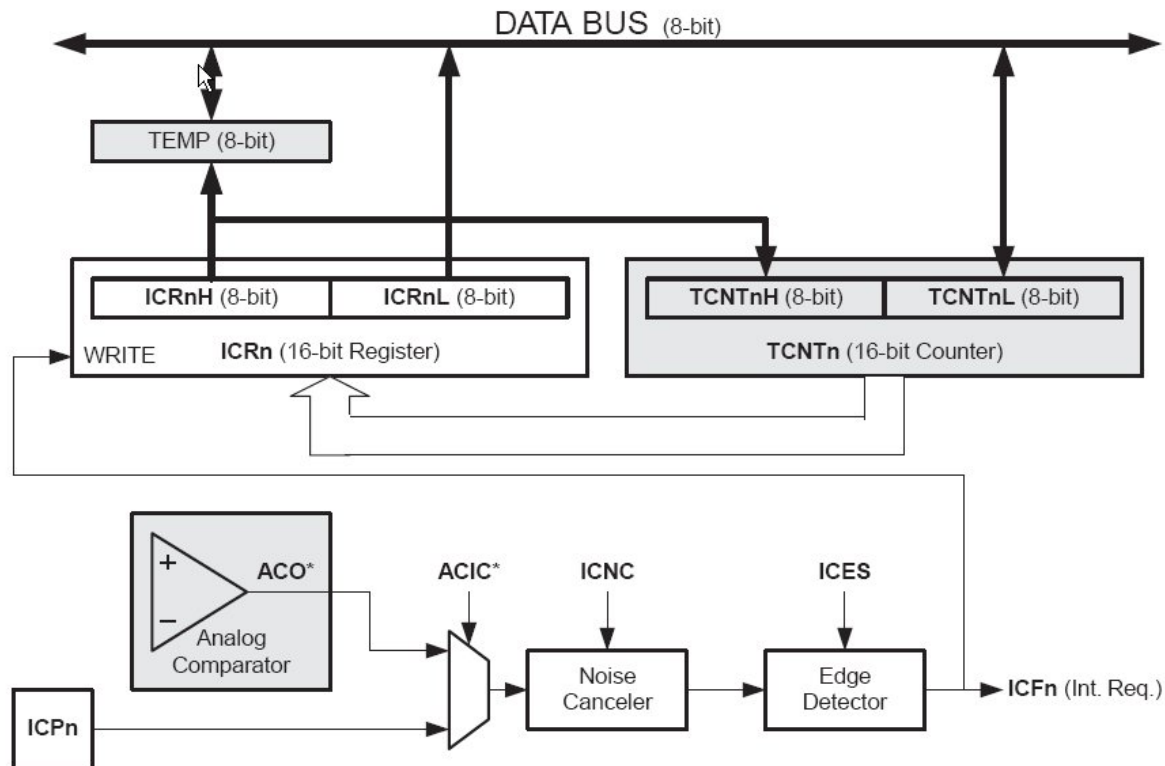
16 – bit Timer / Counter 1

- TCNT1, OCR1A, OCR1B and ICR1 are 16-bit registers
- write high byte first
read low byte first
(automatically done in C)
- clocking via prescaler or external clock on pin T1
- output compare register OCR1A/B can be used for PWM generation
(output pin: OC1A/B)
or for interrupt request



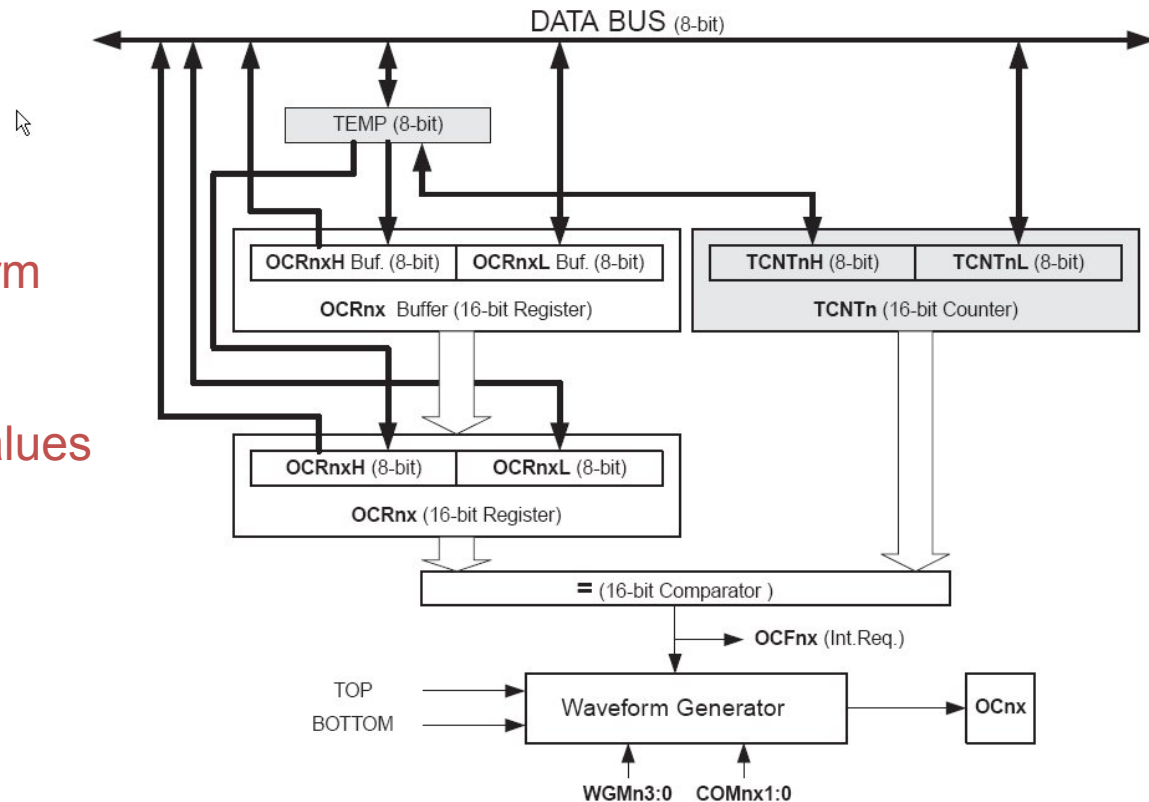
16 – bit Timer / Counter 1

- Input capture Unit: timestamp external events on ICP1 pin (or on Analog comp.)
Can generate Input Capture Interrupt



16 – bit Timer / Counter 1

- Output Compare Units: compare TCNT1 with OCR1A and OCR1B
Can generate Output Capture Interrupt



- 13 PWM / Waveform generation modes
- Top and Bottom values
- Auto reload

16 – bit Timer / Counter 1

- Mode configuration using TCCR1A and TCCR1B

Bit	7	6	5	4	3	2	1	0	
	COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	W	W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

PWM Mode, set/clear/toggle of OC1A/B, auto-reload, Noise canceller

Find Details in the ATmega8 datasheet !

16 – bit Timer / Counter 1

- Clock source configuration using TCCR1B

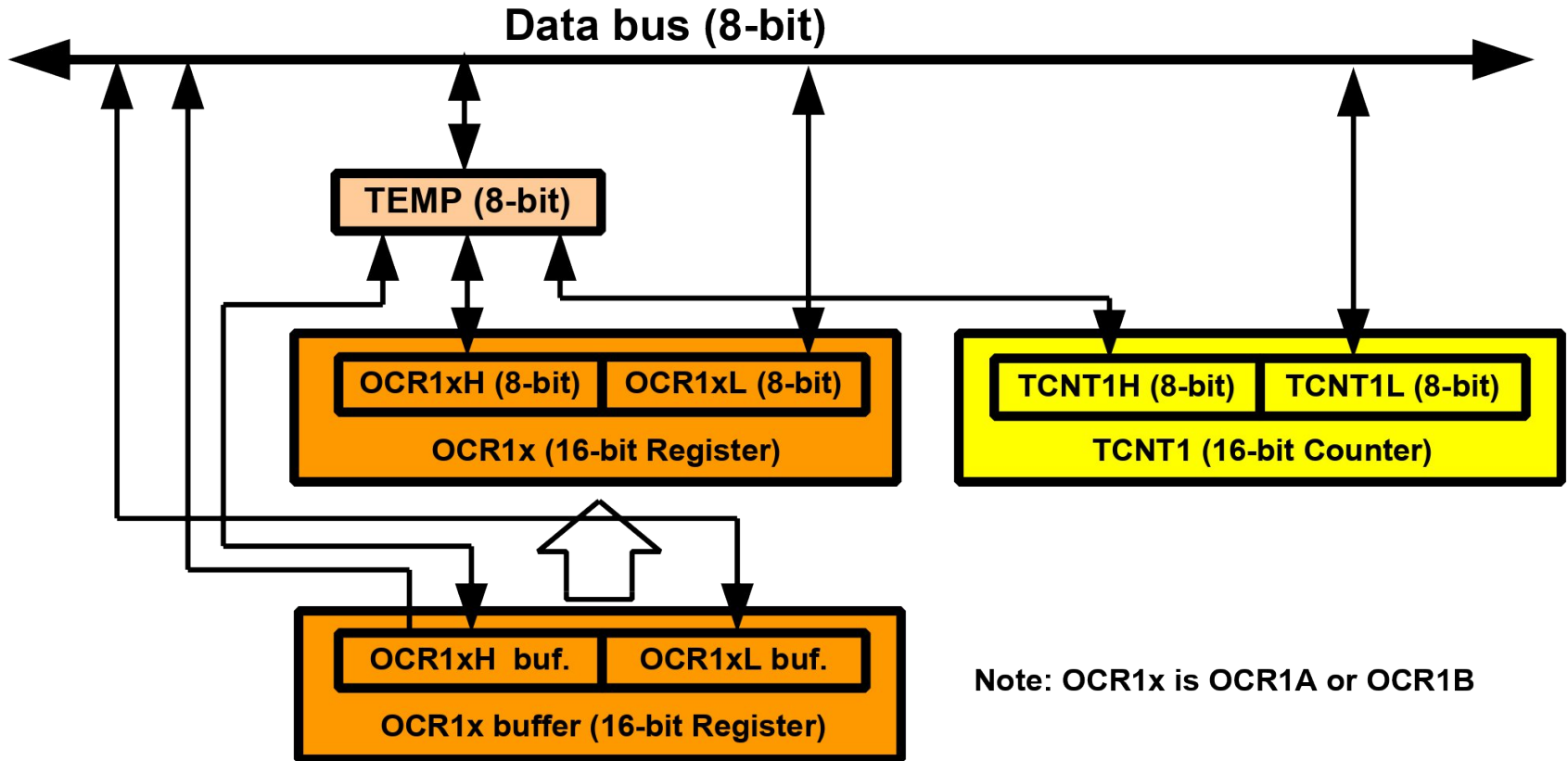
CS12	CS11	CS10	Description
0	0	0	No clock source. (Timer/Counter stopped)
0	0	1	$\text{clk}_{\text{I/O}}/1$ (No prescaling)
0	1	0	$\text{clk}_{\text{I/O}}/8$ (From prescaler)
0	1	1	$\text{clk}_{\text{I/O}}/64$ (From prescaler)
1	0	0	$\text{clk}_{\text{I/O}}/256$ (From prescaler)
1	0	1	$\text{clk}_{\text{I/O}}/1024$ (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Assuming XTAL = 10 MHz write a program that toggles PB5 once per millisecond, using Normal mode.

```
.INCLUDE "M32DEF.INC"
    LDI    R16,HIGH(RAMEND)    ;init stack pointer
    OUT    SPH,R16
    LDI    R16,LOW(RAMEND)
    OUT    SPL,R16
    SBI    DDRB,5              ;PB5 as an output
BEGIN:SBI    PORTB,5           ;PB5 = 1
    RCALL  DELAY_1ms
    CBI    PORTB,5           ;PB5 = 0
    RCALL  DELAY_1ms
    RJMP   BEGIN

DELAY_1ms:
    LDI    R20,0xD8
    OUT    TCNT1H,R20        ;TEMP = 0xD8
    LDI    R20,0xF0
    LDI    R20,HIGH(-10000)
    OUT    TCNT1H,R20
    LDI    R20,LOW(-10000)
    OUT    TCNT1L,R20        ;Timer1 overflows after 10000 machine cycles
AGAIN:IN    R20,TIFR          ;read TIFR
    SBRS   R20,TOV1          ;if OCF1A is set skip next instruction
    RJMP   AGAIN
    LDI    R20,1<<TOV1
    OUT    TIFR,R20          ;clear TOV1 flag
    LDI    R19,0
    OUT    TCCR1B,R19        ;stop timer
    OUT    TCCR1A,R19        ;
    RET
```

TEMP register



Note: OCR1x is OCR1A or OCR1B

```
LDI R20, TCNT1L
OUT R20, TCNT1L
LDI R20, 0x53
OUT TCNT1L, R20
```

```
TCNT1H = 0xF3;
TCNT1L = 0x53;
```


Assuming XTAL = 10 MHz write a program that toggles PB5 once per millisecond, using CTC mode.

```
.INCLUDE "M32DEF.INC"
    LDI    R16,HIGH(RAMEND)
    OUT    SPH,R16
    LDI    R16,LOW(RAMEND)
    OUT    SPL,R16
    SBI    DDRB,5                ;PB5 as an output
BEGIN: SBI    PORTB,5            ;PB5 = 1
    RCALL   DELAY_1ms
    CBI    PORTB,5              ;PB5 = 0
    RCALL   DELAY_1ms
    RJMP    BEGIN

DELAY_1ms:
    LDI    R20,0x00
    OUT    TCNT1H,R20           ;TEMP = 0
    OUT    TCNT1L,R20           ;TCNT1L = 0, TCNT1H = TEMP

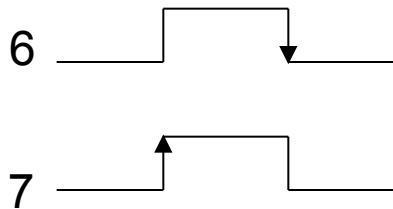
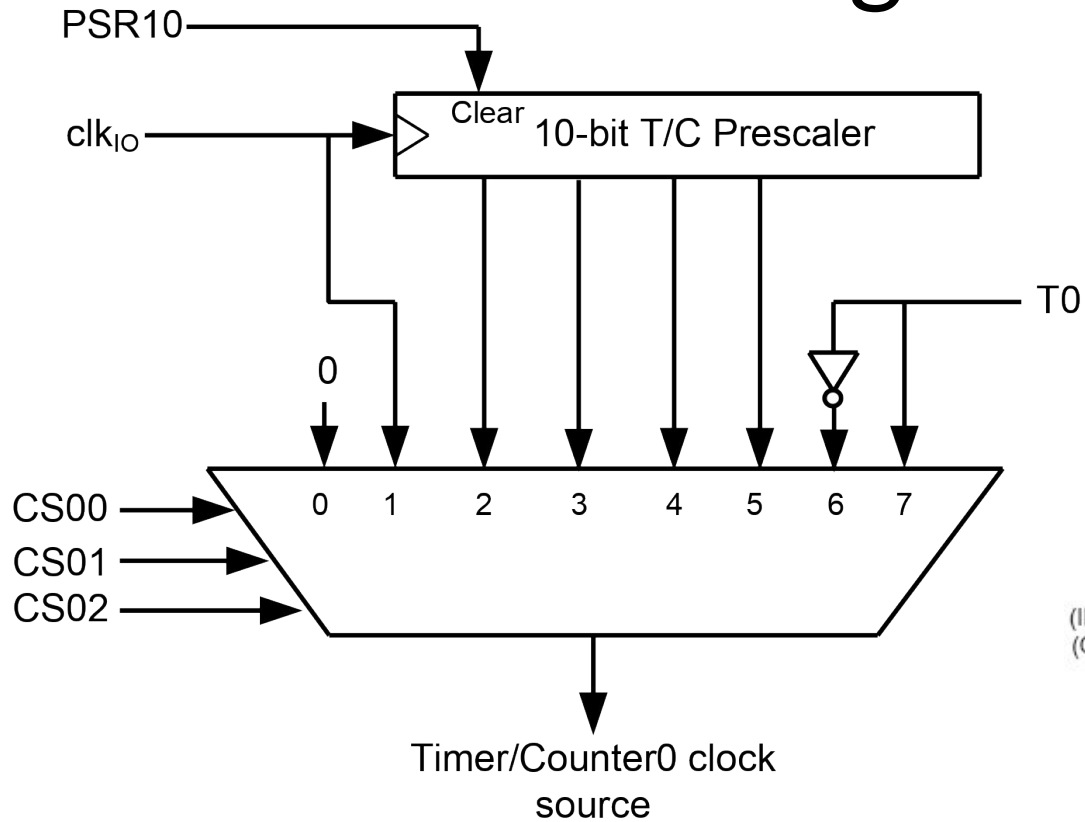
    LDI    R20,0x27
    OUT    OCR1AH,R20           ;TEMP = 0x27
    LDI    R20,0x0F
    OUT    OCR1AL,R20           ;OCR1AL = 0x0F, OCR1AH = TEMP

    LDI    R20,0x0
    OUT    TCCR1A,R20           ;WGM11:10=00
    LDI    R20,0x09
    OUT    TCCR1B,R20           ;WGM13:12=01,CS=CLK

AGAIN: IN     R20,TIFR           ;read TIFR
    SBRS   R20,OCF1A           ;if OCF1A is set skip next instruction
    RJMP    AGAIN
    LDI    R20,1<<OCF1A
    OUT    TIFR,R20            ;clear OCF1A flag
    LDI    R19,0
    OUT    TCCR1B,R19          ;stop timer
    OUT    TCCR1A,R19
    RET
```

Counting

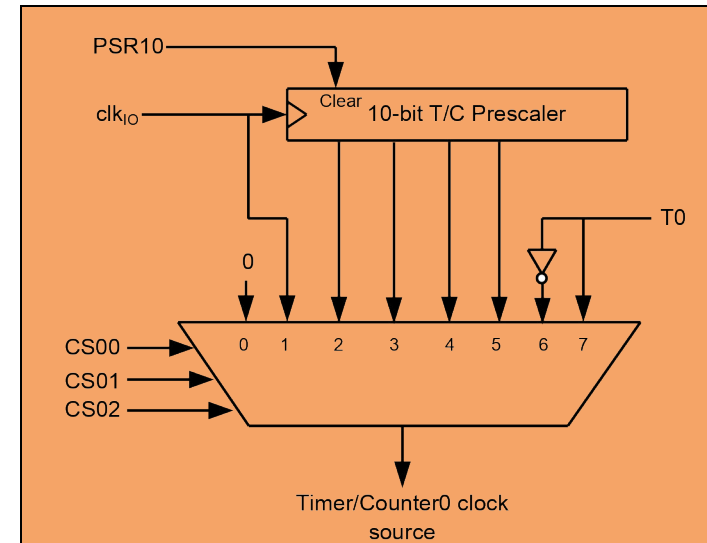
Counting



(XCK/T0) PB0	1	40	PA0 (ADC0)
(T1) PB1	2	39	PA1 (ADC1)
(INT2/AIN0) PB2	3	38	PA2 (ADC2)
(OC0/AIN1) PB3	4	37	PA3 (ADC3)
(SS) PB4	5	36	PA4 (ADC4)
(MOSI) PB5	6	35	PA5 (ADC5)
(MISO) PB6	7	34	PA6 (ADC6)
(SCK) PB7	8	33	PA7 (ADC7)
RESET	9	32	AREF
VCC	10	31	GND
GND	11	30	AVCC
XTAL2	12	29	PC7 (TOSC2)
XTAL1	13	28	PC6 (TOSC1)
(RXD) PD0	14	27	PC5 (TDI)
(TXD) PD1	15	26	PC4 (TDO)
(INT0) PD2	16	25	PC3 (TMS)
(INT1) PD3	17	24	PC2 (TCK)
(OC1B) PD4	18	23	PC1 (SDA)
(OC1A) PD5	19	22	PC0 (SCL)
(ICP) PD6	20	21	PD7 (OC2)

Example Assuming that clock pulses are fed into pin T0, write a program for counter 0 in normal mode to count the pulses on falling edge and display the state of the TCNT0 count on PORTC.

```
.INCLUDE "M32DEF.INC"
CBI   DDRB,0           ;make T0 (PB0) input
LDI   R20,0xFF
OUT   DDRC,R20         ;make PORTC output
LDI   R20,0x06
OUT   TCCR0,R20        ;counter, falling edge
AGAIN:
IN     R20,TCNT0
OUT   PORTC,R20        ;PORTC = TCNT0
IN     R16,TIFR
SBRS  R16,TOV0
RJMP  AGAIN           ;keep doing it
LDI   R16,1<<TOV0
OUT   TIFR, R16
RJMP  AGAIN           ;keep doing it
```



FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00
------	-------	-------	-------	-------	------	------	------

TCCR0

Assuming that clock pulses are fed into pin T1. Write a program for counter 1 in CTC mode to make PORTC.0 high every 100 pulses.

```
.INCLUDE "M32DEF.INC"

CBI    DDRB,1           ;make T1 (PB1) input
SBI    DDRC,0           ;PC0 as an output
LDI    R16, 0x01
LDI    R17, 0

LDI    R20, 0x0
OUT    TCCR1A, R20
LDI    R20, 0x0E
OUT    TCCR1B, R20      ;CTC, counter, falling edge
AGAIN: LDI    R20, 0
OUT    OCR1AH, R20      ;TEMP = 0
LDI    R20, 99
OUT    OCR1AL, R20      ;ORC1L = R20, OCR1H = TEMP
L1: IN    R20, TIFR
SBR    R20, OCF1A
RJMP   L1               ;keep doing it
LDI    R20, 1<<OCF1A    ;clear OCF1A flag
OUT    TIFR, R20

EOR    R17, R16
OUT    PORTC, R17        ;SBI    PORTC, 0           ;PC0 = 1
                        ;CBI    PORTC, 0           ;PC0 = 0

RJMP   AGAIN            ;keep doing it
```

Interrupt

Chapter 10

Contents

- Polling Vs. interrupt
- Interrupt unit
- Steps in executing an interrupt
- Edge trigger Vs. Level trigger in external interrupts
- Timer interrupt
- Interrupt priority
- Interrupt inside an interrupt
- Task switching and resource conflict
- C programming

Polling Vs. Interrupt

- Polling

- Ties down the CPU

- Interrupt

- Efficient CPU use
 - Has priority
 - Can be masked

```
while (true)
{
    if(PIND.2 == 0)
        //do something;
}
```

```
main( )
{
    Do your common task
}
```

whenever PIND.2 is 0 then
do something

Real Time Systems Interrupts

Real Time Systems

" A real-time system is one in which the correctness of the computations not only depends upon the logical correctness of the computation but also upon the time at which the result is produced. If the timing constraints of the system are not met, system failure is said to have occurred. " !

! Donald Gillies, UBC!

- POSIX 1003.1 states: "Realtime in operating systems: the ability of the operating system to provide a required level of service in a bounded response time"!
- There are two major classifications of Real Time:!
 - Hard Real Time is when a task not being completed at the required time causes the operation to fail!
 - Example: A bottle arrives late to the filling nozzle in a bottling plant, causing the product to be deposited on the conveyor belt and not into the bottle!
 - Soft Real Time is when a task not being completed at the desired time causes, for example, a decrease in system efficiency, but does not result in total failure!
 - Example: A bottle arrives early to the filling nozzle, causing the bottle to have to wait until the nozzle can fill it!

Interrupts

- How do we stop whatever else the processor is doing at the appropriate time to “focus” on the time-critical task?!
- Interrupts!!
 - low-level CPU function!
 - External stimulus to the active computations!
 - Interrupt Request or IRQ!
 - Causes the CPU to halt the current execution sequence and jump the program counter to a specific address!
 - Address is called the Interrupt Vector!
 - Interrupt Vector points to a section of your code that performs whatever actions are required upon that specific IRQ!
 - This function is called a handler or Interrupt Service Routine (ISR)! ▪ Upon completion of the ISR, the CPU resumes normal program execution from where it left off!

Interrupts - Context Switch

- How does the processor switch to the ISR?!
 - Move the CPU working registers and Program Counter into RAM, !
 - Repeat above step in reverse to restore the CPU to the pre-IRQ state once the ISR completes!
 - This entire process takes a non-trivial amount of time, sometimes on the order of 10s-100s of CPU cycles. See the datasheet for more details.!
 - Sometimes the CPU hardware will assist with the context switch, but often the software is responsible!
 - The AVR-GCC compiler implements the required software automatically in its libraries.!
- IRQ Sources!
 - Internal Timers (compared to a specific value or upon overflow)!
 - External IRQ (a special Input pin function)!
 - Other Peripherals (UART, I2C, SPI, ADC, Comparator, etc)!

Interrupt Preemption

- It is possible that on a particular system there would be multiple real-time tasks, with some tasks being more time critical than others!
- Simple Buoy example:!
 - The ADC can interrupt when its buffer is full, requiring immediate attention to avoid data loss (hard real time)!
 - The compact flash can also interrupt when it is ready for more data to write, however this is a soft real time task.!
 - If the ADC signaled while the ISR was running for the compact flash, you would want:!
 - to run the ISR for the ADC immediately! ▪
 - then return to the compact flash ISR! ▪ and
 - then resume normal operation.!

Interrupt Preemption

- This is called preemption, and can get extremely complex!
 - Some processors have a hierarchy of IRQ priorities, which can specify which IRQ preempt which ISRs!
 - Other processors do not have this feature, making preemption quite difficult.!
 - Some processors will automatically disable all interrupts while one ISR is running (like the AVR Series). In this case, you must manually re-enable global interrupts at the beginning of the ISR for preemption to be possible at all!
- Preemption is often required for real time systems that have both soft and hard components, as well as those with multiple interrupt sources of non-equivalent priority!

Chain of Events on Interrupt

- Finish executing current instruction
 - Disable all interrupts **CLI**
 - Push program counter on to stack
 - Jump to interrupt vector table
 - Jump to start of complete ISR
 - Save any context that ISR may otherwise change
 - Registers and flags must be saved within ISR and restored before it returns – this is very important!
 - Re-enable interrupts if nested interrupts are ok
 - Complete ISR's code
 - Re-enable interrupts upon return
 - Jump back to next instruction before interruption
-
- Automatic
- Compiler
- SEI
- RETI

Interrupt Processing

- **several Interrupt Sources:**
External Interrupts, Timer, Bus-Peripherals, ADC, EEPROM
- **individual Interrupt-Enable bits in the SFR's**
- **global interrupt enable Bit in SREG,**
set with `sei()` and clear with `cli()` instruction
- **flagged (remembered) and non-flagged interrupt sources**
- **lowest addresses in program memory reserved for the interrupt vector table**
- **higher priority interrupts have lower addresses**

Shared Data Problem

- When you use interrupts you create the opportunity for multiple sections of code to update a variable.
- This might cause a problems in your logic if an interrupt updates a variable between two lines of code that are directly dependent on each other (e.g. if statement)
- One solution is to create critical sections where you disable the interrupts for a short period of time while you complete your logic on the shared variable

```
cli();  
.....critical section code goes here.....  
sei();
```

Reset-Vector and Interrupt-Vectors

- Word addresses 0, 1 – 19 in Flash Ram
- When a reset or interrupt occurs, the CPU calls the address
- Install an Interrupt Handler: modify the vector table to jump to your user-handler
- return from interrupt: reti

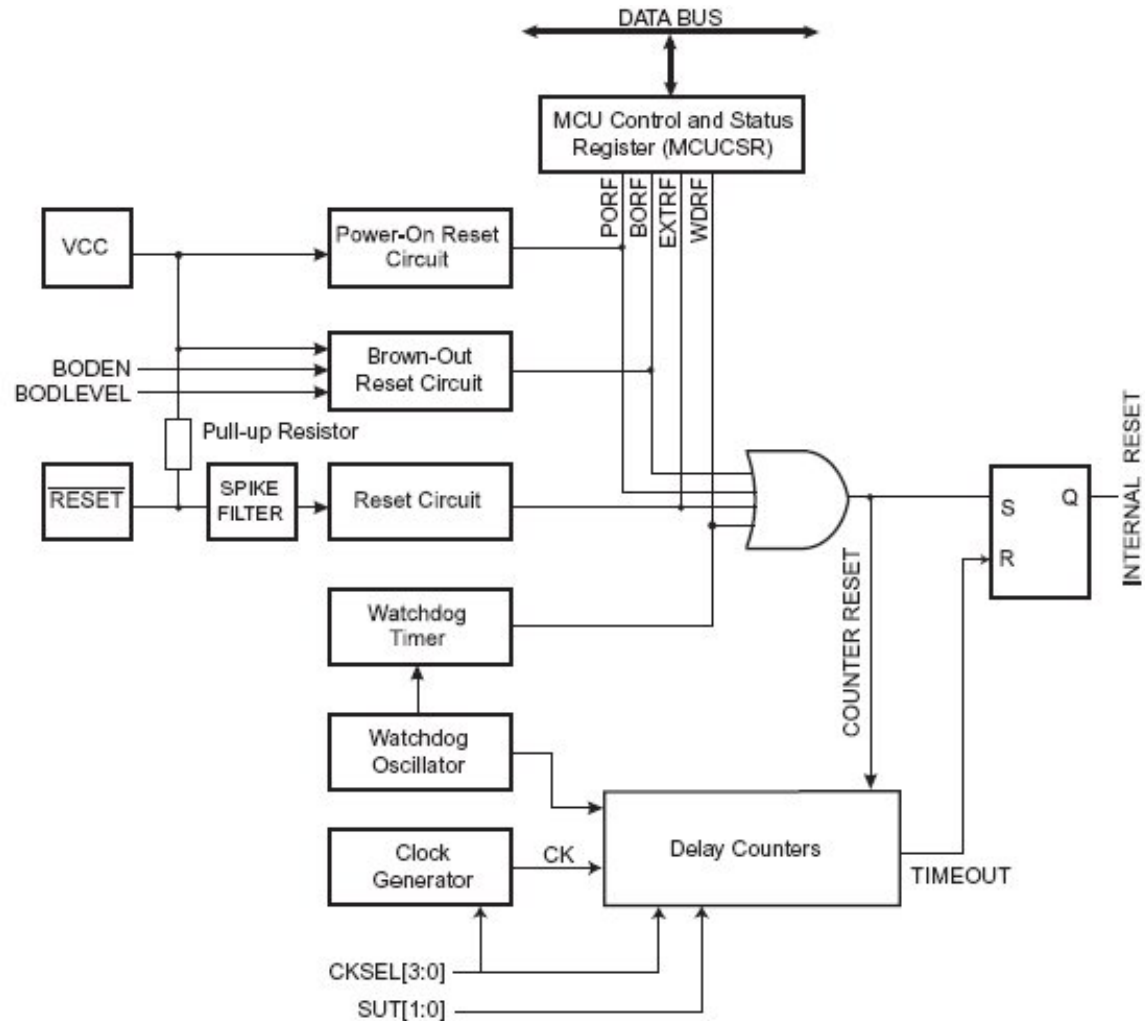
Vector No.	Program Address ⁽²⁾	Source	Interrupt Definition
1	0x000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset, and Watchdog Reset
2	0x001	INT0	External Interrupt Request 0
3	0x002	INT1	External Interrupt Request 1
4	0x003	TIMER2 COMP	Timer/Counter2 Compare Match
5	0x004	TIMER2 OVF	Timer/Counter2 Overflow
6	0x005	TIMER1 CAPT	Timer/Counter1 Capture Event
7	0x006	TIMER1 COMPA	Timer/Counter1 Compare Match A
8	0x007	TIMER1 COMPB	Timer/Counter1 Compare Match B
9	0x008	TIMER1 OVF	Timer/Counter1 Overflow
10	0x009	TIMER0 OVF	Timer/Counter0 Overflow
11	0x00A	SPI, STC	Serial Transfer Complete
12	0x00B	USART, RXC	USART, Rx Complete
13	0x00C	USART, UDRE	USART Data Register Empty
14	0x00D	USART, TXC	USART, Tx Complete
15	0x00E	ADC	ADC Conversion Complete
16	0x00F	EE_RDY	EEPROM Ready
17	0x010	ANA_COMP	Analog Comparator
18	0x011	TWI	Two-wire Serial Interface
19	0x012	SPM_RDY	Store Program Memory Ready

Reset-Vector and Interrupt-Vectors

- example shows full featured vector table
- 19 handlers installed
- program execution after reset:
jmp RESET (\$013)
- Main program is located at \$013, beyond the vectors

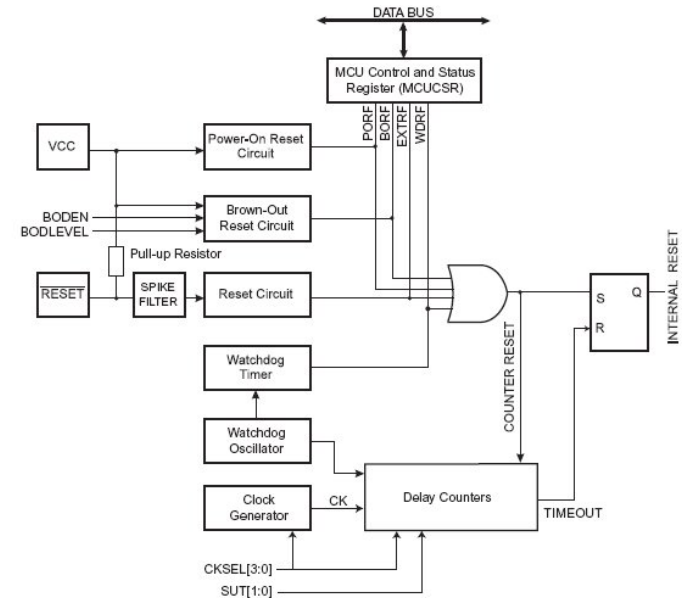
address	Labels	Code	Comments
\$000		rjmp RESET	; Reset Handler
\$001		rjmp EXT_INT0	; IRQ0 Handler
\$002		rjmp EXT_INT1	; IRQ1 Handler
\$003		rjmp TIM2_COMP	; Timer2 Compare Handler
\$004		rjmp TIM2_OVF	; Timer2 Overflow Handler
\$005		rjmp TIM1_CAPT	; Timer1 Capture Handler
\$006		rjmp TIM1_COMPA	; Timer1 CompareA Handler
\$007		rjmp TIM1_COMPB	; Timer1 CompareB Handler
\$008		rjmp TIM1_OVF	; Timer1 Overflow Handler
\$009		rjmp TIM0_OVF	; Timer0 Overflow Handler
\$00a		rjmp SPI_STC	; SPI Transfer Complete Handler
\$00b		rjmp USART_RXC	; USART RX Complete Handler
\$00c		rjmp USART_UDRE	; UDR Empty Handler
\$00d		rjmp USART_TXC	; USART TX Complete Handler
\$00e		rjmp ADC	; ADC Conversion Complete Handler
\$00f		rjmp EE_RDY	; EEPROM Ready Handler
\$010		rjmp ANA_COMP	; Analog Comparator Handler
\$011		rjmp TWSI	; Two-wire Serial Interface Handler
\$012		rjmp SPM_RDY	; Store Program Memory Ready Handler
		;	
\$013	RESET:	ldi r16,high(RAMEND)	; Main program start
\$014		out SPH,r16	; Set Stack Pointer to top of RAM
\$015		ldi r16,low(RAMEND)	
\$016		out SPL,r16	
\$017		sei	; Enable interrupts
\$018		<instr> xxx	
...		...	...

AVR Reset Sources:



Reset Sources:

- **Power-on Reset:** supply voltage is below the Power-on Reset threshold
- **External Reset:** low level is present on /RESET – input pin
- **Watchdog Reset:** Watchdog Timer enabled and period expires
- **Brown-out Reset:** Brown-out Detector enabled and supply voltage below threshold



Bit	7	6	5	4	3	2	1	0	
	–	–	–	–	WDRF	BORF	EXTRF	PORF	MCUCSR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	See Bit Description				

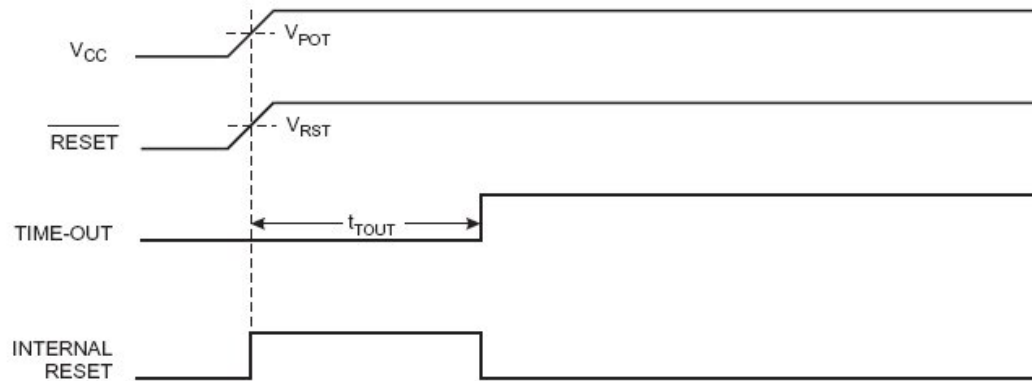
MCUCSR provides information on which reset source caused a CPU reset

Reset Voltage Thresholds

Symbol	Parameter	Condition	Min	Typ	Max	Units
V_{POT}	Power-on Reset Threshold Voltage (rising) ⁽¹⁾			1.4	2.3	V
	Power-on Reset Threshold Voltage (falling)			1.3	2.3	V
V_{RST}	$\overline{RESET}$ Pin Threshold Voltage		0.1		0.9	V_{CC}
t_{RST}	Minimum pulse width on $\overline{RESET}$ Pin				1.5	μs
V_{BOT}	Brown-out Reset Threshold Voltage ⁽²⁾	BODLEVEL = 1	2.4	2.6	2.9	V
		BODLEVEL = 0	3.7	4.0	4.5	
t_{BOD}	Minimum low voltage period for Brown-out Detection	BODLEVEL = 1		2		μs
		BODLEVEL = 0		2		μs
V_{HYST}	Brown-out Detector hysteresis			130		mV

Reset Voltage Thresholds:

Example: Power-on Reset



Example: Brown Out Reset

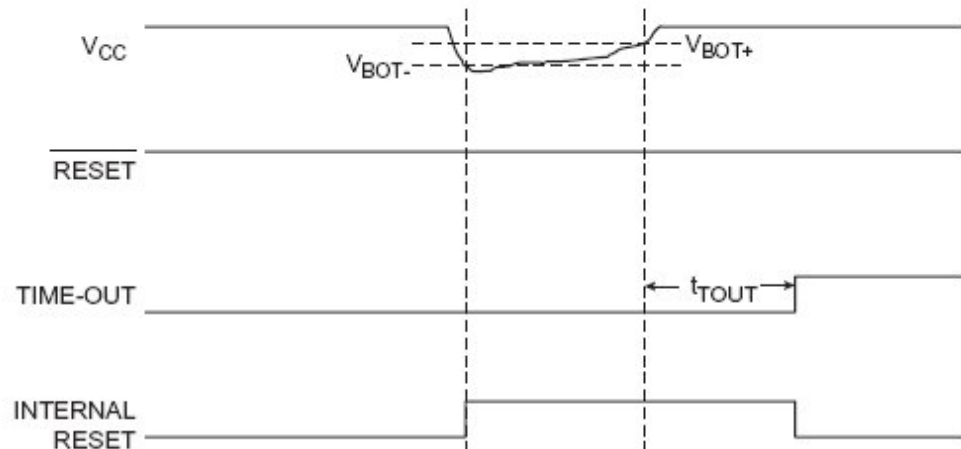
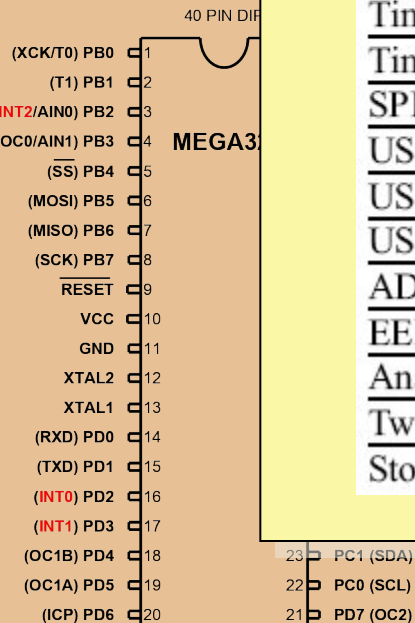


Table 10-1: Interrupt Vector Table for the Mega32

Interrupt	ROM Location (Hex)
Reset	0000
External Interrupt request 0	0002
External Interrupt request 1	0004
External Interrupt request 2	0006
Time/Counter2 Compare Match	0008
Time/Counter2 Overflow	000A
Time/Counter1 Capture Event	000C
Time/Counter1 Compare Match A	000E
Time/Counter1 Compare Match B	0010
Time/Counter1 Overflow	0012
Time/Counter0 Compare Match	0014
Time/Counter0 Overflow	0016
SPI Transfer complete	0018
USART, Receive complete	001A
USART, Data Register Empty	001C
USART, Transmit Complete	001E
ADC Conversion complete	0020
EEPROM ready	0022
Analog Comparator	0024
Two-wire Serial Interface	0026
Store Program Memory Ready	0028



Example: AtMega128 Interrupts

Table 23. Reset and Interrupt Vectors

Vector No.	Program Address ⁽²⁾	Source	Interrupt Definition
1	\$0000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset, and JTAG AVR Reset
2	\$0002	INT0	External Interrupt Request 0
3	\$0004	INT1	External Interrupt Request 1
4	\$0006	INT2	External Interrupt Request 2
5	\$0008	INT3	External Interrupt Request 3
6	\$000A	INT4	External Interrupt Request 4
7	\$000C	INT5	External Interrupt Request 5
8	\$000E	INT6	External Interrupt Request 6
9	\$0010	INT7	External Interrupt Request 7
10	\$0012	TIMER2 COMP	Timer/Counter2 Compare Match
11	\$0014	TIMER2 OVF	Timer/Counter2 Overflow
12	\$0016	TIMER1 CAPT	Timer/Counter1 Capture Event
13	\$0018	TIMER1 COMPA	Timer/Counter1 Compare Match A
14	\$001A	TIMER1 COMPB	Timer/Counter1 Compare Match B
15	\$001C	TIMER1 OVF	Timer/Counter1 Overflow
16	\$001E	TIMER0 COMP	Timer/Counter0 Compare Match
17	\$0020	TIMER0 OVF	Timer/Counter0 Overflow
18	\$0022	SPI, STC	SPI Serial Transfer Complete
19	\$0024	USART0, RX	USART0, Rx Complete
20	\$0026	USART0, UDRE	USART0 Data Register Empty
21	\$0028	USART0, TX	USART0, Tx Complete
22	\$002A	ADC	ADC Conversion Complete
23	\$002C	EE READY	EEPROM Ready
24	\$002E	ANALOG COMP	Analog Comparator
25	\$0030 ⁽³⁾	TIMER1 COMPC	Timer/Counter1 Compare Match C
26	\$0032 ⁽³⁾	TIMER3 CAPT	Timer/Counter3 Capture Event
27	\$0034 ⁽³⁾	TIMER3 COMPA	Timer/Counter3 Compare Match A
28	\$0036 ⁽³⁾	TIMER3 COMPB	Timer/Counter3 Compare Match B
29	\$0038 ⁽³⁾	TIMER3 COMPC	Timer/Counter3 Compare Match C
30	\$003A ⁽³⁾	TIMER3 OVF	Timer/Counter3 Overflow

Vector No.	Program Address ⁽²⁾	Source	Interrupt Definition
31	\$003C ⁽³⁾	USART1, RX	USART1, Rx Complete
32	\$003E ⁽³⁾	USART1, UDRE	USART1 Data Register Empty
33	\$0040 ⁽³⁾	USART1, TX	USART1, Tx Complete
34	\$0042 ⁽³⁾	TWI	Two-wire Serial Interface
35	\$0044 ⁽³⁾	SPM READY	Store Program Memory Ready

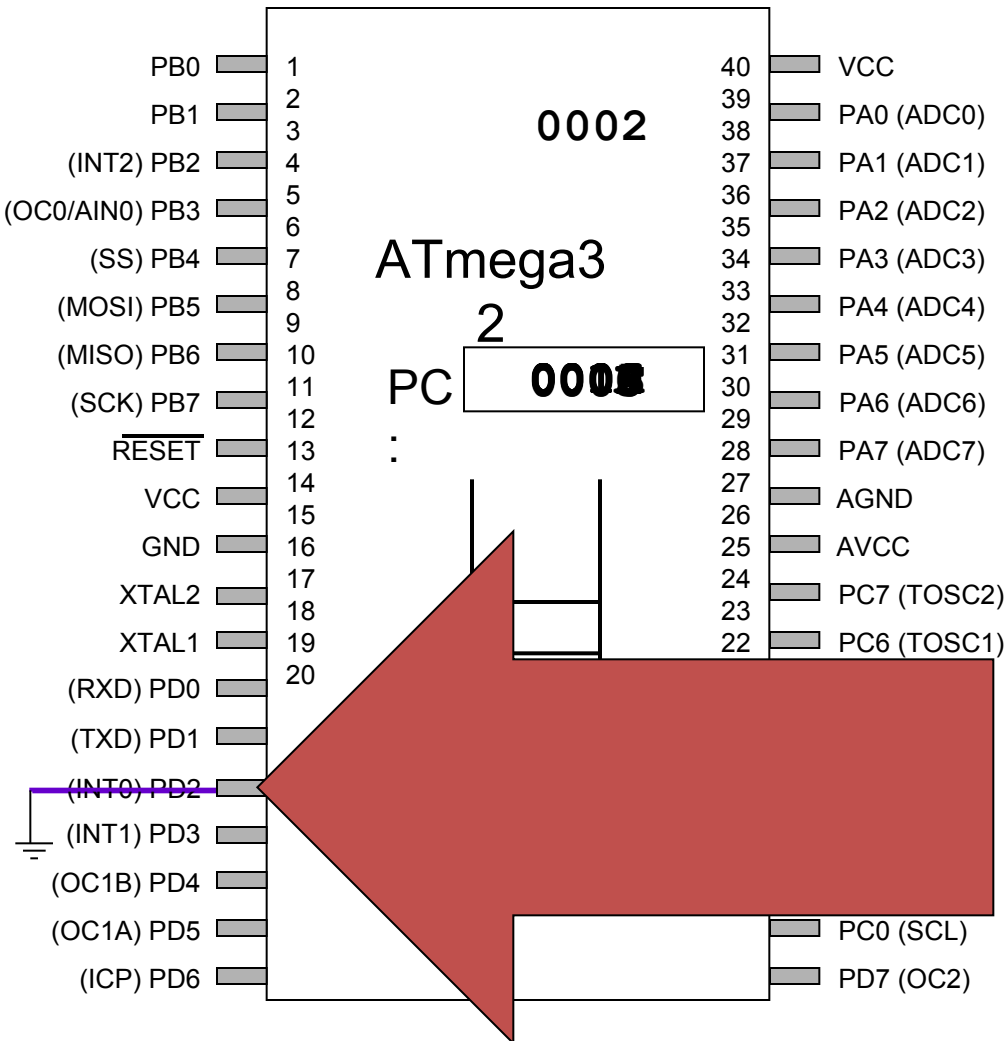
Alternate Functions of Port D The Port D pins with alternate functions are shown in [Table 31](#).

Table 31. Port D Pins Alternate Functions

Port Pin	Alternate Function
PD7	OC2 (Timer/Counter2 Output Compare Match Output)
PD6	ICP1 (Timer/Counter1 Input Capture Pin)
PD5	OC1A (Timer/Counter1 Output Compare A Match Output)
PD4	OC1B (Timer/Counter1 Output Compare B Match Output)
PD3	INT1 (External Interrupt 1 Input)
PD2	INT0 (External Interrupt 0 Input)
PD1	TXD (USART Output Pin)
PD0	RXD (USART Input Pin)

The alternate pin configuration is as follows:

Steps in executing an interrupt



Interrupt Example Code

- Several steps are necessary on the ATmega to enable interrupts!
 - The control register that is responsible for the interrupt you desire must be configured to generate an IRQ upon an event!
 - An ISR must be defined!
 - Global Interrupts must be enabled!

Software example to interrupt on an external pin!

```
#include <io.h>
#include <signal.h>
#include
<interrupt.h>

▪ SIGNAL(SIG_INTERRUPT0)    /* signal handler for external interrupt int0 */
{
    /* Do whatever you want to have happen during the interrupt here */
}

int main(void)
{
    DDRB = 0x00; /* use Port D for input*/
    EIMSK = (1<<INT0); /* Enable external interrupt 0 */
    sei(); /* enable interrupts */

    for (;;) { /*Your main loop here */ }
}
```

External Interrupts Int0 and Int1:

- Int0 connected to PD2
- Int1 connected to PD3
- asynchronous operation: can wake up CPU
- rising/falling edge or low level can trigger interrupt, defined by Interrupt Sense control – bits of MCUCR SFU

Bit	7	6	5	4	3	2	1	0	
	SE	SM2	SM1	SM0	ISC11	ISC10	ISC01	ISC00	MCUCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ISC11	ISC10	Description
0	0	The low level of INT1 generates an interrupt request.
0	1	Any logical change on INT1 generates an interrupt request.
1	0	The falling edge of INT1 generates an interrupt request.
1	1	The rising edge of INT1 generates an interrupt request.

External Interrupts Int0 and Int1:

- **Int0 and Int1 have to be enabled by the GICR (+ I-bit in SREG)**

Bit	7	6	5	4	3	2	1	0
	INT1	INT0	-	-	-	-	IVSEL	IVCE
Read/Write	R/W	R/W	R	R	R	R	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

- **flagged interrupts: General Interrupt Flag Register (GIFR) indicates when an interrupt request happened**
- **flags are cleared by executing the interrupt service routine (ISR) or by writing 1 to the flag bit of GIFR**

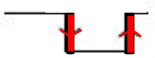
[illegible]

Edge trigger Vs. Level trigger in external interrupts

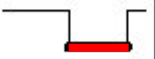
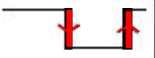
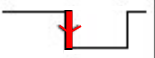
MCUCR	SE	SM2	SM1	SM0	ISC11	ISC10	ISC01	ISC00
-------	----	-----	-----	-----	-------	-------	-------	-------

ISC01, ISC00 (Interrupt Sense Control bits) These bits define the level or edge that activates the interrupt, as shown in the following table.

```
LDI R20,0x02 ;falling
OUT MCUCR,R20
```

ISC01	ISC00		
0	0		The low level of INT0 generates an interrupt request.
0	1		Any logical change on INT0 generates an interrupt request.
1	0		The falling edge of INT0 generates an interrupt request.
1	1		The rising edge of INT0 generates an interrupt request.

ISC11, ISC10 These bits define the level or edge that activates the INT1 pin.

ISC11	ISC10		Description
0	0		The low level of INT1 generates an interrupt request.
0	1		Any logical change on INT1 generates an interrupt request.
1	0		The falling edge of INT1 generates an interrupt request.
1	1		The rising edge of INT1 generates an interrupt request.

Edge trigger Vs. Level trigger (Cont.)

MCUCSR

JTD	ISC2	-	JTRF	WDRF	BORF	EXTRF	PORF
-----	------	---	------	------	------	-------	------

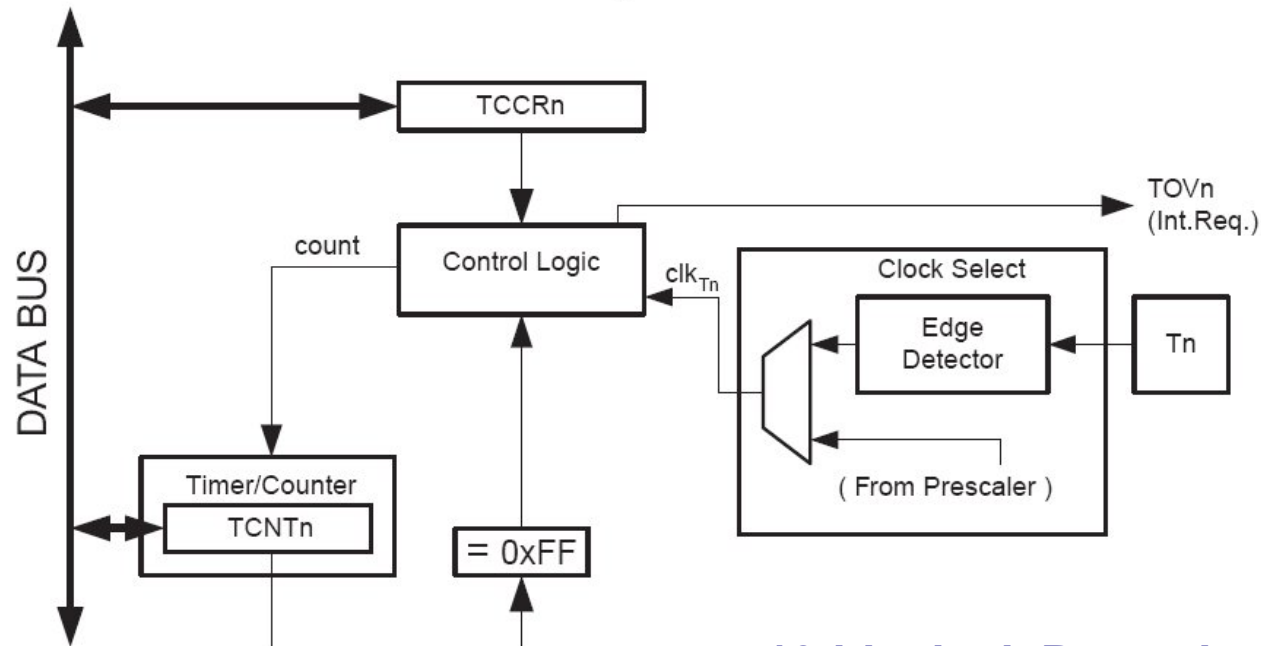
ISC2 This bit defines whether the INT2 interrupt activates on the falling edge or the rising edge.

ISC2		Description
0		The falling edge of INT2 generates an interrupt request.
1		The rising edge of INT2 generates an interrupt request.

Timers Interrupts

- Timers are counters that are incremented on a clock tick, or after a certain number of clock ticks!
 - Clock Sources, availability dependent on peripheral!
 - Main Oscillator (16MHz / 8MHz)!
 - Watch Crystal (32.768KHz)!
 - External Clock!
 - Often Scaled internally!
 - Factors of 1, 8, 64, 256, 1024 on AVR!
- Interrupts, flags, and pin state changes can be generated on timer overflow or compare!
 - High resolution timers by comparing with a target value! ▪
 - Long running timers by counting overflows! ▪ Pulse-Width Modulation!

8-bit Timer / Counter0 for AVR 8

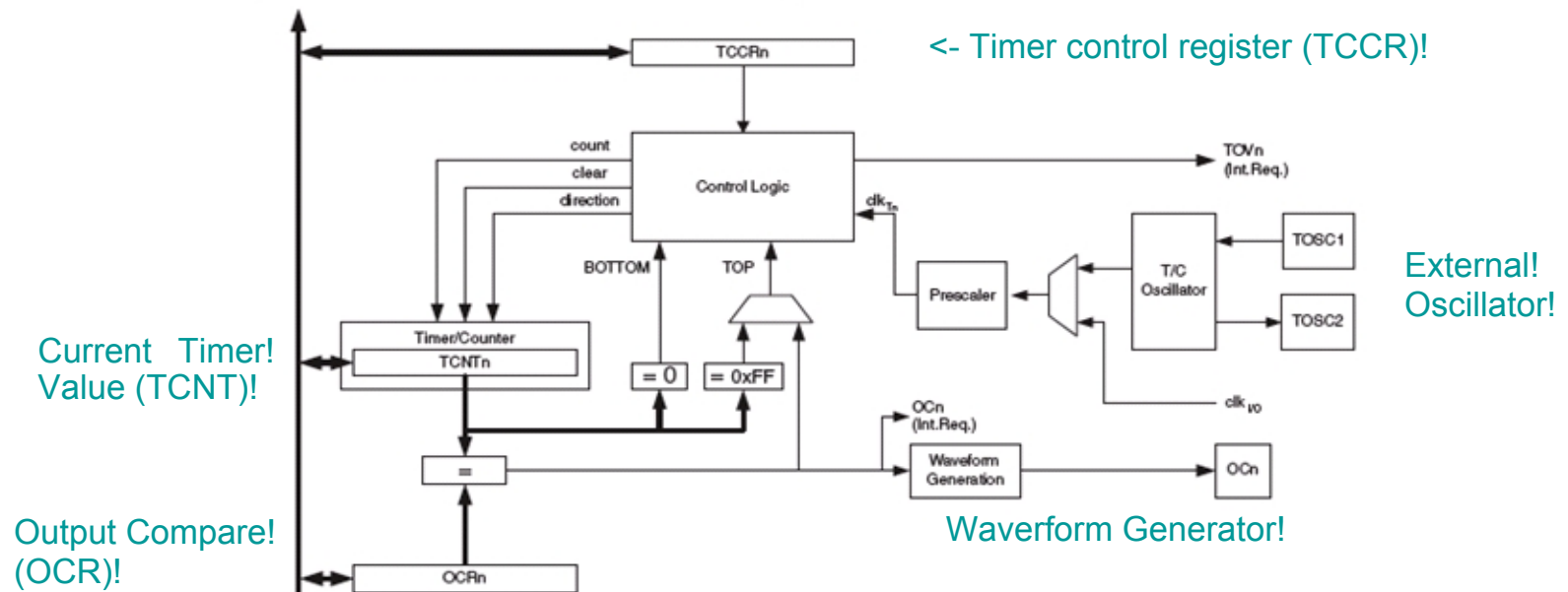


- 10-bit clock Prescaler
timer-clk (t0) = clk (IO) / prescaler
- External clock source T0 connected to PD4
cannot be prescaled, $\text{clk}(\text{ext}) \leq \text{clk}(\text{IO}) / 2.5$

8-bit Timer Peripheral

Sample 8-bit Timer on the ATmega128!

Figure 34. 8-bit Timer/Counter Block Diagram

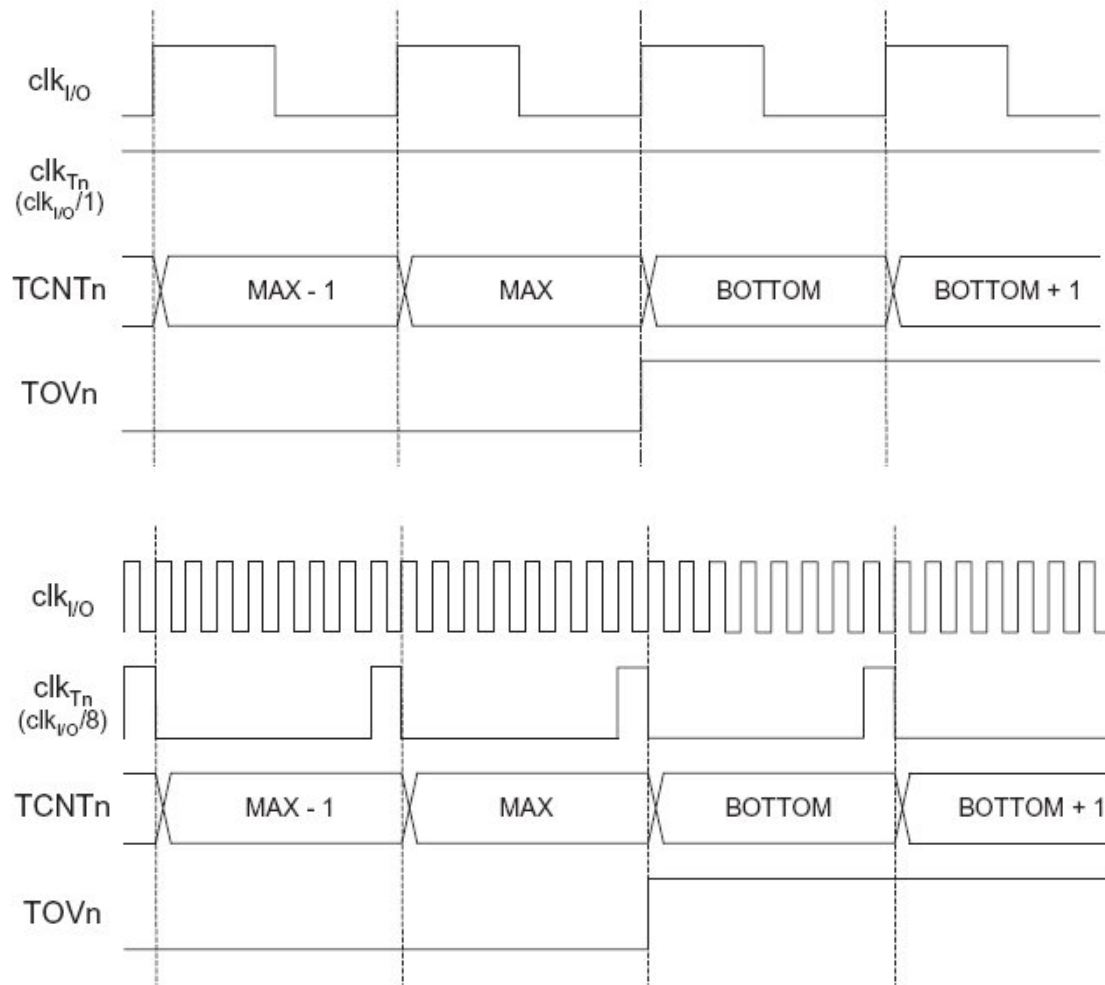


8-bit Timer / Counter0 - prescaler operation

No prescaler

MAX=0xff
BOTTOM=0

Prescaler = 8



8-bit Timer / Counter0 usage

Timer/Counter0 Control Register (TCCR0), Bits CS02-CS00 select Clock Source and Prescaler Value :

Bit	7	6	5	4	3	2	1	0	
	–	–	–	–	–	CS02	CS01	CS00	TCCR0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$\text{clk}_{I/O}/(\text{No prescaling})$
0	1	0	$\text{clk}_{I/O}/8$ (From prescaler)
0	1	1	$\text{clk}_{I/O}/64$ (From prescaler)
1	0	0	$\text{clk}_{I/O}/256$ (From prescaler)
1	0	1	$\text{clk}_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

8-bit Timer / Counter0 usage

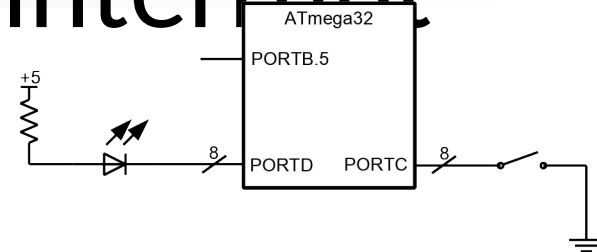
Timer/Counter0 Register (TCNT0) :
read/write, incremented per CLK cycle, overflow: 0xff

Bit	7	6	5	4	3	2	1	0	
	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- A Reload-Value is used to fine-tune the interrupt interval
- write Reload-Value to TCNT0 in the ISR

Using Timer0 overflow interrupt

- This program uses Timer0 to generate a square wave on pin PORTB.5, while at the same time data is being transferred from PORTC to PORTD.



```

1 ;Program 10-1
2 .INCLUDE "M32DEF.INC"
3 .ORG 0x0 ;location for reset
4 JMP MAIN
5 .ORG 0x16 ;loc. for Timer0 over.
6 JMP T0_OV_ISR
7 ;----main program for initialization
8 .ORG 0x100
9 MAIN: LDI R20,HIGH(RAMEND)
10 OUT SPH,R20
11 LDI R20,LOW(RAMEND)
12 OUT SPL,R20
13 SBI DDRB,5 ;output
14 LDI R20,0
15 OUT DDRC, R20
16 LDI R20,0xFF
17 OUT DDRD, R20
    
```

Timer int.
init.

```

20 LDI R20,(1<<TOIE0)
21 OUT TIMSK,R20
22 SEI
    
```

Timer init.

```

23 LDI R20,-32 ;value for 4μs
24 OUT TCNT0,R20
25 LDI R20,0x01
26 OUT TCCR0,R20
    
```

```

27 HERE: IN R20,PINC
28 OUT PORTD,R20
29 JMP HERE
    
```

-----ISR for Timer 0

```

30 T0_OV_ISR:
31 IN R16,PORTB
32 LDI R17,0x20
33 EOR R16,R17
34 OUT PORTB,R16
35 RETI
    
```

Timer0 compare match interrupt

- using Timer0 and CTC mode generate a square wave on pin PC
- data is being transferred from PORTC to PORTD.

```
.INCLUDE "M32DEF.INC"
.ORG 0x0000 ;location for reset
    JMP     MAIN

.ORG 0x14    ;location for Timer0 compare match
    JMP     T0_CM_ISR
```

; -main program for initialization and keeping CPU busy

```
.ORG 0x100
MAIN:  LDI     R20, HIGH(RAMEND)
        OUT     SPH, R20
        LDI     R20, LOW(RAMEND)
        OUT     SPL, R20
```

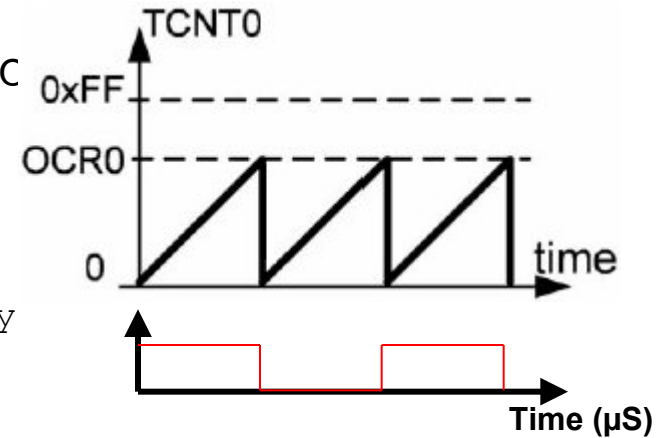
```
LDI     R20, 39
OUT     OCR0, R20    ;OCR0 = 39
LDI     R20, 0x09
OUT     TCCR0, R20    ;Start Timer0
```

```
SBI     DDRB, 5      ;PB5 as an output
```

```
LDI     R20, (1<<OCIE0) ;Timer0 compare match
OUT     TIMSK, R20
SEI                                     ;Set I
```

```
LDI     R20, 0x00
OUT     DDRC, R20    ;make PORTC input
LDI     R20, 0xFF
```

```
OUT     PORTD, R20    ;make PORTD output
```



```
HERE:   IN      R20, PINC
        OUT     PORTD, R20
        JMP     HERE
```

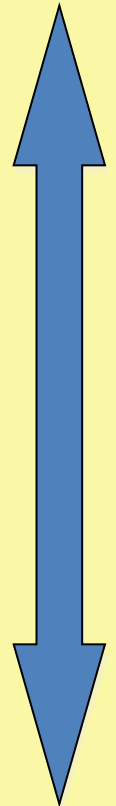
```
;-----ISR for Timer 0
T0_CM_ISR:
    IN      R16, PORTB
    LDI     R17, 0x20
    EOR     R16, R17
    OUT     PORTB, R16
    RETI
```

Interrupt priority

Table 10-1: Interrupt Vector Table for the Mega32

Interrupt	ROM Location (Hex)
Reset	0000
External Interrupt request 0	0002
External Interrupt request 1	0004
External Interrupt request 2	0006
Time/Counter2 Compare Match	0008
Time/Counter2 Overflow	000A
Time/Counter1 Capture Event	000C
Time/Counter1 Compare Match A	000E
Time/Counter1 Compare Match B	0010
Time/Counter1 Overflow	0012
Time/Counter0 Compare Match	0014
Time/Counter0 Overflow	0016
SPI Transfer complete	0018
USART, Receive complete	001A
USART, Data Register Empty	001C
USART, Transmit Complete	001E
ADC Conversion complete	0020
EEPROM ready	0022
Analog Comparator	0024
Two-wire Serial Interface	0026
Store Program Memory Ready	0028

Highest
priority



Lowest
priority

Interrupt inside an interrupt

- The I flag is cleared when the AVR begins to execute an ISR. So, interrupts are disabled.
- The I flag is set when RETI is executed.

Task switching and resource conflict

- Does the following program work?

```
1 .INCLUDE "M32DEF.INC"
2 .ORG 0x0 ;location for reset
3 JMP MAIN
4 .ORG 0x14 ;Timer0 compare match
5 JMP T0_CM_ISR
6 ;-----main program-----
7 .ORG 0x100
8 MAIN: LDI R20,HIGH(RAMEND)
9 OUT SPH,R20
10 LDI R20,LOW(RAMEND)
11 OUT SPL,R20 ;set up stack
12 SBI DDRB,5 ;PB5 = output
13 LDI R20,160
14 OUT OCR0,R20
15 LDI R20,0x09
16 OUT TCCR0,R20
```

```
17 LDI R20,(1<<OCIE0)
18 OUT TIMSK,R20
19 SEI
20 LDI R20,0xFF
21 OUT DDRC,R20
22 OUT DDRD,R20
23 LDI R20,0
24 HERE: OUT PORTC,R20
25 INC R20
26 JMP HERE
27 ;-----ISR for Timer0
28 T0_CM_ISR:
29 IN R20,PIND
30 INC R20
31 OUT PORTD,R20
32 RETI
```

Solution 1: different registers

- Use different registers for different tasks.

```
1 .INCLUDE "M32DEF.INC"
2 .ORG 0x0 ;location for reset
3 JMP MAIN
4 .ORG 0x14 ;Timer0 compare match
5 JMP T0_CM_ISR
6 ;-----main program-----
7 .ORG 0x100
8 MAIN: LDI R20,HIGH(RAMEND)
9 OUT SPH,R20
10 LDI R20,LOW(RAMEND)
11 OUT SPL,R20 ;set up stack
12 SBI DDRB,5 ;PB5 = output
13 LDI R20,160
14 OUT OCR0,R20
15 LDI R20,0x09
16 OUT TCCR0,R20
17 LDI R20,(1<<OCIE0)
18 OUT TIMSK,R20
19 SEI
20 LDI R20,0xFF
21 OUT DDRC,R20
22 OUT DDRD,R20
23 LDI R20,0
24 HERE: OUT PORTC,R20
25 INC R20
26 JMP HERE
27 ;-----ISR for Timer0
28 T0_CM_ISR:
29 IN R21,PIND
30 INC R21
31 OUT PORTD,R21
32 RETI
```

Solution 2: Context saving

- Save the contents of registers on the stack before execution of each task, and reload the registers at the end of the task.

```

1 .INCLUDE "M32DEF.INC"
2 .ORG 0x0 ;location for reset
3 JMP MAIN
4 .ORG 0x14 ;Timer0 compare match
5 JMP T0_CM_ISR
6 ;-----main program-----
7 .ORG 0x100
8 MAIN: LDI R20,HIGH(RAMEND)
9 OUT SPH,R20
10 LDI R20,LOW(RAMEND)
11 OUT SPL,R20 ;set up stack
12 SBI DDRB,5 ;PB5 = output
13 LDI R20,160
14 OUT OCR0,R20
15 LDI R20,0x09
16 OUT TCCR0,R20
17 LDI R20,(1<<OCIE0)

```

```

18 OUT TIMSK,R20
19 SEI
20 LDI R20,0xFF
21 OUT DDRC,R20
22 OUT DDRD,R20
23 LDI R20,0
24 HERE: OUT PORTC,R20
25 INC R20
26 JMP HERE
27 ;-----ISR for Timer0
28 T0_CM_ISR:
29     PUSH R20 ;save R20
30     IN R20,PIND
31     INC R20
32     OUT PORTD,R20
33     POP R20 ;restore R20
34     RETI

```

Saving SREG

- We should save SREG, when we change flags in the ISR.

```
PUSH    R20
IN       R20,SREG
PUSH    R20
...
POP      R20
OUT      SREG,R20
POP      R20
```


C programming

- Using Timer0 generate a square wave on pin PORTB.5, while at the same time transferring data from PORTC to PORTD.

```
#include "avr/io.h"
#include "avr/interrupt.h"
int main ()
{
    DDRB |= 0x20;
    TCNT0 = -32;
    TCCR0 = 0x01;
    TIMSK = (1<<TOIF0);
    sei ();
    DDRC = 0x00;
    DDRD = 0xFF;
    while (1)
        PORTD = PINC;
}
ISR (TIMER0_OVF_vect)
{
    TCNT0 = -32;
    PORTB ^= 0x20;
}
```

Table 10-3: Interrupt Vector Name for the ATmega32/ATmega16 in WinAVR

Interrupt	Vector Name in WinAVR
External Interrupt request 0	INT0_vect
External Interrupt request 1	INT1_vect
External Interrupt request 2	INT2_vect
Time/Counter2 Compare Match	TIMER2_COMP_vect
Time/Counter2 Overflow	TIMER2_OVF_vect
Time/Counter1 Capture Event	TIMER1_CAPT_vect
Time/Counter1 Compare Match A	TIMER1_COMPA_vect
Time/Counter1 Compare Match B	TIMER1_COMPB_vect
Time/Counter1 Overflow	TIMER1_OVF_vect
Time/Counter0 Compare Match	TIMER0_COMP_vect
Time/Counter0 Overflow	TIMER0_OVF_vect
SPI Transfer complete	SPI_STC_vect
USART, Receive complete	USART0_RX_vect
USART, Data Register Empty	USART0_UDRE_vect
USART, Transmit Complete	USART0_TX_vect
ADC Conversion complete	ADC_vect
EEPROM ready	EE_RDY_vect
Analog Comparator	ANA_COMP_vect
Two-wire Serial Interface	TWI_vect
Store Program Memory Ready	SPM_RDY_vect

C programming Example 2

- Using Timer1 and CTC mode write a program that toggles pin PORTB.5 every second, while at the same time transferring data from PORTC to PORTD. Assume XTAL = 8 MHz.

```
#include "avr/io.h"
#include "avr/interrupt.h"

int main ()
{
    DDRB |= 0x20;           //make DDRB.5 output
    OCR0 = 40;
    TCCR0 = 0x09;           //CTC mode, internal clk, no prescaler
    TIMSK = (1<<OCIE0);    //enable Timer0 compare match int.
    sei ();                 //enable interrupts
    DDRC = 0x00;            //make PORTC input
    DDRD = 0xFF;            //make PORTD output
    while (1)               //wait here
        PORTD = PINC;
}

ISR (TIMER0_COMP_vect)     //ISR for Timer0 compare match
{
    PORTB ^= 0x20;         //toggle PORTB.5
}
```

C programming Example 3

- Assume that the INT0 pin is connected to a switch that is normally high. Write a program that toggles PORTC.3, whenever INT0 pin goes low. Use the external interrupt in level-triggered mode.

```
#include "avr/io.h"
#include "avr/interrupt.h"

int main ()
{
    DDRC = 1<<3;           //PC3 as an output
    PORTD = 1<<2;           //pull-up activated
    GICR = (1<<INT0);       //enable external interrupt 0
    sei ();                 //enable interrupts

    while (1);              //wait here
}

ISR (INT0_vect)             //ISR for external interrupt 0
{
    PORTC ^= (1<<3);        //toggle PORTC.3
}
```

Timer Overflow Example Code

Sample Timer Overflow C Code!

```
#include <io.h>
#include <interrupt.h>
#include <signal.h>

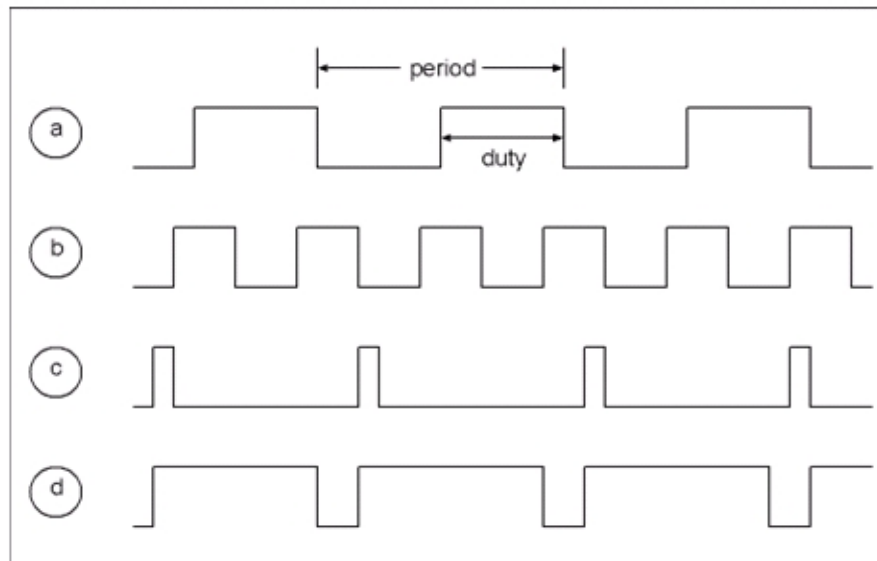
SIGNAL(SIG_OVERFLOW0)          /* ISR for TCNT0 overflow */
{
    /* DO YOUR WORK HERE */

    TCNT0 = 0;                  /* Reset TCNT0 to get the interrupt again */
}

void main(void)
{
    TCCR0 = 0x05;               /* TCNT0 is clocked with cpu/1024 clock */
    TIMSK = 1<<TOIE0;          /* Timer Overflow Interrupt Enable on TCNT0 */
    TCNT0 = 0;                  /* Reset TCNT0 */
    sei();                      /* Enable interrupts */
    for (;;) {
        /* Main loop */
    }
}
```

Pulse Width Modulation

- Vary the duty cycle of a fixed period “square” wave to vary output power!
 - DC Motor Speed Control, LED Brightness, Servo Positioning, “Poor-man’s DAC”!
- Waveform Generator / Output Compare module!
 - Coupled with timer peripheral!

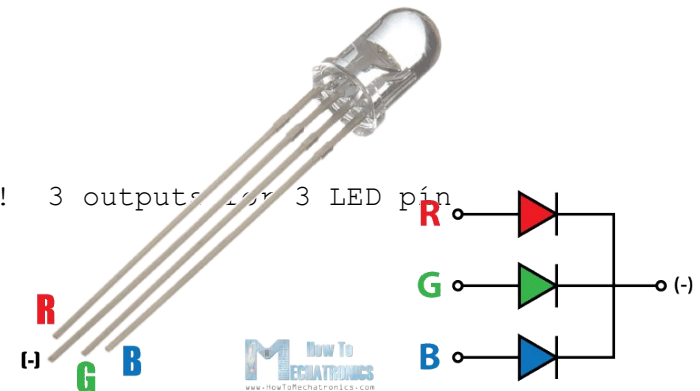


<http://www.ece.utep.edu/courses/web3376/concepts/pwm.html>!

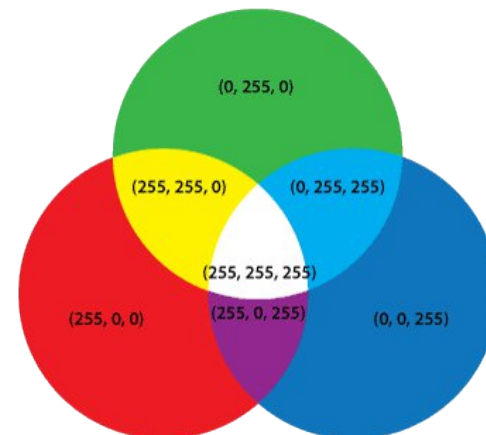
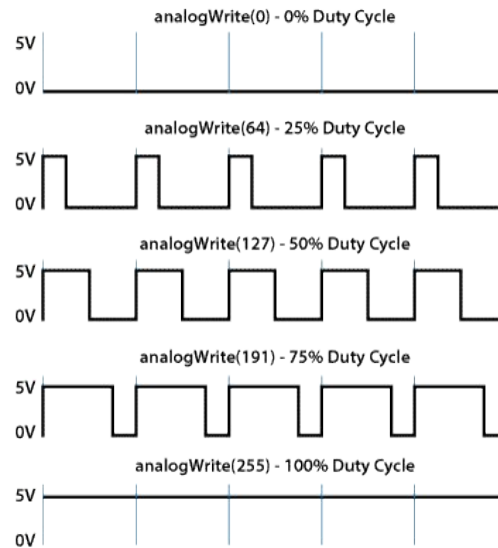
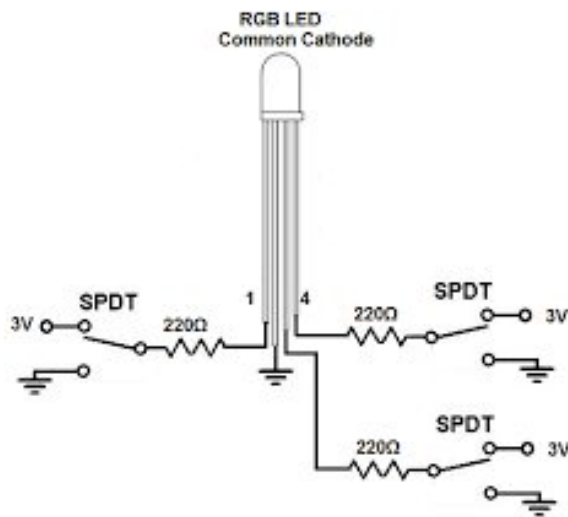
PWM Example Code (1/2)

```
#include <io.h>!  
#include <interrupt.h>!  
#include <signal.h>!  
  
void main(void)!  
{!  
    !DDRL=0x38;!  
    !PORTL=PORTL | 0x38; //start off with the tri-color LED off! 3 outputs for 3 LED pin  
    !for (;;) {!  
        !    /* Main loop */!  
    }!  
}!
```

Tri-Color LED



PWM - Pulse Width Modulation



PWM Example Code (2/2)

```
void ColorCycle( void )!
{!
  //!First, set up the timer/counter #5 for the color LED control! !TCCR5A
  = (1 << COM5A1) | (1 << COM5B1) | (1 << COM5C1) | (1 << WGM50);!
  !    TCCR5B = (1 << CS52) | (1 << CS50) | (1 << WGM52); // clk/1024, PWM (mode 5 in table 82 of
  datasheet)! !//Start with just green!
  !OCR5A = 255;!
  !OCR5B =
  255;! !OCR5C =
  0;! !sei();!
  !while
  ( 1 )! !{!
    !    !for (OCR5A = 255; OCR5A>1; OCR5A--)!
    !    !    !DelayMS();!
    !    !for (OCR5C = 0; OCR5C<255; OCR5C++)!
    !    !    !DelayMS();!
    !    !for (OCR5B = 255; OCR5B>1; OCR5B--)!
    !    !    !DelayMS();!
    !    !for (OCR5A = 0; OCR5A<255; OCR5A++)!
    !    !    !DelayMS();!
    !    !for (OCR5C = 255; OCR5C>1; OCR5C--)!
    !    !    !DelayMS();!
    !    !for (OCR5B = 0; OCR5B<255; OCR5B++)!
    !    !    !DelayMS();!
  !}!
}!
```

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references